

エージェント・クオンツトレーディング

Agentic Quant Trading

with Python

HMMレジーム検出からSAC強化学習まで

ブローカー連携まで扱う実践クオンツシステム



forward-only HMM ・ Kalman spread ・ SAC agent ・ IBKR/KIS

Ted Park

依頼 | 実装 | 検証 | 反復

Agentic Quant Trading with Python

リクエスト・ビルド・検証—AIエージェントで株式取引システムを完成させる

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Hiragino Kaku Gothic ProN | Menlo.

目次

まえがき	1
この章の流れ	1
学習目標	1
チャートに騙された日	2
テクニカル分析の現実	2
ルネサンス・テクノロジーズが見抜いたもの	3
市場を見るもう一つの方法	3
なぜ本書を書いたか	4
AI コーディングアシスタントが可能にしたこと	4
本書を機械学習トレーディングの学習書として読む	5
迷ったときに戻る地図	6
1日を追う全体例	7
モデルをもう少し分解して考える	8
本書が扱うものと扱わないもの	9
本書を読み終えたときに作れるもの	10
ルネサンスの事例から取り出した原則	10
1. 因果性の原則	11
2. アンサンブルの原則	11
3. ウォークフォワード検証の原則	11
これからの道のり	11
参考:バックテストの数字をどう読むか	12
全章ロードマップ	13
回ごとに読み方を変える	14
Partごとの最低理解目標	15

AI をペアプログラマとして使う	15
この章で必ず覚えておくこと	16
前提知識	16
第0章 バイブコーディング・クイックスタート：まず小さな例を検証する	17
最初に期待する結果を決める	18
本文だけで実行できる確認コード	18
AIへの依頼方法	19
時間の順序を確認する	19
失敗の原因を絞り込む	20
次へ進む前の確認	20
第1章 システムアーキテクチャ — AIクオンツトレーディングの設計思想	21
この章の流れ	22
学習目標	23
1.1 システム構造	23
なぜレイヤーに分けるのか	24
機械学習の視点で読み直す	25
ここで言うモデルとは何か	26
すべての章を同じ型で読む	27
1.2 ディレクトリ構造	28
1.3 コア構成要素	28
SignalGenerator — 共通の約束	28
SIGNAL_REGISTRY — 登録された戦略一覧	29
BacktestResult — 標準化された結果フォーマット	29
SAC — 自前で実装した強化学習エージェント	30
1.4 因果性の原則	31
1.5 データの流れ — バックテストはどんな順番で回るか	32
1.6 開発環境のセットアップ	32
Python 3.12環境	32
主要パッケージ	33
設定ファイル	33
1.7 データキャッシュの初期化	34

1.8 最初のバックテストを実行する	35
Exercises	36
まとめ	36
さらに読むなら	37
理解を確認する	37
第2章 データパイプライン—DuckDBとリアルタイムフィード	38
この章の流れ	39
学習目標	39
2.1 全体像—Data Flow	40
原データが特徴量になるまで	41
2.2 download_bars() —単一のダウンロード窓口	42
関数がサポートするインターバル	42
中核フロー—2段階のフォールバック	43
フォールバックが実際に発生したかを確認する	44
2.3 DuckDBCache —Parquet + メモリ上DuckDB	44
なぜDuckDBか	44
保存構造—月別Parquet	45
同時アクセスをどう扱うか	45
保存関数の4ステップ	46
2.4 tz_localize(None) —このプロジェクトの暗黙のルール	46
正しいパターン	47
2.5 特徴量エンジニアリング—4次元の日足特徴量	47
4つの特徴量が見ているもの	47
中核となる計算—次元0 (対数リターン)	48
欠損値—モデルに渡してはいけない空欄	49
2.6 カルマン平滑化—ノイズを抑える一行の補助ツール	50
2.7 時間足の特徴量—6次元への拡張	50
2.8 Zスコア—平均回帰のシグナル	51
2.9 モデル入力へ渡す—実戦パターン	52
Exercises	53
まとめ	54

さらに読むには	54
第3章. バックテストエンジン—信頼できるシミュレーション	55
この章の流れ	56
学習目標	56
3.1 signal・約定・資産の時間順序	57
3.2 実行できる損益会計の実習	57
3.3 費用と最終清算を一度だけ数える	60
3.4 パフォーマンス指標— <code>_build_result()</code>	60
リスク調整後リターン(シャープレシオ)	61
MDDとWin Rate	61
さらに踏み込んだ指標(カタログ)	61
3.5 PnL Gini —集中度の指標	62
3.6 DSR(Deflated Sharpe Ratio) —多重検定の補正	62
3.7 PortfolioConfig —戦略別の推奨設定	63
3.8 パフォーマンス指標の読み方	64
Exercises	64
まとめ	65
さらに読むには	65
第4章 HMMの基礎—市場レジームを確率で推論する	67
HMMを機械学習モデルとして見る	68
同じシグナルでもレジームが違えば意味が変わる	70
この章の流れ	71
学習目標	71
最初の通読で押さえること	72
4.0 全体像— <code>hmm_daily_mr.py</code> パイプライン	73
4.1 HMMの3要素— A , B , π	74
遷移行列 A	74
放出分布 B	74
初期状態分布 π	74
4.2 Baum-Welch —学習の直感	75
4.3 <code>_DailyHMM.fit()</code> —実装	76

covariance_type="full" を使う理由	78
min_covar=1e-3 の役割	78
4.4 分散ベースのレジーム整列	79
4.5 _DailyHMM.predict_proba() —レジーム確率の計算	80
4.6 _update_hmm() —10本ごとの再学習	82
4.7 _check_entry() —Z-score + HMMゲート	83
4.8 _check_exit() —6つの手仕舞い条件	84
4.9 PureZscoreDailySignal —ベースライン	85
4.10 全体の流れ—on_bar() の呼び出し順	86
演習問題	86
この章で必ず覚えること	87
さらに読むなら	88
理解を確認する	88

第5章. HMM Forward-Only —未来情報を使わないリアルタイム推論 89

Forward-onlyを一つの場面で見る	92
この章の流れ	94
学習目標	94
5.0 全体像—hmm_core.py パイプライン	95
5.1 なぜ predict_proba がルックアヘッドバイアスを起こすか	96
5.2 Forwardアルゴリズムの再帰式	97
5.3 _BWHMM.forward_proba_causal() —学習後の α 初期化	98
5.4 _BWHMM.step_forward() —1ステップのオンライン推論	99
訓練区間とOOS区間をつなぐ流れ	101
5.5 _precompute_cov_fast() —速度最適化	101
5.6 7-Stateのレジーム構造—REGIME_A_SCALE	102
5.7 GlobalRegimeLayer.step() —単一の乗数にまとめる	102
5.8 dominant_state —3本連続確認による安定化	103
5.9 Walk-Forward の利用例	104
5.10 遷移risk	105
演習問題	105

この章で必ず覚えること	106
さらに読むには	107
第6章. カルマンフィルターノイズ除去とイノベーションZスコア	108
この章の流れ	111
学習目標	111
6.0 全体像—カルマンフィルタの2つの役割	111
6.1 ローカルレベルモデル—直感的な数式	112
6.2 KalmanFilter1D —HMM特徴量の平滑化	114
6.3 4次元の特徴量ごとのカルマンパラメータ	115
6.4 _kalman_innovations() —価格からInnovationを取り出す	116
6.5 _build_z_series() —因果的なZスコア	117
6.6 _calc_half-life() —AR(1)半減期推定	118
6.7 KalmanSelfMRSigal —エントリーと手仕舞いのロジック	118
エントリーの4重フィルタ	119
手仕舞いの6重チェック	120
6.8 kalman_self_mr vs hmm_daily_mr の比較	121
6.9 バックテスト実行の例	122
演習問題	122
この章で必ず覚えておくこと	123
さらに読むには	124
第7章 クロスアセットJoint HMM —複数市場を同時に見るレジーム検出	125
この章の流れ	127
学習目標	128
7.0 全体パイプライン—6段階	128
7.1 Why Cross-Asset?	129
7.2 bar単位のノイズ除去—Kalmanフィルタ	130
7.3 20次元のセッション観測ベクトル	131
7.4 Layer 1 —Macro HMM(日足)	132
7.5 Layer 2 —Micro HMM(セッション)	133
7.6 Walk-forward構造	133

7.7 macro_bias -> Kelly変調	134
7.8 P(up) の計算—上昇確率はどう作られるか	134
7.9 結果を生んだ構造的な要因	135
コードで見る—合成データでMacro HMMの学習の流れを確認する	135
Exercises	137
まとめ	137
さらに読みたい人へ	138
8章. 2階層HMMアンサンブル—Layer A × Layer B	139
この章の流れ	141
学習目標	141
8.1 Why Two Layers?	142
8.2 効果から確認—バックテスト性能	143
8.3 Layer A: GlobalRegimeLayer	143
8.3.1 レジーム安定化—3バー連続確認	144
8.4 Layer B: 8個のサブモデル概要	145
8.4.1 HMMサブモデル(代表2個)	145
8.4.2 非HMMサブモデル(代表2個)	146
8.5 SubModelSpec —サブモデルをデータとして宣言する	146
8.6 HMMForwardPredictor: Layer B HMM Predictor	147
8.6.1 fit() —学習段階がやること	147
8.6.2 step() —新バーごとに期待リターンを更新	148
8.7 W2 Wasserstein Voting —外れ値の抑制	148
8.8 RenaissanceEnsemble5m.step() —二レイヤーの結合	149
8.9 設計原則のまとめ	150
コードで見る—Two-Layer HMM の最小実装	151
Exercises	152
8.10 この章で必ず覚えておくこと	152
参考文献	154
第9章 ハーフ・ケリーによるポジションサイジング	155
この章の流れ	157

学習目標	158
9.1 ケリー基準の基本	158
9.2 calculate_kelly_size() の構造	159
9.3 vol_mult —ボラティリティが高まったらポジションを減らす	161
9.4 macro_gate —市場状態が紛らわしければ縮める	162
9.5 novelty_gate —見たことのない市場なら縮める	162
9.6 ERアンプリファイア—シグナルがそろったときに増やす	163
9.7 資産クラスごとのマクロバイアス調整	164
9.8 fractional_kelly() —5分足向けの簡略版	165
9.9 2つのケリー実装の比較	165
9.10 KELLY_MIN = 0.10 に設定した理由	166
シナリオで見る	167
演習	167
まとめ	168
さらに読むには	168
理解を確認する	169
第10章. HMM フォワードアンサンブル—V1 -> V3 への進化	170
なぜアンサンブルなのか	171
この章の流れ	172
学習目標	172
初回読破で押さえること	173
10.1 アーキテクチャ比較—V1, V2, V3	173
10.2 V1/V2 共通コア—Forward α で $E[r]$ を計算する	174
10.2.1 因果性 (causality) の落とし穴を避ける	175
10.3 バージョンごとにどんな情報を足したのか	176
V2 で追加—セクター・オーダーフロー	176
V3 で追加—マルチタイムフレーム & 非-HMM	176
V3 の動的な重み—成果の良いサブモデルにより大きな比重	177
10.4 V2 の中心アイデア—Optuna による閾値の再最適化	178
10.5 V2/V3 成果比較のまとめ	179
10.6 適応的な閾値— $\sigma \times \text{scale}$	179

やってみる—Optuna で閾値最適化(オプション)	180
練習問題	181
まとめ	181
さらに読むなら	182
第11章 統計的裁定取引—ペア選定	183
この章の流れ	185
学習目標	185
11.1 なぜセクターの中でペアを選ぶのか	186
11.2 ペア選定パイプライン	187
共和分検定—1 ペアを判定する	188
パイプライン全体—セクター別に全ペアを検定	188
11.3 半減期(half-life)の計算	190
11.4 Kalman フィルタで動的ヘッジ比率を求める	191
11.5 ペアの品質指標	191
11.6 ペア選定パラメータを一望する	192
11.7 ペア選別基準のまとめ	193
試してみよう—合成ペアでパイプラインを動かす	193
練習問題	195
まとめ	195
さらに読むには	196
第12章 統計的裁定取引—シグナル生成	197
エントリーよりエグジットが先である	198
この章の流れ	199
学習目標	199
12.1 エントリー資格(Entry Prerequisites)	200
12.2 Entry Score(entry_score)	201
なぜ掛け算なのか	202
数値の一例	202
収束モメンタム(conv_boost)	203
12.3 エグジット条件(優先順位順)	203
どの条件が先に効くか—二つのシナリオ	204

12.4 BW-HMM スプレッドレジーム補正	205
12.5 エントリー・エグジットフローまとめ	206
12.6 コードで見る—Entry Score 計算とエントリー資格判定	207
12.7 RL との組み合わせ方(次章の予告)	208
まとめ	209
練習問題	210
さらに読むには	211

まえがき

この章の流れ

このまえがきは、本書の方向性を最初に固めるためのものだ。チャート中心の判断がなぜ限界にぶつかるのかを見たうえで、本書がどのような方法でクオンツシステムを組み立てるのかを確認する。

- ・ チャートに騙された経験
- ・ テクニカル分析が今の市場で揺らいでいる理由
- ・ ルネサンス・テクノロジーズの事例から得られる中心的な視点
- ・ 本書が扱うものと扱わないもの
- ・ これから作るシステムと必要な準備

学習目標

このまえがきを読み終えたとき、次のことを説明できるようになっているはずだ。

- ・ なぜ本書はチャートパターンではなく、データパイプラインと検証から始めるのか

- ・「未来のデータをこっそり参照してしまう誤り (Look-ahead Bias)」が、なぜクオンツシステムにとって致命的なのか
- ・HMM、強化学習、統計的裁定取引が、本書のなかでそれぞれどんな役割を担うのか
- ・学習、推論、特徴量、欠損値といった機械学習の基本用語が、トレーディングシステムの中で何を意味するのか
- ・本書を読み始める前に、どの程度の Python と統計の知識が必要なのか

チャートに騙された日

ローソク足チャートを長いあいだ見つめて、買い時だと判断したことがある。同じ支持線で 3 度も反発しており、RSI は売られすぎ圏にあり、MACD のゴールデンクロスも教科書通りの形をしていた。複数のシグナルが同じ方向を指しているように見え、私はそのシグナルを信じて買いを入れた。

翌日、価格はその「完璧な」支持線をそのまま下抜けていった。初めて経験することでもなく、最後でもなかった。ただ、その日から私の問いは少し変わった。単に自分のチャートの読み方が下手だったのか、それともそもそも自分が頼っていた道具が、今の市場を説明するには不十分なのか。それを真剣に確かめる必要があった。

テクニカル分析の現実

それから数ヶ月のあいだ、論文、トレーディングフォーラム、成績データを調べながら同じ問いを繰り返した。結論から言えば、自分が頼りにしていた道具は今の市場を説明するには足りなかった。

伝統的なテクニカル分析とは、ローソク足パターン、移動平均のクロス、ボリンジャーバンド、RSI といった道具のことを指す。これらは市場の動きがゆっくりで、人間の心理が価格を強く揺さぶっていた時代に作られた。当時はたしかに機能していた。今は違う。

現在の市場では、アルゴリズムが人間よりもはるかに速く注文をさばき、取引量の大半をアルゴリズムが占めている。かつては群衆心理を反映していたパターンが、今では機械が先に検知し、先に取引する合図になってしまった。

チャート上に支持線が見えるのは人間だけではない。アルゴリズムも見ている。ストップロス注文が集まっている地点は流動性が積み上がっている地点であり、そこは意図的に狙われることが少なくない。古い市場でよく機能していた合図が、今の市場では同じようには働かないかもしれない、ということだ。

悲観的な話をしたいわけではない。個人投資家がよく使う道具の多くは、今とは違う市場を前提に作られたという事実を指摘しているだけだ。

だから本書は「もっとうまくチャートを読む方法」から始めない。代わりに、市場をデータを生み出す一つのプロセスとして見る。そのなかに潜む状態を推定し、その推定が未来のデータをこっそり参照していないかを検証する方向へ進めていく。

ルネサンス・テクノロジーズが見抜いたもの

このアプローチを最も早く、そして最も徹底して実践したのがルネサンス・テクノロジーズだ。

1988年、数学者のジム・サイモンズはルネサンス・テクノロジーズというヘッジファンドを立ち上げた。彼は株価チャートを読むトレーダーではなく、数学者、物理学者、コンピュータ科学者を採用した。狙いはローソク足の形を分析することではなかった。価格データのなかで繰り返される統計的なパターンを探し当てることだった。その成果であるメダリオン・ファンドは、数十年にわたり投資史上屈指の長期成績を残してきた。

市場を見るもう一つの方法

サイモンズが生まれつき特別な直観を備えていたわけではない。違いは、市場を見る方法にあった。

ルネサンスは、市場が一つの固定された法則で動いているとは考えなかった。代わりに、市場はいくつかの「隠れた状態(hidden state)」のあいだを行き来していると見た。隠れた状態とは、表には現れないが市場を支配している局面のことだ。ある時期は強いトレンドが続き、ある時期は上がった価格が平均に戻ろうとし、ある時期は急にボラティリティが膨らむ。

これらの状態を直接目で見ることはできない。だがデータから推定することはできる。今の市場がどの状態にあるのかを把握できれば、その状態に合った戦略を選べる。

HMM(隠れマルコフモデル、Hidden Markov Model)はまさにこの「見えない状態をデータから推定する」問題を扱う道具だ。本書の中盤で詳しく取り上げる。

私はルネサンス・テクノロジーズの内部関係者ではなく、その会社の正確な手法を知っているわけでもない。ただ公開されているインタビューや学術研究をもとに、その思想の一部をコードとして実装してみることはできた。本書はその過程の記録である。

なぜ本書を書いたか

クオンツトレーディングを学ぶのは難しい。一つひとつの概念が難しいからではなく、同時に揃えなければならないものが多すぎるからだ。統計、信号処理、Python の実装、バックテストエンジン、ブローカー連携、リスク管理。そのすべてが必要になる。

その中で最も厄介なのが、**システムが自分自身を騙さないように作ること**だ。

「Look-ahead Bias」という落とし穴がある。戦略を組むときに、未来のデータをこっそり参照してしまう誤りだ。コードのあちこちに潜んでいて、見つけるのが容易ではない。検証をきちんとしないと、未来を覗き見た戦略が、まるで素晴らしい成績を上げたかのように見えてしまう。市場平均を超える超過収益はふつう「アルファ (alpha)」と呼ばれるが、それが本当はアルファではなく、未来の情報を写しただけの結果ということもあり得る。

クオンツトレーディングを始めようとする人の大半は、この準備を整えられないまま諦める。そしてまたローソク足と RSI に戻っていく。それで十分だからではなく、手近で扱いやすいからだ。私はこのギャップを埋めたかった。複雑な数学と実装を隠さず、しかし読者がついていける順番でほどいていきたかった。

AI コーディングアシスタントが可能にしたこと

2023 年以降、AI コーディングアシスタントは実用的な水準に達した。そのおかげで本書を書くことができた。GitHub Copilot、Claude、ChatGPT のような道具は、複雑な

アルゴリズムを説明し、見つけにくい実装ミスを拾い上げ、数式がコードのなかでどんな意味を持つのかを解きほぐすのに役立つ。理解そのものを代わりにやってくれるわけではない。だが、理解にたどり着くまでの時間と試行錯誤は大きく減らしてくれる。

私は本書のシステムを AI と一緒に組み上げた。難しい数式に詰まったとき、見つけにくいバグを追ったとき、ハイパーパラメータ(モデルの性能を左右する設定値)を決めるとき、いずれも AI の力を借りた。本書はその過程をもう一度最初からたどり直す。理論の解説で止まらず、今手に入る道具で実際のシステムをどう作るのかを見せることが目標だ。

本書を機械学習トレーディングの学習書として読む

本書はHMMと強化学習を扱うが、より大きく見れば、機械学習と深層学習で株式トレーディングシステムを作るための学習書である。

基本の流れは単純だ。

株式市場
-> データ
-> 特徴量
-> 機械学習モデル
-> 深層学習 / RLエージェント
-> 検証
-> 注文実行

モデルは明日の価格を当てる魔法の箱ではない。過去データから内部のパラメータを学習した関数だと考えるとよい。普通の関数は人がルールを直接書く。機械学習モデルも入力と出力の形は同じだが、内部の重みをデータから学習する。

今日の特徴量1行 -> 学習済みモデル -> レジーム確率 / 行動提案

それでも**モデル**という言葉が曖昧に感じるなら、次のように考えるとよい。

モデル = データから内部の数値を学習した判断計算機

普通の計算機は2 + 3を入れると5を返す。トレーディングモデルは、リターン、ボラティリティ、出来高、Zスコアのような数値を受け取り、「平均回帰レジーム70%」や「許容ポジションの35%だけ使う」といった判断を返す。モデルはデータそのものではない。アルゴリズム名だけでもない。アルゴリズムがデータから生成した、学習済みの判断関数である。

```
def model(today_features):  
    return prediction
```

だから本書では、次の3つの言葉を分けて読む。

言葉	やさしい意味	本書での例
アルゴリズム	モデルを作る方法	HMMのBaum-Welch、SAC学習
パラメータ	モデル内部で学習される数値	HMMの遷移行列、ニューラルネットの重み
モデル	学習済みの判断関数	今日の特徴量 -> レジーム確率

料理にたとえるなら、データは材料、アルゴリズムはレシピ、パラメータは火加減と調味料の比率、モデルはその比率が決まった調理方法である。同じレシピでも、どんな材料で練習したかによって出来上がりは変わる。機械学習も同じだ。同じHMMアルゴリズムでも、どの市場期間で学習したかによって別のモデルになり得る。

学習は過去データでモデルを合わせること。推論は学習済みモデルを固定し、新しいデータを1行ずつ入れて判断を得ること。OOS検証は、後半のデータを学習時には隠しておき、学習済みモデルがそこでまだ通用するかを確かめる作業である。

最初からすべてのアルゴリズム名を理解しようとしなくてよい。まずは **データ -> 特徴量 -> 学習 -> 推論 -> 検証 -> 注文** という骨格だけをつかむ。HMM、SAC、リスク管理、MLflowはすべてこの骨格に接続される。

迷ったときに戻る地図

同じ内容をトレーディングの言葉に置き換えると次のようになる。

機械学習の言葉	本書でのトレーディング上の意味
データ	過去価格、出来高、マクロ指標、ブローカー記録
特徴量	モデルが読める数値。リターン、ボラティリティ、モメンタム、z-scoreなど
学習	過去データからモデル内部の数値を合わせること
推論	学習済みモデルに今日の新しい1行を入れて判断を得ること
OOS	学習時に隠しておく模擬試験区間
サービング	学習済みモデルをトレーディングエンジンから呼べるサービスとして動かすこと
リスクレイヤー	モデルの提案を最後に止めたり小さくしたりする安全装置

迷ったらこの表に戻ればよい。難しく見える章の多くは、この表の1行を詳しく扱っているだけである。2章はデータと特徴量、4〜5章はHMMの学習と推論、13〜14章はRLエージェント、16章はリスク、27章は学習済みモデルのサービングを扱う。

初回の通読に使うチェックリストは次のとおりである。

- [] モデルに入るデータを説明できるか。
 - [] そのデータから作る特徴量を説明できるか。
 - [] 学習データと推論データを分けられるか。
 - [] OOS区間がどこかを指せるか。
 - [] モデルが何を出力するかを言えるか。
 - [] 注文前にどのリスクチェックが入るかを言えるか。

この問いに答えられれば、HMM、SAC、Kelly、CVaRがまだ難しく感じても、本書の本当の構造は追えている。パイプラインが見えれば、細部はあとから理解しやすくなる。

1 日を追う全体例

具体例で見ると分かりやすい。5年分の日足データがあるとしよう。初心者向けの分け方なら、最初の2年でモデルを学習し、残り3年で推論と検証を行う。後半3年もディ

スク上では過去データだが、検証中は未来データのように扱わなければならない。モデルはそこに合わせ込んではいけない。1行ずつ特徴量を受け取り、出力だけを返す。

5年分の特徴量

最初の2年：学習データ

- > スケーラーを合わせる
- > モデルを学習する
- > モデルパラメータを保存する

次の3年：OOS推論データ

- > 学習済みモデルをロードする
- > 1行ずつ特徴量を入れる
- > モデル出力を記録する
- > リスクチェックを通す

モデルをもう少し分解して考える

モデルファイルが何を保存しているのかを理解するにも、この例が分かりやすい。モデルファイルは学習データそのものではない。学習の結果である。学習済みパラメータ、スケーラー、マッピング規則、場合によっては推論を続けるための最後の状態が入っている。サービングとは、そのモデルファイルを読み込み、リクエストに答えられる状態にすることだ。

学習がモデルファイルを作る

サービングがモデルファイルをロードする

推論が新しい特徴量をロード済みモデルへ送る

リスクがモデル出力を注文にしてよいか決める

同じパターンは本書全体に繰り返し現れる。HMMの学習はレジームパラメータを作る。HMMの推論は今日のレジーム確率を読む。SACの学習はActorを作る。SACの推論は行動を提案する。MLflowは、どの学習実行からどのモデルファイルが生まれたかを記録する。リスク管理は、モデルの出力が大きすぎる注文にならないよう、最後の段階で抑えたり止めたりする役割を担う。

実運用では、この流れは2つの時間軸に分かれる。

遅い時間軸：学習 / 検証 / モデル登録

過去データ -> 学習 -> OOS検証 -> モデルファイル保存 -> 承認

速い時間軸：リアルタイム推論 / 注文

今日のデータ -> 特徴量 -> モデルファイルをロード -> 推論 -> リスク確認
-> 注文 -> 約定確認 -> モニタリング

取引時間中のモデルは、毎分もう一度勉強している学生ではない。試験会場で答えを出す計算機に近い。学習は別の時間に終わっている。ライブシステムは承認済みのモデルファイルを読み込み、今日の特徴量1行を入れ、出力を受け取り、その出力を注文にしてよいかをリスクレイヤーで判断する。

したがって、モデルファイルを保存するとは「ディスクにファイルを1つ置く」という意味だけではない。入力形式、学習済みの数値、出力の意味を一緒に固定するという意味である。学習時の順序が[return, volatility, momentum, volume]だったのに、推論時に[return, volume, volatility, momentum]を送れば、モデルは4つの数字を受け取っていても意味を間違えて読む。後の章で特徴量順序、スケーラー、レジームラベルの対応付け、サービング、運用検証を繰り返し扱う理由はここにある。

本書が扱うものと扱わないもの

本書の範囲をまずはっきりさせておく。

本書が扱わないこと

- ・ Python の入門書ではない。ループを書ける、ライブラリをインポートできる、という前提から始める。
- ・ チャートトレーディングの教本ではない。支持線や MACD ダイバージェンスのようなチャートベースのテクニカル分析は扱わない。
- ・ 一攫千金の手引きでもない。クオンツトレーディングは難しく、リスクも大きい。

本書が扱うこと

クオンツトレーディングシステムを最初から自分の手で組み立てるための実践ガイドだ。ルネサンス・テクノロジーズのようなクオンツヘッジファンドが用いるタイプのシ

システムを、AI の支援を受けながら段階を追って構築する。本書に載っているコードは実際に動き、結果は数値で確かめられる。

本書を読み終えたときに作れるもの

システム	一行で言うと
HMM 市場局面ディテクター	未来のデータを使わず、これまでのデータだけで市場が今の局面(上昇・横ばい・下落)にあるのかを推定する。
複数資産を同時に見る HMM	株式や債券など複数の資産のシグナルを同時に読み、より精度の高い局面把握を行う。
2 段階のマクロ/ミクロ HMM	マクロ経済と個別資産を 2 段階で見て、市場を Risk-On(リスク選好)、Neutral(中立)、Risk-Off(リスク回避)に分類する。
強化学習トレーディングエージェント	HMM が検出した局面を入力として受け取り、ポジションを自分で決める。
統計的裁定取引システム	一緒に動く資産ペアを見つけ、その価格差をねらう。
実運用デブロイシステム	IBKR(Interactive Brokers) と KIS(韓国投資証券) のブローカーに接続し、実際の注文を出す。

それぞれの部品がなぜ必要なのか、間違っているとどんなふうに崩れるのかを理解すること。それが本書の本当の目標だ。

ルネサンスの事例から取り出した原則

本書全体を貫く原則が三つある。ルネサンス・テクノロジーズのトレーディング手法を分析するなかで取り出したものだ。

1. 因果性の原則

未来のデータを現在の判断に使ってはいけない。

当たり前に聞こえるが、コードに落とし込んでいくと意外と簡単に破ってしまう。よく使うライブラリ関数のなかにも、内部で未来情報をこっそり混ぜているものがある。本書ではそうした関数の代わりに、「今までのデータしか見ない」という約束を守る方法(フォワードアルゴリズム)を自分で実装する。詳しい理由は 5 章で説明する。一度理解してしまえば、以降のコードで同じ問題がすぐ目に飛び込んでくるようになる。

2. アンサンブルの原則

一つのモデルに頼り切らない。

このシステムでは、日足の HMM、時間足の HMM、強化学習エージェント、統計的裁定取引のシグナルを同時に使う。各モデルの判断が食い違うときは待つ。すべて一致したときに動く。

3. ウォークフォワード検証の原則

戦略が本当に機能するかを確かめる方法は一つしかない。

モデルが一度も見なかったデータを、時間の順序に沿って繰り返しテストすることだ。データを 1 回だけ訓練用とテスト用に分ける方法では足りない。ウォークフォワード検証は、本書のすべての戦略に適用する基本ラインだ。

これからの道のり

システム全体は、データ -> ノイズ除去 -> 局面検出 -> ポジション判断 -> 検証と実運用、という順に積み上げる。それぞれの段階が、一つの章のまとまりとして続いていく。

段階	やること
1	市場データを集めて保存する
2	価格に混ざったノイズを減らす
3	複数資産を同時に見る HMM で市場局面を推定する
4	マクロ/ミクロの 2 段階で局面を Risk-On / Neutral / Risk-Off に整理する
5	局面に合わせてどれだけのポジションを持つかを定める
6	統計的裁定取引で追加の収益機会を探す
7	強化学習エージェントがポジション判断を補助する
8	リスク管理、ブローカー接続、モニタリングで運用条件を整える

最初に読むときは、1～18 章を**基本システムを一周完成させる流れ**として眺めるとよい。19～27 章は同じ素材をもう一歩深く扱う応用編だ。ペア理論、完全な HMM パイプライン、運用検証、分布型 RL、MLflow サービングといった、実際の運用段階で改めて必要になる内容をまとめた区間になっている。

参考:バックテストの数字をどう読むか

本書にはいくつものバックテスト結果が登場するが、特定の数値を将来の成績を保証するものとしては読まないでほしい。同じ戦略でも、データ期間、取引コスト、銘柄ユニバース、再学習頻度、スリッページの仮定によって結果は大きく変わる。

最初に読むときは、数字よりも次の三つを見る。

- ・ 未来のデータを覗いていないか
- ・ 最大ドロウダウンと取引コストも一緒に確認しているか
- ・ OOS(サンプル外)区間でも似たパターンが保たれているか

全章ロードマップ

Part	章	テーマ
I	まえがき・1～3 章	システム全体像、Look-ahead Bias、データパイプライン、バックテストエンジン
II	4～5 章	HMM の基本構造と学習アルゴリズム、未来情報なしで推論する方法
III	6～8 章	カルマンフィルタ(ノイズ除去)、複数資産を同時に見る HMM、2 段階のマクロ/ミクロ HMM (システムの中核)
IV	9～10 章	ポジションサイズを決める、HMM アンサンブルで方向シグナルを作る
V	11～12 章	一緒に動くペアを見つけ、統計的裁定取引のシグナルを作る
VI	13～15 章	強化学習環境、SAC エージェント、ハイパーパラメータチューニング
VII	16～18 章	リスク管理、ブローカー連携、リアルタイムモニタリング
VIII	19～27 章	応用編:ペア理論、レジームベースの Z スコア、完全パイプライン、運用検証、分布型 RL、MLflow サービング

表に出てきた用語の多くは、初見ではどれもなじみが薄い。今この場で覚える必要はない。最初の通読では Part I～VII を通して「データから注文・モニタリングま

で」を一周する。Part VIII は二周目、あるいは実システムを磨き上げるときに必要になる応用区間だと思っておけばよい。

回ごとに読み方を変える

本書は一度ですべての数式とコードを理解するための本ではない。最初からすべてのパラメータや実装の細部を追おうとすると、全体の流れを見失いやすい。読む回ごとに目標を変えるとよい。

周	目標	読む基準
1 周目	大きな流れをつかむ	データがどうモデル判断と注文へつながるかを見る
2 周目	コードと概念をつなぐ	各章の入力、出力、失敗点をコードで確認する
3 周目	実験と修正	パラメータを変え、OOS結果がどう動くかを見る

1 周目では数式を完全に理解できなくてもよい。代わりに、各章が次のどの問いに答えているかをつかむ。

この章はデータを準備する章か？
市場を読む章か？
機会を探す章か？
サイズを決める章か？
リスクを止める章か？
運用状態を確認する章か？

2 周目ではコードに降りる。`fit()` がどこで呼ばれるか、`predict()` や `step_forward()` がどの区間でだけ実行されるか、OOS 区間が学習に混ざっていないかを確認する。この段階になると、数式は暗記対象ではなく、コードがなぜその形をしているかを説明する地図になる。

3 周目では実験する。ウィンドウ長、エントリー閾値、レジーム数、取引コスト、リスク上限を少しずつ変える。ただし、同じ OOS 結果を見ながら何度もチューニングして

はいけない。値を変えるなら、どんな仮説で変えたのかを記録し、検証区間と最終テスト区間を分ける。

Partごとの最低理解目標

最初に読むとき、各Partで最低限これだけは説明できるようになっておきたい一文は次のとおりである。

Part	最低理解目標
I	データとバックテストが嘘をつかない土台を作る
II	HMMは未来価格ではなく、市場状態を確率で読む
III	ノイズを減らし、複数市場を同時に見るほどレジーム判断は安定する
IV	シグナルは方向であり、Kellyはその方向を資金サイズに変える
V	ペイトレードは一銘柄の方向ではなく、二つの価格の関係を取引する
VI	RLはすべてのシグナルを作る魔法のAIではなく、ポジションサイズを調整する層である
VII	モデルが正しくても、リスク、注文、モニタリングが壊れれば実運用は失敗する
VIII	同じ材料をより厳しく検証し、運用可能な形へ整える

この表を最初の基準にすればよい。細かい数式がまだ難しくても、各Partの最低目標を説明できれば1周目は十分に成功である。

AI をペアプログラマとして使う

本書では、行き詰まったところごとに AI コーディングアシスタントをどう活用したかを直接見せていく。アルゴリズムを理解するために使ったプロンプト、AI が拾ってくれた見つけにくいバグ、AI のおかげでより良い方向に修正できた瞬間が、本文のあちこちに織り込まれている。

ねらいは思考そのものを AI に渡してしまうことではない。**頼れる先輩エンジニアに相談するように**使えばいい。分からないことは聞き、理解できたものは確認し、知っている部分はさっと飛ばし、詰まった部分は一緒にほどこいていく。

クオンツトレーディングは博士号を持つ人だけのものではない。道具と方向さえきちんと選べば、実用的なトレーディングシステムを自分の手で組み立てられる。本書はその過程の記録だ。

この章で必ず覚えておくこと

- ・ 本書はチャートパターンをより正確に覚えるための本ではない。市場を「データで読み解くシステム」を作るための本だ。
- ・ 最も重要な約束はたった一つ、「**未来のデータを現在の判断に使わない**」ことだ。この約束が破れた瞬間、見栄えの良い成績はすべて偽物になる。
- ・ 一つのモデルにすべてを賭けない。複数のモデルが同じ方向を指したときだけ動く。
- ・ 最初からすべての数学を完璧に押さえる必要はない。各章で「なぜこの概念が必要なのか」「間違っているとどんな問題が起きるのか」を順番に追っていけばよい。

前提知識

本格的な内容に入る前に、以下くらいは準備しておいてほしい。

- ・ **Python 3.10 以上**
- ・ **NumPy と Pandas の基本的な使用経験** — 配列を扱い、データフレームを操作できる程度で十分だ。
- ・ **基礎的な統計の概念** — 平均、分散、正規分布が何かを知っている程度で構わない。それより難しい概念は、必要なところで本のなかで説明する。
- ・ **市場がなぜそんなふう動くのかを知りたいという好奇心**

準備ができたなら、次の章でシステム全体の設計図を広げていこう。

第0章 バイブコーディング・クイックスタート：まず小さな例を検証する

この章の目的は、小さな数値例を使い、**AIが説明した損益が正しいかを自分で確認すること**だ。著者の運用リポジトリ、証券口座、市場データは必要ない。Pythonを実行できるなら以下のコードを動かす。実行環境がなければ表を手計算すればよい。

最初に期待する結果を決める

現金1,000から始め、価格100の資産を10単位購入する。その後、価格は80まで下がり、100に戻る。最初の計算では費用をゼロとする。

時点	現金	保有数量	価格	総資産
取引前	1,000	0	100	1,000
購入後	0	10	100	1,000
価格下落時	0	10	80	800
価格回復時	0	10	100	1,000
決済後	1,000	0	100	1,000

総資産は現金 + 数量 × 価格だ。最終損益はゼロだが、最大ドロウダウンは $(1000-800)/1000=0.2$ 、つまり20%になる。最終残高だけを見ると、途中で負ったリスクを見落とす。

本文だけで実行できる確認コード

次のコードをfirst_check.pyという新しいファイルに貼り付ける。Pythonの標準機能だけを使うため、追加ファイルのダウンロードは不要だ。

```
prices = [100.0, 80.0, 100.0]
initial = 1000.0
quantity = initial / prices[0]
cash = initial - quantity * prices[0]
equity = [initial] + [cash + quantity * p for p in prices]
peak = initial
mdd = 0.0
for value in equity:
    peak = max(peak, value)
    mdd = max(mdd, (peak - value) / peak)
final_cash = cash + quantity * prices[-1]
```

```
assert final_cash == 1000.0
assert mdd == 0.2
print(final_cash, mdd)
```

ターミナルで`python3 first_check.py`を実行すると、`1000.0 0.2`と表示される。WindowsでPythonのコマンドが`py`なら、`py first_check.py`を使う。これは数量を固定した損益の検算であり、シグナル生成、実注文、HMMの学習は含まない。

AIへの依頼方法

上の表とPythonコードと一緒に確認してください。

1. 総資産と最大ドローダウンを手計算してください。
 2. 現金だけを記録すると、どのリスクを見落とすか説明してください。
 3. 購入と売却にそれぞれ10 bpの手数料がかかる場合、最終現金を計算してください。
 4. 証券会社、外部データ、新しいパッケージを追加しないでください。
- 回答後、表のどの値で結果を検証したか示してください。

10 bpは取引金額の0.1%だ。10単位を価格100で購入・売却すると、それぞれの費用は1、合計2となり、最終現金は998になる。この例では初期現金の全額を使って数量を決めるため、購入手数料の1だけ現金が一時的にマイナスになる。借入を認めない実際の口座では、手数料分を残すよう購入数量を減らす必要がある。固定数量の演習と実口座の制約を区別しよう。

時間の順序を確認する

今日の終値を見て作ったシグナルが、それより前の今日の始値で約定したとは仮定できない。本書の基本演習は、**t本目の終値後にシグナル確定 -> t+1本目の始値で約定 -> t+1本目の終値で評価**という順序だ。前もって確定したシグナルに従い始値で建てたポジションについて、その日の損益を含めることには問題がない。

失敗の原因を絞り込む

症状	最初に確認すること
Pythonコマンドが見つからない	インストール済みのコマンドがpython3かpyかを確認する
インデントエラー	コード部分だけをコピーし、字下げを保ったか確認する
ドロダウンがゼロになる	価格80の時点で総資産800を記録したか確認する
最終現金と損益合計が一致しない	損益の二重加算や費用の重複を確認する

新しいパッケージを入れる前に、上の例と異なる行を確認する。生成された説明のもっともらしさよりも、期待値と実際の出力が一致することを優先する。

次へ進む前の確認

- ・ 最終損益ゼロと最大ドロダウン20%が同時に成立する理由を説明できる。
- ・ 手数料を実際の約定ごとに一度だけ計上する。
- ・ シグナル生成時刻と約定時刻を区別する。

この三つを確認したら、第1章の構成説明へ進もう。accounting-labのZIPは、追加テストとCSVを含む任意の補助資料だ。提供状況は販売者の配布案内で確認する。本文のapp.tradingなどの名前は、著者のシステム構成を説明する例であり、概念を理解するために非公開パッケージへアクセスする必要はない。

第1章 システムアーキ テクチャ — AIクオオンツ トレーディングの設計 思想

Part I — 基本となる骨格(1～3章) このパートでは利益を生む戦略よりも前に、戦略が自分自身を欺かないための土台をつくる。1章は全体設計図、2章はデータパイプライン、3章はバックテストエンジン。最初の通読では「データが入ってきて、検証可能なバックテスト結果として出てくる順番」だけ掴めれば十分だ。

ローソク足チャートを初めて学んだとき、これで市場が読めると思った。サポートラインとレジスタンスラインを引き、ローソクのパターンを覚えた。十字線、ハンマー、抱

き線パターンまで勉強するほど、より多くの形が目に入るようになった。初めてチャートが読めるように感じた瞬間を今でも覚えている。

ところが実際に売買してみると違った。サポートだと思った位置で価格が反発する日もあった。しかし次のときは、その位置をそのまま突き抜けて下がっていった。同じローソク足パターンがある日は当たり、別の日には逆方向に動いた。パターンが本当に存在するのか、それとも自分が事後的につくり出しているのか、見分けがつかなかった。

理由を後で知った。市場の取引量の70%以上をアルゴリズムが占めている。アルゴリズムは個人投資家がどこにサポートラインを引くか知っている。そこに損切り注文(損失が一定水準を超えると自動で売却する注文、ストップロス)が集まることも知っている。だからアルゴリズムはその地点を正確に突いてから方向を変えたり、ストップロスを発動させて強制売却を誘発したうえで価格を引き上げたりする。チャートパターンが機能する理由と失敗する理由は、結局は同じだ — 自動化システムがそのパターンをすでに知っているからである。

この事実を受け入れると選択肢は二つしかない。チャートを見続けてアルゴリズムにやられないことを祈るか、市場の見方そのものを変えるかだ。

私は後者を選んだ。その結果がこの本である。本書の核となる視点はひとつだ — **価格そのものを予測しようとはしない。代わりに価格の裏に隠れている市場の局面(レジーム、regime)を推論する。**その道具がHMM(隠れマルコフモデル、Hidden Markov Model)だ。詳しい内容は後の章で扱う。

AIが登場する前まで、こうしたシステムを個人で構築するのは事実上不可能に近かった。統計の知識、データパイプライン、バックテストエンジン、執行システム、モニタリング — これらすべてを一人で組み立てなければならなかったからだ。今ならAIを作業パートナーにして、このアーキテクチャを一人でも構築できる。本書はそのプロセスを最初から最後まで扱う。

1章ではシステム全体の設計思想を扱う。各構成要素がなぜその位置にあるのか、互いにどうつながっているのかを先に把握しておく、その後の内容を追いやすくなる。

この章の流れ

まず全体像を掴み、その後で環境構築と初回実行に降りていく。

- ・ システム構造 — 5つのレイヤーがどう積み上がるか

- ・ ディレクトリ構造 — どのフォルダがどんな役割を持つか
- ・ コア構成要素 — すべての戦略が従う共通の約束
- ・ 因果性の原則 — 未来データを誤って使わないためのルール
- ・ データの流れ — バックテストが回る順番
- ・ 開発環境のセットアップと初めてのバックテスト実行

学習目標

この章を終えると、

- ・ 5つのレイヤーで構成されたアーキテクチャで、データがどの経路を通して流れるかを描ける
- ・ HMM Signal、SAC RL Agent、StatArbシグナルがそれぞれどんな役割を持つかを一文で言える
- ・ Look-ahead Bias(未来データを誤って使ってしまうバグ)とは何で、なぜ危険なのかを説明できる
- ・ 登録されている戦略一覧を確認し、最初のバックテストを実行できる

この章でつくるもの:戦略コードは一行も書いていないが、この章の最後では下のコマンド一行が実行できるようになる。

```
python -m app.trading.backtest.runner --signal hmm_daily_mr
--days 365
```

実行すると、シャープレシオ(リスクに対するリターン比率)、Sortino(下方リスクを基準にしたリターン比率)、MDD(最大ドローダウン)といった数字が出力される。動作原理は本書全体で説明していく。今は全体構造を把握できれば十分だ。

1.1 システム構造

なぜレイヤーに分けるのか

戦略を一つの大きなスクリプトとして書くと、データ収集、シグナル生成、注文執行が一つのファイルに混ざる。こうなると「データを差し替えてもう一度回してみよう」という単純な要望でも、コード全体をいじらないといけなくなる。責任を層に分けておけば、ある層を変えても他の層はそのまま済む。

このシステムは5つのレイヤーに分かれる。各レイヤーは個別にテストでき、新しい戦略を追加するときの下のレイヤーを修正する必要はない。

レイヤー	役割
Layer 5 — モニタリング	ダッシュボードで結果を見る
Layer 4 — 執行 & リスク	注文を出し、リスクを管理する
Layer 3 — Ensemble	複数のシグナルを合成して最終ポジションを決める
Layer 2 — Signal生成	HMM、強化学習、統計的裁定で売買シグナルをつくる
Layer 1 — データ	価格データを受け取り特徴量に変換する

データは下から上へ流れる。Layer 1で原データの価格を受け取り、特徴量に変換する。Layer 2のシグナル生成器がその特徴量を見て売買シグナルをつくる。Layer 3のEnsembleがシグナルを合成して最終ポジションを決定する。Layer 4が注文を出し、Layer 5が結果を見せる。

後の章でHMM、SAC、StatArbが順番に登場するが、互いに競合する手法ではなく、**それぞれ別のレイヤーで別の役割を担う部品**だ。

レイヤーに分けるもう一つの理由は、失敗箇所を早く見つけるためである。収益率が悪くなったとき、すぐに「モデルが悪い」と決めつけてはいけない。データが壊れているかもしれないし、特徴量の順序が変わったのかもしれないし、注文が一部しか約定していないのかもしれない。

レイヤー	担当すること	防ぐべき失敗
Layer 1 — データ	市場記録を特徴量に変換する	欠損値、タイムゾーン誤り、古いデータ
Layer 2 — Signal	HMM / Pairs / RL の判断を作る	未来情報の漏れ、誤ったレジーム、過剰なシグナル
Layer 3 — Ensemble	複数の判断をまとめる	1つのモデルの錯覚に頼りすぎる
Layer 4 — 執行 & リスク	注文前の最終安全確認	過大ポジション、損切り失敗、約定不一致
Layer 5 — モニタリング	運用状態を監視する	接続切断、成績悪化、通知漏れ

初心者は、この構造を市場を分解していく順番として読んでもよい。

- | | |
|--------------|-----------------|
| 1. 市場を記録する | Data |
| 2. 読める数値に変える | Features |
| 3. 状態を読む | HMM |
| 4. 機会を探す | Pairs / Z-score |
| 5. サイズを決める | SAC / RL |
| 6. 悪いリスクを止める | Risk |
| 7. 注文を送る | Broker |
| 8. 監視を続ける | Monitoring |

機械学習は主に3番で市場状態を読むために使う。深層学習/RLは主に5番で行動の大きさを決めるために使う。両方を一度に理解しようとする大きく見える。まずは**HMMは市場を読み、SACは行動サイズを選ぶ**と分けて考えればよい。

機械学習の視点で読み直す

5つのレイヤーは、1つの機械学習パイプラインとしても読める。

Layer 1は原データを**特徴量**に変換する。Layer 2はその特徴量を使って**モデル推論**を行う。Layer 3は複数のモデル出力をまとめて意思決定に変える。Layer 4以降は、もはや機械学習だけの話ではない。実際にお金動く、リスク・執行・運用の領域に入る。

このシステムでは、HMMが市場状態を読み、ペアトレードが機会を探し、SACが行動の大きさを調整し、リスク管理がその行動を口座が許容できるか判断する。1章でまず覚えるべき文はこれでよい。**モデルは提案し、リスクと執行が実際に何をしてよいかを決める。**

ここで言うモデルとは何か

「モデル」という言葉は曖昧に聞こえやすい。本書ではかなり具体的に読む。

モデルとは「ファイル一つ」や「数式一つ」のことではない。直観的には**判断を返す計算機**だと考えればよい。市場の数字を入れると、現在の市場状態やポジションサイズが返ってくる。

入力数値 → モデル → 出力数値

アルゴリズムはHMMやSACのような一般的な方法である。学習済みモデルは、そのアルゴリズムが特定の学習区間を見て内部パラメータを合わせた結果である。違う期間で学習した2つのHMMは、同じアルゴリズムでも別のモデルになり得る。

初心者が最も混同しやすいのが**学習**と**推論**だ。学習とは、過去データを使ってモデル内部の値を合わせることだ。HMMなら遷移確率や状態ごとの分布を見つけることであり、SACならニューラルネットワークの重みを調整することである。推論とは、すでに学習済みのモデルに今日の特徴量を入れ、未来データを混ぜずに出力を得ることだ。

どの章を読むときも、まず次の3つを確認するとよい。

1. モデルには何を入れるのか。
2. モデルから何が出るのか。
3. どのデータがモデルを形作ることを許されたのか。

これは「モデルは賢いのか」と考えるより実用的である。入力がきれいで、出力が明確で、OOS検証が正直なら、そのモデルは役に立つ。未来データが混ざったモデルは、バックテストが良く見えても役に立たない。

初回読みでは、各コンポーネントを次の文に当てはめて追うとよい。

このレイヤーは ____ を受け取り、 ____ に変換し、 ____ へ渡す。

この一文があるだけで、システムは名前の羅列ではなくなる。データは特徴量になり、特徴量はモデル出力になり、モデル出力はポジション提案になり、ポジション提案はリスクと執行を通る。これが本書全体の縮図である。

すべての章を同じ型で読む

本書の各章は名前こそ違うが、多くは同じ構造を持つ。章を細かく読む前に、まず次の4つの欄を埋めるとよい。

確認すること	問い
入力	この章のコードはどんなデータを受け取るか？
変換	そのデータをどんな判断用の数値へ変えるか？
出力	どんなシグナル、確率、サイズ、状態が出るか？
リスク	ここが間違うと、バックテストや実運用はどう壊れるか？

例えば6章のカルマンフィルタは次のように読める。

項目	答え
入力	ノイズを含む価格または特徴量
変換	新しい観測値をどれだけ信じるか判断して平滑化する
出力	ノイズの少ない特徴量、Innovation Z-score
リスク	ノイズをレジーム変化と誤解する

17章のIBKR接続も同じ型で読める。

項目	答え
入力	リスクレイヤーを通過した注文意図
変換	ブローカーへ提出し、応答を確認する
出力	約定、ポジション、現金残高の更新

項目	答え
リスク	注文が通っていないのに通ったと思い込む、または部分約定を見落とす

こう読むと、機械学習の章と運用の章は別々の話ではない。すべて **入力 -> 変換 -> 出力 -> 検証** という同じ構造を持っている。

1.2 ディレクトリ構造

ファイル構造を暗記する必要はない。初心者は下の4つの塊の役割だけ覚えればよい。

```
stock_trading_ai_py/
+-- app/trading/
|   +-- backtest/           # 戦略を検証するエンジン
|   +-- backtest/signals/   # HMM、StatArb、RLシグナル一覧
|   +-- rl/                 # SAC強化学習の実装
|   +-- cache/              # 価格データのキャッシュ
+-- refresh_cache.py        # データキャッシュの更新
+-- pyproject.toml          # Pythonの依存関係
```

backtest/は約定と会計、signals/は未来情報を使わない判断、rl/は強化学習モデル、cache/はデータの読み書きを担当する。これらのディレクトリ名は責任分割の例である。元のファイルツリーを復元しなくても、入力がどの境界を通るかを追える。

1.3 コア構成要素

SignalGenerator — 共通の約束

戦略ごとに入出力の形式が違っていると、バックテストエンジンが戦略を一貫して扱えない。そこで、すべての戦略は同じ約束事(インターフェース)に従う。どんな戦略も、2つのステップで動く。

1. initialize() — 戦略が始まるときに一度だけ呼ばれ、データを準備する。

2. `on_bar()` — 新しいバー(価格データ1コマ)が入るたびに呼ばれ、売買シグナルを返す。

戦略インターフェースの最小契約は「過去の資料で準備する」と「現在までの資料から目標シグナルを返す」の二段階である。入力にはバーの時刻、確定観測、現在の保有状態を含め、出力には方向、目標サイズ、有効時刻、拒否理由を含める。シグナル関数が残高を直接変更しないよう、約定と会計は別のエンジンが担当する。

SIGNAL_REGISTRY — 登録された戦略一覧

SIGNAL_REGISTRYは戦略名と実際の実装クラスを結びつける一覧だ。ここで覚えるべきは個別の名前ではなく、「**戦略を実行するときに文字列ひとつで実装を選ぶ**」という仕組みである。

代表的なシグナルは3つの塊に分かれる。

塊	例	役割
HMM系列	<code>hmm_daily_mr</code> など	市場レジームを推定する
StatArb系列	<code>stat_pair_rl</code> など	ペアおよび平均回帰シグナルをつくる
統合系列	<code>regime_orchestrator</code> など	複数シグナルを組み合わせる

登録されている全リストは次のコマンドで見られる。

```
python -m app.trading.backtest.runner --list
```

BacktestResult — 標準化された結果フォーマット

戦略ごとに結果の形式が違っていると比較しにくい。BacktestResultは、すべての戦略の成績を同じ形式で格納する結果オブジェクトだ。最初は下の4指標だけ見ればよい。

指標	意味	先に見る理由
score	変動性に対するリターン	戦略品質の一次基準
md	最大ドロダウン	口座がどれだけ揺れたか
win_rate	勝ちトレードの比率	シグナルがランダムすぎないかの確認
annual_return	年率換算リターン	実際に期待できる収益規模

sortino、cvar_95、calmarなどの指標は後の章で再び登場する。1章では「**すべての戦略結果が同じ標準フォーマットに集まる**」という点だけ理解しておけばよい。

最低資金の要件:*capital* = \$100,000がデフォルト値だが、SPYを500株買うには\$300k以上必要になる。口座資金が不足するとポジション計算がきちんと動かない。実際の口座規模に合わせて*capital*を修正するか、*max_position_usd*パラメータで銘柄ごとの上限を別途指定すること。

SAC — 自前で実装した強化学習エージェント

本書の強化学習部分は、汎用ライブラリ(stable-baselines3)を使わず、**SAC(Soft Actor-Critic)**をPyTorchで自前実装している。トレーディングに合わせて細部の挙動を制御するためだ。

詳しい構造は13章と14章で扱う。今は入出力の形だけ覚えておけばよい。

- ・ **入力:** 市場の特徴量28個
- ・ **出力:** ポジションサイズ1個

学習設定の全体はapp/trading/rl/config.pyにある。

1.4 因果性の原則

WARNING

未来のデータを現在の意思決定に使うと、バックテストの成績は高くなるが、実運用のリターンはマイナスになる。訓練データだけにfitを適用し、推論では時点 t までのデータだけを使うこと。

Look-ahead Biasは、バックテストのリターンを偽って膨らませる最も多いバグだ。時点 t で意思決定するときに、 $t + 1$ 以降のデータが入り込んでしまう状況のことを指す。バックテストでは未来のデータを知っているため、うっかり混入させやすい。

初心者が最もよく出会う2つの落とし穴は次のとおり。

落とし穴1 — HMMで未来の観測値まで参照する

`hmm.predict_proba(X_all)`は便利に見えるが危険だ。この関数は時系列全体を一度に見た後で各時点のレジーム確率を再計算するため、時点 t の結果に未来情報が混ざる。時点 t の結果は $x_1, \dots, x_t$ だけを見て計算すべきだ。本書では`_do_forward_pass`だけを使ってこのルールを守る(具体的なコードは5章)。

落とし穴2 — スケーラを全期間でfit

```
# Incorrect/unsafe example.
scaler = StandardScaler()
scaler.fit(X_all)

# Correct pattern.
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

`fit`が平均と標準偏差を計算するときに未来データを見てしまうと、バックテストでは良く見えるが、実運用ではその平均は知りようがない。**ルールはシンプルだ — fitは訓練区間だけで、テスト区間ではtransformだけ呼ぶ。**

Walk-forward検証:一度の分割で終わらせず、時系列順に繰り返し分割する。このプロジェクトの`OOS_RATIO=0.40`は、全期間の最後の40%をOOS(Out-of-Sample、学習に使っていない検証区間)として使う。3章で詳しく扱う。

1.5 データの流れ — バックテストはどんな順番で回るか

バックテスト1回の流れは4ステップだ。

1. 実行するシグナル群がどの時間間隔(1d、1h、5m)を要求しているかをまとめる。
2. 必要な価格データをキャッシュから読むか、新しくダウンロードする。
3. 各シグナルを同じデータ、同じ条件で実行する。
4. 結果をBacktestResultのリストとして返す。

比較実行器は同じデータスナップショット、評価期間、初期資産、コスト、ポジション上限を各戦略へ渡し、結果を戦略IDで集める。並列実行でも戦略ごとに独立した口座状態を使う。ある戦略が他の戦略のキャッシュや乱数状態を変更しないよう入力を固定する。

1.6 開発環境のセットアップ

Python 3.12環境

仮想環境は一度だけ作ればよい。下の4行を順番に打つ。

```
cd stock_trading_ai_py
uv venv .venv --python 3.12
source .venv/bin/activate
uv sync
```

uv syncはOSに合わせてCUDA(Linux)またはMPS(macOS)ビルドを自動で選ぶ。失敗したらずPythonバージョンを確認する — pyproject.tomlにrequires-python = ">=3.12,<3.13"が固定されている。

主要パッケージ

名前と役割だけ一通り眺めておこう。詳しい使い方は該当機能を扱う章で再び出てくる。

パッケージ	役割
hmmlearn	HMM(レジーム推定)
torch	SACニューラルネット
optuna	ハイパーパラメータ最適化
duckdb	ローカルデータキャッシュ
yfinance	株価収集
pykalman	カルマンフィルタ
statsmodels	時系列の統計検定
ib-async	IBKR注文API

正確なバージョンは`pyproject.toml`に固定されている。

バージョンは正確に固定する:`torch>=2.0`のように範囲だけ指定すると、数か月後にパッケージが更新されたときコードが合わなくなる可能性がある。本番デプロイ前に`uv lock`または`pip freeze > requirements.txt`で正確なバージョンを記録しておくこと。

設定ファイル

APIキー、ファイルパス、定数は`.env`ファイル1つにまとめて管理する。コードにハードコードしなければ、キーを誤ってGitリポジトリにアップロードする危険が減る。

```
cp .env.example .env
```

```
# .env
IBKR_HOST=127.0.0.1
IBKR_PORT=7497
IBKR_CLIENT_ID=1
DASHBOARD_TOKEN=<CHANGE_ME>
DASHBOARD_SERVER_URL=http://127.0.0.1:8765
```

DASHBOARD_TOKENは必ず自分の値に変える。デフォルト値のままだとダッシュボードの認証は事実上無効になる。

1.7 データキャッシュの初期化

S&P500の日足/時間足データを一気に取得する。初回ダウンロードはおよそ30分かかる。

```
python refresh_cache.py
python _check_duckdb_cache.py
```

_check_duckdb_cache.pyの出力は次のような形になる。

インターバル	カバレッジ	期間
daily_730d	498/500銘柄	2023-04 ~2025-04
hourly_730d	493/500銘柄	2023-04 ~2025-04
5m	312/500銘柄	2025-02 ~2025-04 (60日)

キャッシュサイズは約3.2 GBだ。500銘柄のうち一部が欠けるのは上場廃止やyfinanceの応答欠落によるもので、正常な範囲である。失敗したらまずディスク容量とネットワークを確認する。

1.8 最初のバックテストを実行する

この節で1章の約束を果たす。コマンド一行で実際にバックテストが回る。

```
# 단일 전략
python -m app.trading.backtest.runner --signal hmm_daily_mr
--days 365

# 두 전략 비교
python -m app.trading.backtest.runner \
    --signal hmm_daily_mr hmm_daily_pure_z \
    --days 730

# 등록된 전략 목록
python -m app.trading.backtest.runner --list
```

比較コマンドを回すと、次のような結果表が出力される。

Strategy	主な変化	確認するポイント
HMM Daily MR	レジームフィルタを使用	損失区間が減ったか
Pure Z-score (HMMなし)	固定Z-scoreのみ使用	市場局面の変化に弱いか

ここで確認するのはコードではなく、結果表の読み方だ。同じZ-scoreシグナルでも、HMMレジームフィルタを加えると損失区間や取引頻度が変わる。本書で繰り返し確認していくことになる中心的なパターンである。

結果の読み方: 初回実行のとき、どの数字を基準に見ればよいか。

指標	先に問うべき点
損益カーブ	特定区間にだけ収益が偏っていないか
MDD	口座が耐えられるドロウダウンか
勝率	シグナルがランダムに見えていないか

初回実行では、数字そのものよりも結果が再現するか、コストを入れても方向性が保たれるかを確認する。

失敗したら見る順序:出力に「正常」ではなく「エラー」が表示されたら、(1)キャッシュが空になっていないか(`python _check_duckdb_cache.py`)、(2)`capital`が小さすぎないか、(3)`--days`がキャッシュ期間を超えていないか、の順に点検する。

Exercises

1. 5レイヤーアーキテクチャで、データがLayer 1からLayer 4まで流れる過程を自分の言葉で説明せよ。各レイヤーの責務を一行ずつで十分だ。
2. 登録された戦略一覧を出力するコマンドは何か。出力からHMM系列の戦略を選んでみよ。
3. 次のコードがなぜLook-ahead Biasを引き起こすのか、1〜2文で説明せよ。

```
scaler = StandardScaler().fit(X_all)
X_scaled = scaler.transform(X_all)
```

まとめ

この章で必ず覚えておきたいのは次の5点だ。

- ・ **5つのレイヤー** — データ -> 特徴量 -> Signal -> Ensemble -> 実行 の順にデータが流れる。
- ・ **Signalは共通の約束に従う** — `initialize()`で準備し、`on_bar()`でシグナルを出す。
- ・ **すべての戦略は同じ形式で比較される** — `BacktestResult`に標準化されて集まる。
- ・ **未来データを使ってはいけない** — `fit`は訓練区間だけ、推論は時点 t まで。
- ・ **最初のバックテストを自分で回せる** — コマンド一行で結果表が出る。

次の章では`download_bars()`がDuckDBにデータをキャッシュする仕組みと、特徴量エンジニアリングの構成をコードで直接見ていく。

さらに読むなら

- ・ 書籍: Marcos Lopez de Prado, *Advances in Financial Machine Learning* (Wiley, 2018) — 因果性と Look-ahead Bias を丁寧に扱っている。
- ・ 論文: “Why Most Backtests Fail” — バックテストが実運用で崩れる理由を分析している。
- ・ コード: `mlfinlab` ライブラリ — 時系列特化の特徴量と検証ツールが含まれている。

理解を確認する

「data・signal・約定・会計の入力出力と、未来情報が入り得る箇所を表にして。」

第2章 データパイプライン—DuckDBとリアルタイムフィード

クオンツシステムの品質は、データの品質を超えられない。当たり前のように聞こえるが、パイプラインを整える前に戦略コードから書き始めてみると、この言葉がなぜ重要なのかをすぐ確認することになる。モデルがどれほど精緻でも、入力データが間違っていれば、バックテストの好成績は実際の収益可能性ではなく、データの誤りを学習しただけの結果かもしれない。

最初にバックテストを始めたとき、手元にあったのはYahoo Financeから落としてきたCSVファイルだけだった。OHLCVデータでバックテストを回すと成績は良かったのに、実取引に乗せると結果が違った。戦略ロジックが壊れていたわけではない。最初からバックテストが、実際の取引条件とは違うデータを見ていたのだ。

原因は三つあった。

第一に、**生存者バイアス**である。バックテスト期間中に上場廃止になった銘柄は、現在のデータには残っていないことがある。それらを除外してしまえば「生き残った銘柄」だけを見ていることになり、成績が実際より良く出る。

第二に、**未来データの参照**である。終値で計算した指標を当日の売買シグナルとして使うと、引け後でなければ分からない情報を先取りしていることになる。現実には不可能な取引だ。

第三に、**配当・分割の未調整**である。株式分割を調整しないと、その日付の前後で価格が突然半分になったように見える。戦略はこの急変を本物のシグナルと受け取ってしまう。

このうち一つでもあれば、バックテスト結果は信用しにくくなる。

クオントのデータパイプラインは、こうした問題をコードのレベルで解決する。中核となる道具はDuckDB(ParquetファイルにそのままSQLを投げられる軽量データベース)だ。Pandasより大きなデータを安定して扱えるし、巨大なデータを一度にメモリへ載せなくても処理できる。リアルタイムフィードの連携、データ品質の検証、生存者バイアスの回避をパイプライン内で済ませてしまえば、戦略コードはずっと単純になる。

この章の流れ

まず全体像をつかみ、次に実装と検証へ進んでいく。

1. データフロー全体—APIからモデル入力まで
2. `download_bars()` —単一のダウンロード窓口とフォールバック構造
3. `DuckDBCache` —Parquetファイルとメモリ上DuckDBの組み合わせ
4. `tz_localize(None)` —このプロジェクトの暗黙のルール
5. 4次元の日足特微量と欠損値処理
6. カルマン平滑化、Zスコア、モデル入力の形

学習目標

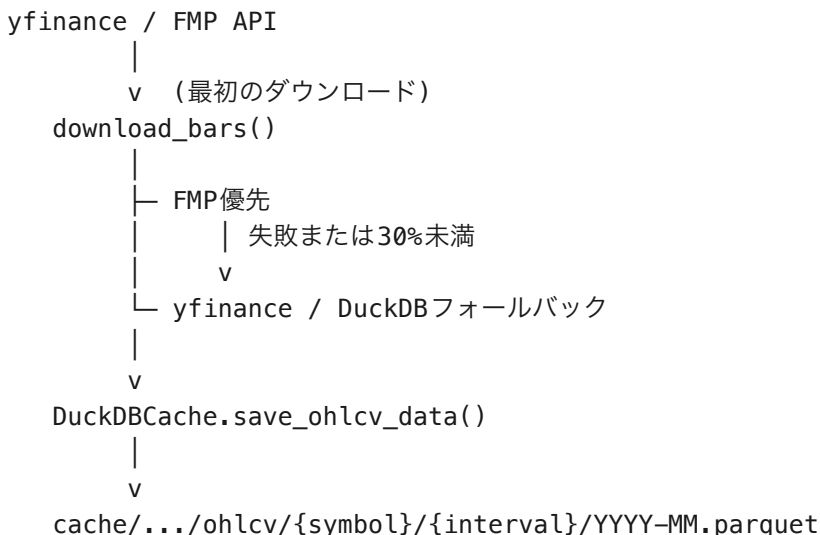
この章を終えると、次のことができるようになる。

- ・ `download_bars()` が FMP に先にリクエストし、失敗すると `yfinance` へ切り替える **フォールバック (fallback、代替経路)** 構造を説明できる。
- ・ `DuckDBCache` が Parquet ファイルとメモリ上 DuckDB を併用する理由、そして複数プロセスから同時にアクセスしても衝突が起きない理由を説明できる。
- ・ 日足データからリターン・実現ボラティリティ・モメンタム・出来高比率という4つの特徴量を計算し、カルマンフィルタでノイズを抑える意味を理解する。
- ・ rolling window によって生じる欠損値 (NaN) をなぜ除去または別途処理する必要があるのかを説明できる。
- ・ `DataFrame` にタイムゾーン情報が混ざるとどんなエラーが出るか、そして `tz_localize(None)` がそれを防ぐ理由を説明できる。

この章で作るもの: `download_bars()` で S&P500 の日足データを取得して DuckDB の Parquet キャッシュに保存し、4 つの特徴量配列へ変換するパイプラインを組み立てる。

2.1 全体像—Data Flow

まずデータがどのように流れるのかを見る。API リクエストから始まり、モデルに入る特徴量行列で終わる、一本道の経路だ。



```

      |   (月別パーティション、Append-only)
      v
compute_daily_features()
      |
      v
np.ndarray shape=(T, 4)  ->  HMM.fit()

```

キャッシュがすでに存在すれば、ネットワークに出ずにParquetファイルから直接読む。この構造が大事なのは速度だけが理由ではない。同じ入力データで同じバックテストを再現できなければ、結果を比較する意味がなくなる。毎回APIから少しずつ違うデータが返ってきたら、戦略変更の効果とデータ変更の効果が混ざってしまう。

原データが特徴量になるまで

初心者が最初に迷いやすいのは、「価格データがどうやってモデル入力になるのか」という点である。モデルはチャート画像を見ているわけではない。数値の表を読んでいる。だからOHLCVをそのまま入れるのではなく、モデルが読みやすい要約値に変換する。

原データ

date	open	high	low	close	volume
2020-01-02	...	...	...	...	...
2020-01-03	...	...	...	...	...

特徴量行列 X

date	return	volatility	momentum	volume_ratio
2020-01-02	...	...	...	...
2020-01-03	...	...	...	...

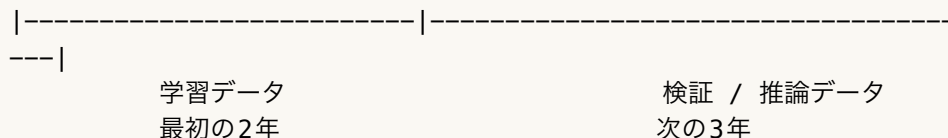
OHLCVは人間にはなじみがあるが、モデルにとってはまだ原材料に近い。特徴量行列Xは、「今日はどれだけ動いたか」「ボラティリティは高まったか」「出来高は普段より多いか」という問いに変換した表である。

1行は1つの時点を表し、1列はモデルが読む1つの観点を表す。

行 (row)	= 1つの時点
列 (column)	= モデルが読む1つの問い
X[t]	= 時点tでモデルに渡す数値の束

このデータの見方をすると、学習と推論は同じ特徴量行列を時間で分ける作業だと分かる。

特徴量行列 X: 5年分



```
X_train -> fit(model)
X_oos   -> 1行ずつ前に進めて推論
```

X_oosはファイルの中では過去データだが、学習時のモデルには未来として隠しておく。検証ではコードが1日ずつ前に進み、その日に分かるx_todayだけをモデルへ渡す。これがデータの視点から見たOOS検証である。

2.2 downloadBars() —単一のダウンロード窓口

downloadBars()は、この本全体を通してデータを取得する唯一の窓口である。Yahoo FinanceでもFMPでも、すべてこの一関数を経由する。

データ取得コードが複数のファイルに散らばっていると、フォールバックの方針、タイムゾーン処理、キャッシュの保存形式がファイルごとにばらつく。関数を一つに集めておけば、こうしたルールを一箇所で管理できる。

関数がサポートするインターバル

インターバル	最大期間	一次ソース	フォールバック
1d	無制限	FMP	yfinance
1h	730日	FMP	DuckDB
5m~30m	60日	FMP	yfinance

4hと1wkは基底インターバル（1h、1d）を集約して作る。同じ取引所の時間帯とセッション境界内で、始値は最初の値、高値は最大値、安値は最小値、終値は最後の値、出来高は合計とする。未確定の集約バーは確定済みバーと区別する。週足には休場日を考慮した当該取引週の最後の実際の取引時刻を記録する。UTCの午前0時から単純に4行ずつまとめるとセッションがずれる場合がある。

中核フロー—2段階のフォールバック

ローダーの契約は（銘柄，周期，開始，終了）→ 時刻順のOHLCVである。重複時刻はデータ源と更新時刻を基準に解決し、価格が正、高値が安値以上、出来高が非負であることを検証する。失敗したダウンロードを正常な空データとしてキャッシュに保存してはいけない。動作順序は次のとおりである。

1. **第1段階:** FMPにリクエストする。
2. **第2段階:** 返ってきた銘柄数がリクエストの30%未満なら失敗とみなし、日足はyfinanceへ、時間足はDuckDBキャッシュへ切り替える。

結果は{symbol: DataFrame}という形の辞書である。どの銘柄も取れなかった場合は空の{}が返る。

なぜフォールバック基準が30%か: FMPに500銘柄をリクエストして150銘柄未満しか返ってこなければ失敗とみなす。市場の休場日、APIレート制限、ネットワーク障害などで一部の銘柄が欠けることがあるためだ。

リトライなし—運用時の注意: 現在の実装では、APIリクエストが失敗するとすぐにフォールバックへ移る。ネットワークが1秒切れて復旧したような場合も同じ扱いだ。リアルタイム運用環境では、*tenacity*のようなライブラリでリトライを足しておくほうが安全である。

フォールバックが実際に発生したかを確認する

フォールバックは静かに起きるので、デバッグログをオンにして、どこで切り替わったのかを見る必要がある。

```
import logging
logging.getLogger("quant.data").setLevel(logging.DEBUG)
```

出力例は次のようになる。

```
DEBUG: FMP coverage 18% < 30% for AAPL ... — falling back to yfinance
WARNING: All sources failed for TSLA 2008-01-01 — returning empty
DataFrame
```

失敗時に確認すること: 末尾に `returning empty DataFrame` の警告が見えたら、その区間のシグナルは0として扱われる。バックテスト結果がおかしいときは、まずこのログから確かめる。

2.3 DuckDBCache —Parquet + メモリ上 DuckDB

キャッシュ層は二つのことを担う。**どこに**保存するか(Parquetファイル)、そして**どう読むか**(メモリ上のDuckDB)である。

パーティショニング (Partitioning) とは? データを複数のファイルに分けて保存する方式のことだ。DuckDBは `WHERE symbol='AAPL' AND ym='2024-01'` のようなクエリを実行するとき、条件に該当するファイルだけを読み、それ以外はスキップする。

なぜDuckDBか

CSVやSQLiteと比較した長所と短所を表にまとめる。

	SQLite	CSV	Parquet + DuckDB
クエリ速度	普通	遅い	速い
マルチプロセス	単一writer	O	O (Append-only)
圧縮	任意	なし	自動
ファイルサイズ	普通	大きい	小さい

特にバックテストは「特定の銘柄、特定の期間」を繰り返し読む作業なので、条件に合うファイルだけを選んで読む方式が効く。

保存構造—月別Parquet

銘柄・インターバル・年月の3段のディレクトリに分かれる。

```
cache/historical_data/ohlcv/
├── AAPL/
│   ├── 1d/
│   │   ├── 2023-01.parquet
│   │   ├── 2023-02.parquet
│   │   └── ...
│   └── 1h/
│       └── 2024-11.parquet
└── MSFT/...
```

月単位で細かく分ける理由は単純だ。バックテストが1年分のデータを見るなら12個のファイルを開けば済む。銘柄ごとに一塊で保存していたら、10年分全体を読まなければならない。

同時アクセスをどう扱うか

複数プロセスが同じキャッシュを更新する場合、ファイルロックと原子的な置換を区別する。ロックは書き手の同時実行を防ぎ、置換は読者が書きかけのファイルを読むことを防ぐ。プロセス内のスレッドロックだけでは別プロセスを排除できない。以下の二つを併用する。

- **Singleton:** プロセス内でDuckDBCacheオブジェクトをただ一つだけ生成する。キャッシュパスの設定が複数箇所ですれてしまう事故を防ぐ。
- **ThreadLocal接続:** スレッドごとに自分専用のDuckDB接続を持つ。スレッド間で接続を共有するとクエリが絡まることがあるためだ。

Parquetファイルは複数プロセスから同時に読むことができる。書き込みは月別パーティションファイルに追記する (Append-only) 方式でしか起こらない。同じファイルを同時に上書きしないので、ロックの衝突は発生しない。

保存関数の4ステップ

保存の契約は、検証済みテーブルを一時パスへ書き、成功した場合だけ最終パスへ置き換えることである。例外時は一時ファイルを片付け、既存の正常なキャッシュを維持する。データとメタデータを別々に置き換える場合は世代IDを付け、同じバージョンであることを確認する。

1. DatetimeIndexをdateカラムに移す。
2. タイムゾーンを外す(tz_localize(None))。
3. カラム名を小文字に揃える(open -> open)。
4. 月ごとにグループ化し、YYYY-MM.parquetファイルに保存する。

該当月のファイルがすでに存在する場合は、新しいデータを既存データに結合し、日付の重複を除いてから書き戻す。同じダウンロードを何度実行しても、重複が積もることはない。

2.4 tz_localize(None) —このプロジェクトの暗黙のルール

yfinanceや一部のAPIは、UTCタイムゾーンが付いたDatetimeIndexを返してくる。これをそのまま放置すると、二つの問題が起きる。

問題1 —pandas concatのエラー: タイムゾーン付きのDataFrameと付いていないDataFrameを結合しようとすると、`TypeError: Cannot join tz-naive and tz-aware DatetimeIndex`が発生する。

問題2 —DuckDB保存エラー: DuckDBのTIMESTAMP型は基本的にtimezone-naiveを受け取る。tz-aware値をそのまま送ると変換に失敗する。

正しいパターン

ダウンロード直後、他の処理を挟む前に1行入れておく。

```
def download_ohlc(symbol, interval="1d"):
    df = yf.download(symbol, period="5y", interval=interval,
                      auto_adjust=True)
    idx = pd.to_datetime(df.index)
    if idx.tz is not None:
        idx = idx.tz_convert("UTC").tz_localize(None)
    df.index = idx
    df.columns = [c.lower() for c in df.columns]
    return df
```

確認すること: `df.index.tz`がNoneになっていること。UTCのままだと、後続の処理がすべて壊れる。

2.5 特徴量エンジニアリング—4次元の日足特徴量

HMMは株価そのものを入力として受け取らない。株価から計算した派生値、つまり特徴量(feature)を入力に取る。

4つの特徴量が見ているもの

次元	意味	何を表しているか
0	1日対数リターン	昨日に比べて今日の価格がどれだけ動いたか
1	5日実現ボラティリティ	直近の価格がどれだけ揺れているか
2	20日モメンタム	1ヶ月前と比べてトレンドがどちら向きか
3	出来高比率	普段に比べて取引がどれだけ活発か

次元0・3のように素早い変化を捉えたい特徴量ではQ/Rを相対的に大きく、次元1・2のようにノイズを抑えたい特徴量では小さくする設計を比較できる。Qは状態変化の分散、Rは観測雑音の分散である。両者の単位は対象特徴量の単位の二乗なので、生の価格用の数値を標準化リターンへ流用しない。第6章の予測・更新式とイノベーション残差を基準に学習区間で設定を選び、OOSでは固定する。

中核となる計算—次元0 (対数リターン)

```
import numpy as np

def compute_daily_features(close, volume):
    close = np.asarray(close, dtype=float)
    volume = np.asarray(volume, dtype=float)
    if close.ndim != 1 or volume.shape != close.shape:
        raise ValueError("equal-length 1D arrays required")
    if not np.all(np.isfinite(close)) or np.any(close <= 0):
        raise ValueError("close must be finite and positive")
    if not np.all(np.isfinite(volume)) or np.any(volume < 0):
        raise ValueError("volume must be finite and nonnegative")
    n = len(close)
    feat = np.full((n, 4), np.nan)
    ret = np.full(n, np.nan)
    ret[1:] = np.diff(np.log(close))
    feat[:, 0] = ret
    for t in range(n):
        if t >= 5:
            feat[t, 1] = np.std(ret[t-4:t+1], ddof=1)
```

```

    if t >= 20:
        feat[t, 2] = close[t] / close[t-20] - 1
        base = np.mean(volume[t-20:t])
        if base > 0:
            feat[t, 3] = volume[t] / base
    return feat

features = compute_daily_features([100.0] * 25, [10.0] * 25)
assert features.shape == (25, 4)
assert np.allclose(features[20], [0.0, 0.0, 0.0, 1.0])
assert np.isfinite(features).all(axis=1).sum() == 5

```

このNumPy関数は5returnの標本標準偏差を年率化せず使う。momentumには20足前の終値、volumeは当日を除く前20足を使う。初めの20行は未準備で、各行は終値確定後だけ使う。欠測を正常な0へ置き換えてはいけない。

```

# Incorrect/unsafe example.
X = np.nan_to_num(features, nan=0.0)

```

重要な行: `np.maximum(close, 1e-10)` は、終値が0や負の値になったときに $\log(0)$ になるのを防ぐための安全弁である。

欠損値—モデルに渡してはいけない空欄

rolling windowで作る特徴量は、十分な過去行がそろうまで計算できない。たとえば20日モメンタムは3日目には存在しようがない。こうした初期行はNaNになり、HMMや多くの機械学習モデルはNaNを含む行をそのまま読めない。

確認する出力: `feat.shape == (T, 4)` であり、先頭19行はNaNになっている(20日モメンタムが一番遅く埋まるため)。NaN行を除いてからモデルに渡す必要がある。

2.6 カルマン平滑化—ノイズを抑える一行の補助ツール

上記の特徴量はノイズが多い。特に出来高比率は1日ごとに大きく跳ねる。これを滑らかに均し、モデルがトレンドをつかみやすくする道具が**カルマンフィルタ (Kalman Filter)**だ。

直感だけ覚えておけば足りる。

- ・ 毎日新しい観測値が入るたびに、「これまでの推定値」を少しずつ更新していく。
- ・ 観測値をどれくらい信じるか(R)、信号そのものがどれくらいゆっくり変化すると仮定するか(Q)、この二つのパラメータで調整する。
- ・ Rが大きい -> 観測値をあまり信じない -> 平滑化が強くなる(線が滑らかになる)。
- ・ Rが小さい -> 観測値をより信じる -> 元の値に近くなる。

1次元カルマン更新は、予測分散 $P_{pred}=P+Q$ 、ゲイン $K=P_{pred}/(P_{pred}+R)$ 、推定値 $x=x+K*(z-x)$ 、分散 $P=(1-K)*P_{pred}$ の順である。最初の有効な観測で x を初期化し、 $P \cdot Q \cdot R$ を有限の正数にする。観測がNaNなら観測更新を省略し、欠測状態を別途記録する。観測が長期間ない場合、平滑値が存在しても取引入力に使わない。以上が収集・保存・特徴量・平滑化の基本的な流れである。

2.7 時間足の特徴量—6次元への拡張

時間足の戦略は4次元では足りないので6次元を使う。追加される2次元だけ書いておく。

追加次元	意味
Dim 4	バーの約定方向比率—その時間内で価格がどちら向きに終わったか

追加次元	意味
Dim 5	VWAP乖離—平均約定価格に対して終値がどれくらい上/下にあるか

二つの次元とも判断時点までに確定した観測だけを使う。例えば5分リターンは連続する終値の比から、実現ボラティリティは過去のリターン窓から計算する。履歴が十分に蓄積するまでは0で埋めて正常入力として扱わず、ウォームアップ中とする。同じ本数でも日中足と日足では経過時間が異なる。

2.8 Zスコア—平均回帰のシグナル

平均回帰 (Mean Reversion) 戦略のエントリーシグナルは**Zスコア**である。「今の価格は直近の平均から標準偏差で何個分離れているか」を見る。

```
def compute_zscore(close, window=20):
    T = len(close)
    z = np.full(T, np.nan, dtype=np.float64)
    for i in range(window, T):
        win = close[i - window: i]
        mu = float(np.mean(win))
        sigma = float(np.std(win))
        if sigma > 1e-10:
            z[i] = (close[i] - mu) / sigma
    return z
```

重要な行: ウィンドウのインデックスは `i - window : i` であり、現時点 `i` を含めていない。未来データを見ないための**因果性 (causality) 保証**である。

解釈: `- z > +1.5` -> 買われすぎの状態 -> ショートエントリー候補 `- z < -1.5` -> 売られすぎの状態 -> ロングエントリー候補

日足は `window=20` (およそ1ヶ月)、時間足は `window=140` を使う。

2.9 モデル入力へ渡す—実戦パターン

ここまでで作った部品を組み合わせ、HMMにデータを渡すまでの全体フローを見る。

```
from app.data.download import downloadBars
from app.trading.backtest.features import
compute_daily_features

data = downloadBars(["AAPL"], days=730, interval="1d")
df = data["AAPL"]

feat = compute_daily_features(df["close"].values,
df["volume"].values)

valid = ~np.isnan(feat).any(axis=1)
X = feat[valid]                                # shape: (T_valid, 4)

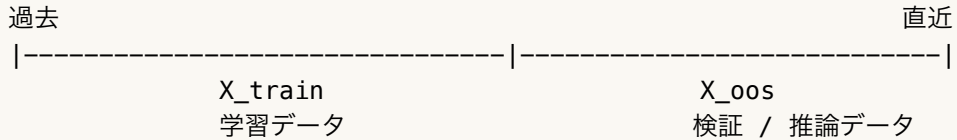
train_len = int(len(X) * 0.7)
X_train = X[:train_len]
X_oos    = X[train_len:]
```

確認する出力: - `X.shape[1] == 4` (特徴量の次元) - `X.shape[0] < len(df)` —NaN行が落ちているので、元より短い - `len(X_train) + len(X_oos) == len(X)` - 時系列を絶対にシャッフルして分割してはいけない。前から後ろへの時間順を維持する。

失敗時に確認すること: HMMの`fit()`で`ValueError: Input contains NaN`が出たら、ステップ3のNaN除去を忘れている。

この分割を時間軸で描くと次のようになる。

X = モデルが読む特徴量行列



```
scaler.fit(X_train)
```

```
hmm.fit(X_train)
```

```
for x in X_oos:
```

```
    proba = hmm.step_forward(x)    # 判断だけ。再学習はしない
```

重要なのは、X_oosをfit()に入れないことである。X_oosは過去ファイルの中にあるが、検証中はまだ到着していない未来データのように扱う。モデルはX_trainから規則を学び、その後X_oosを1行ずつ受け取って推論だけを行う。

Exercises

1. `download_bars('AAPL', days=365)` を初めて実行したときと、2回目に実行したときで動作がどう違うかを説明せよ。Parquetファイルがディスクのどこに生成されるかを自分の目で確認すること。
2. 現在のパーティション基準はsymbol / interval / 年月である。これをsymbol / interval / 四半期に変えると、どんなクエリが速くなり、どんなクエリが遅くなるか？ それぞれ具体例を挙げよ。
3. `tz_localize(None)` の一行をわざと抜いて、yfinanceダウンロード -> DuckDB保存を試みて、実際のエラーメッセージを再現してみよ。

まとめ

この章で組んだパイプラインは二つの仕事をしている。APIから受け取ったOHLCV(始値・高値・安値・終値・出来高)データをParquetファイルに保存し、そのデータをモデルが読める特徴量行列に変換することだ。成果物は次章のBacktest Engineにそのまま渡せる。

この章で必ず覚えておくこと

- ・ `download_bars()` はデータを取得する唯一の窓口である。FMPが失敗すると `yfinance` や DuckDB キャッシュへ自動で切り替わる。
- ・ DuckDB キャッシュは銘柄・インターバル・年月の単位で Parquet ファイルを分けて保存する。必要なファイルだけを選んで読むので速い。
- ・ `yfinance` から受け取ったデータは、`tz_localize(None)` でタイムゾーンを必ず外す。
- ・ 日足モデルの入力は4次元の特徴量(リターン・実現ボラティリティ・モメンタム・出来高比率)の行列である。カルマンフィルタはその上に重ねるノイズ減衰の道具だ。
- ・ Zスコアは現時点を含まない過去のウィンドウだけで計算する。未来データを覗くと、バックテストは嘘をつく。

次章ではBacktest Engineを作る。Portfolio Simulatorの`on_bar()` ループ、**ウォークフォワード検証**(時間順にデータを区切って過学習を防ぐ方法)、リターン・Sortino・MDDといった成績指標をコードで確認していく。

さらに読むには

- ・ 公式ドキュメント: [DuckDB Python API](#)
- ・ ライブラリ: `yfinance` GitHub —ダウンロード失敗時の自動リトライ設定を参照

第3章. バックテストエンジン—信頼できるシミュレーション

「バックテストでは良かったのに、本番では通用しない」

クオンツコミュニティではジョーク交じりに語られるが、もっとも多く、もっとも代償の大きい失敗だ。バックテストは過去データで戦略を検証する手法である。過去に通用したことを示すだけで、未来でも通用する保証ではない。

多くのバックテストは現実を楽観的に仮定しすぎている。スリッページ(注文しようとした価格と実際の約定価格の差)もなく、手数料も実際より低い。マーケットインパクト(大口注文が価格を押し動かす現象)もない。こうした条件で出た成績は、現実ではなく理想化された仮想環境の結果だ。

信頼できるバックテストエンジンには三つの要素が欠かせない。

第一に、**ルックアヘッドバイアス (look-ahead bias) の防止**だ。シグナルを作るときは、その時点で実際に知り得たデータだけを使う必要がある。未来のデータが混ざると、戦略は実態より良く見える。

第二に、**取引コストを現実的に反映**することだ。スプレッド、スリッページ、手数料を漏れなく含める。

第三に、**流動性の検証**だ。注文サイズが市場で実際にさばける出来高を超えていないかを確認する。

どれか一つでも欠けると、バックテスト結果は実態より良く見える。

この章では、これら三つを備えたエンジンを段階的に作っていく。

この章の流れ

まずバックテストが足(バー)ごとに何をしているか、全体像を見る。次に中核コードを読み、パフォーマンスを数値でどう評価するかを確認する。

- ・ 3.1 Big Picture —足ごとに回る5段階ループ
- ・ 3.2 中核コード—`simulate()`
- ・ 3.3 ポジションを閉じるときのPnL計算
- ・ 3.4 パフォーマンス指標—`_build_result()`
- ・ 3.5 PnL Gini —集中度の指標
- ・ 3.6 DSR(Deflated Sharpe Ratio) —多重検定の補正
- ・ 3.7 PortfolioConfig —戦略別の推奨設定
- ・ 3.8 パフォーマンス指標の読み方

学習目標

この章を読み終えると、次のことができるようになる。

- ・ `simulate()` が足ごとにどの順序で処理を行うか説明できる。
- ・ OOS(Out-of-Sample) 60/40 splitがコードのどこにあるかを指し示せる。
- ・ シャープレシオ、MDD、Win Rateがそれぞれ何を測っているかを一言で言える。
Sortino/Calmar/Omega/VaRはカタログ程度に把握する。

- ・ DSR(Deflated Sharpe Ratio) がなぜ必要かを説明し、p 値を基準に解釈できる。

この章で作るもの: 足ごとに決まった順序でシグナル -> 約定 -> 評価を処理するシミュレータを作る。「バックテストでは良かったのに、本番ではなぜ通用しないのか」 — この章はその問いに構造的に答える。

3.1 signal・約定・資産の時間順序

学習と評価を時系列で分ける。60/40は導入例で保証ではない。OOSを見て反復調整すれば学習に使ったことになる。別検証と最終holdoutまたはwalk-forwardを使う。

過去でfitし、初期cash・quantity・equityを記録する。前終値後のtargetを今足の始値で約定し、cash・数量・費用を更新する。最終足なら清算し、終値で資産評価、差分損益を計算し、今までの情報で次targetを決める。確定終値で作ったsignalをその同じ終値で既に約定できたと仮定しない。

3.2 実行できる損益会計の実習

関数と確認codeをbacktest.pyへ保存する。Python標準libraryだけで、broker・CSV・HMMは不要だ。任意のaccounting-lab archiveには同じ関数と追加test、CLIがある。cashは通貨、quantityは符号付き数量、equityはcash+quantity*closeで、足損益は資産差分である。含み損益の省略や清算時の累積損益再加算は結果を歪める。targets[t]はclose[t]後に既知となりopen[t+1]で約定する。因果的なtarget生成はsignal側の責任だ。最終清算と費用を最後の資産記録より先に処理する。

```
import math

def simulate(
    bars, targets, initial=1000.0, fee_bps=0.0, slippage_bps=0.0
):
    """Fill targets[t], known after close[t], at open[t+1].
```

Use fixed unit size.

*cash is currency; quantity is signed units; equity = cash + quantity*close.*

Omit financing, margin, taxes, dividends and market impact.
 """

```

if len(bars) != len(targets) or not bars:
    raise ValueError("Provide one target per nonempty bar
sequence")
if not math.isfinite(initial) or initial <= 0:
    raise ValueError("initial must be positive and finite")
if any(
    not math.isfinite(x) or x < 0 or x >= 10000
    for x in (fee_bps, slippage_bps)
):
    raise ValueError(
        "costs must be finite and between 0 and 10000 bps"
    )
if any(t not in (-1, 0, 1) for t in targets):
    raise ValueError("target must be -1, 0 or 1")
if any(
    not math.isfinite(b[k]) or b[k] <= 0
    for b in bars
    for k in ("open", "close")
):
    raise ValueError("prices must be positive and finite")
cash, quantity = initial, 0.0
unit = initial / bars[0]["open"]
equity, pnl, fills = [initial], [], []

def trade(delta, price, index, reason):
    nonlocal cash, quantity
    execution = price * (
        1 + math.copysign(slippage_bps / 10000, delta)
    )
    fee = abs(delta) * execution * fee_bps / 10000
    cash -= delta * execution + fee
    quantity += delta

```

```

fills.append(
    {
        "bar": index,
        "quantity": delta,
        "price": execution,
        "fee": fee,
        "reason": reason,
    }
)

for i, bar in enumerate(bars):
    desired = targets[i - 1] * unit if i else 0.0
    delta = desired - quantity
    if delta:
        trade(delta, bar["open"], i, "previous_close_target")
        # Liquidate before recording the last equity sample.
        # Include the final fill cost.
        if i == len(bars) - 1 and quantity:
            trade(-quantity, bar["close"], i,
                "final_liquidation")
            current = cash + quantity * bar["close"]
            pnl.append(current - equity[-1])
            equity.append(current)
    peak, mdd = initial, 0.0
    for value in equity:
        peak = max(peak, value)
        mdd = max(mdd, (peak - value) / peak)
    assert math.isclose(initial + sum(pnl), equity[-1],
        abs_tol=1e-8)
    return {
        "equity": equity,
        "bar_pnl": pnl,
        "final_equity": equity[-1],
        "return": equity[-1] / initial - 1,
        "mdd": mdd,
        "fills": fills,
        "fees": sum(f["fee"] for f in fills),
    }

```

次も同fileへ追加しpython3 backtest.pyで実行する。

```
bars = [
    {"open": 100.0, "close": 100.0},
    {"open": 100.0, "close": 80.0},
    {"open": 80.0, "close": 100.0},
]
result = simulate(bars, [1, 1, 0])
assert result["equity"] == [1000.0, 1000.0, 800.0, 1000.0]
assert math.isclose(result["mdd"], 0.2)
print(result["final_equity"], result["mdd"]) # 1000.0 0.2
```

3.3 費用と最終清算を一度だけ数える

価格100->80->100、初期1000なら最終gross損益0でもdrawdownは20%。初期基準を含む資産は[1000,1000,800,1000]で、最終cashだけでは損失経路が消える。

10単位を100で売買し各10bpなら費用2、最終998。参入費用を引いた後で往復分を再度引かない。この固定数量例は費用でcashが少し負になり得て、現金だけで買える数量の制約は実装しない。

simulate(bars, [1,1,0], fee_bps=10, slippage_bps=5)で費用を変えられる。任意archiveはunittestとCLIを追加する。買いは高く、売りは低く滑る。0/5/10/25bpは感度確認で普遍的適正費用ではない。

この小engineはfinancing、margin、税、企業行動、市場impactを含まず、運用portfolioやbroker接続ではない。

3.4 パフォーマンス指標—_build_result()

バックテストが終わると、結果ビルド段階でパフォーマンス指標を計算し、結果オブジェクトとして返す。最初の通読で必ず押さえておきたい指標は三つだ。

- ・ **シャープレシオ**: リスク(ボラティリティ)1単位あたりいくら稼いだか。高いほど良い。

- ・ **MDD(Maximum Drawdown)**: 資産が高値から最大でどれだけ落ち込んだか。MDDが深いと、シャープレシオが良くても実戦で耐えられない。
- ・ **Win Rate**: 全取引のうち勝ったものの比率。コイン投げ(50%)を超えているかをまず確認する。

NOTE

ひとつの指標だけで戦略を判断するのは危険だ。シャープレシオだけでは損失の深さがわからないし、MDDだけでは収益が安定しているかわからない。チェックリストのように一緒に見る。

リスク調整後リターン(シャープレシオ)

平均収益だけを見ると戦略を簡単に誤解する。日々大きく揺れながら稼いだ収益と、小さく揺れながら稼いだ収益は別物として扱うべきだ。だから結果ビルド段階では、収益率とボラティリティをセットで見る。

最初の通読では計算式を暗記する必要はない。要点は**収益が大きくても、ボラティリティと最大下落が同時に大きいなら良い戦略とは見なさない**ことだ。具体的な計算式はリポジトリの結果ビルド関数に任せ、本書では解釈に集中する。

MDDとWin Rate

MDDは高値から安値までの最大下落比率、Win Rateはプラスの PnL を出した取引の比率だ。どちらも単純な計算なので、コードはリポジトリの結果ビルド関数を参照する。

$$\text{MDD} = \max_t \left(\frac{\text{Peak}_t - \text{Value}_t}{\text{Peak}_t} \right)$$

さらに踏み込んだ指標(カタログ)

同じデータを別の角度から見るための指標群。最初の通読では名前と意味だけを押さえ、実装はリポジトリに任せる。

指標	一言の意味
----	-------

Sortinoは平均超過return/下方偏差、Calmarは年率return/正MDD、Omegaは基準を上回る利得/下回る不足だ。分母0は未定義とし、return・期間・符号を揃える。| Calmar | 年率収益 ÷ |MDD| 一下落幅に対する収益 | | Omega | プラスの収益の合計 ÷ マイナスの収益の絶対値の合計 | | VaR(95%) | 日次収益率の下位5%地点—まれに起こる大きな損失の大きさ | | CVaR(95%) | VaRの下5%区間の平均損失 | | Profit Factor | 総利益 ÷ 総損失 | | PnL Gini | 銘柄別の収益集中度(3.5) |

3.5 PnL Gini —集中度の指標

PnL Giniは、収益が特定の銘柄にどれだけ偏っているかを測る。0であればすべての銘柄が均等に貢献していて、1に近いほど一つの銘柄がほぼ全ての収益を稼いだ状態になる。実戦の目安は < 0.4 。これを超えると、一つや二つの銘柄が消えた途端に戦略全体が崩れる。計算式はリポジトリの結果ビルド関数を参照する。

3.6 DSR(Deflated Sharpe Ratio) —多重検定の補正

バックテストを多く回すほど、偶然良く見える結果が出る確率は高くなる。**Deflated Sharpe Ratio(DSR)**はこの偏りを補正する指標だ。

戦略をひとつだけ試して良い結果が出たなら、その結果は検討に値する。だが500個の戦略を試してから一番良いものを選んだのなら、その戦略は単に運が良かった可能性のほうが圧倒的に高い。DSRは「何回試したか」を反映して、結果の信頼度を割り引いて見るための装置だ。

`n_trials`には最終候補の数だけを入れてはいけない。途中で捨てたアイデア、変えてみたパラメータの組み合わせ、形を変えて再実行した実験まで、すべて含めて初めてDSRが過剰最適化のリスクを正しく反映する。

```

from app.trading.backtest.shared import
compute_deflated_score_pvalue

dsr_pval = compute_deflated_score_pvalue(
    score_obs      = score,          # 観測された成績
    n_obs          = len(bar_ret),   # 足の本数
    n_trials       = n_strategies_tested, # 試した戦略の数
    skew           = sk,
    kurtosis_excess = ku,
)

```

p値	解釈
< 0.005	統計的に強く有意 (採択)
0.005 ~0.05	有意 (検討のうえ採択)
> 0.05	偶然の可能性あり (棄却)

数式の導出など統計的な背景は、Bailey & Lopez de Prado (2014) の論文を参照する。

3.7 PortfolioConfig —戦略別の推奨設定

戦略によって適切なポジション数とサイズは異なる。トレンドフォローのように少数の銘柄に大きく賭ける戦略と、ペアトレーディングのように複数のペアに分散する戦略では、設定がそもそも違ってくる。各シグナルは自身の性格に合った推奨設定を返し、シミュレータがそれをそのまま使うか、既定値で上書きする。

主 な フ ィ ー ル ド (max_positions、per_trade_alloc、commission、is_pair_strategy、rebalance_freq) や、戦略別の推奨値の比較表は分量が長くなるため、付録とリポジトリ (PortfolioConfig、各シグナルの recommended_config()) に譲る。最初の通読では「戦略ごとに設定値を変えられる」という事実だけ覚えておけばよい。

3.8 パフォーマンス指標の読み方

max_positionsは同時上限、per_trade_allocは一取引比重、commissionは約定費率、is_pair_strategyは両脚連動、rebalance_freqは周期。5 枠×10%なら新規配分50%。pairの10%が両脚合計か各脚かを明示し、口座全体riskを優先する。

指標	問い
シャープレシオ	収益はボラティリティに対して誇張されていないか
Sortino	損失方向のボラティリティは耐えられる範囲か
MDD	最悪の下落幅を実際の口座で耐えられるか
Calmar	下落幅に対して収益は十分か
Omega	収益分布が極端に片寄っていないか
Win Rate	シグナルがランダムにしか見えない状態ではないか
Profit Factor	総利益が総損失を十分に上回っているか
PnL Gini	収益が一つや二つの銘柄に偏っていないか
DSR p値	多数の実験のなかから偶然良かったものだけを選んでいないか

本書で扱う戦略の結果も、以上の問いで読む。特定の数字だけを目標にせず、下落幅・取引コスト・サンプル外の成績・収益の集中度を併せて確認する必要がある。

Exercises

1. commission=0とcommission=0.001で同じ戦略をそれぞれバックテストせよ。二つの結果について、シャープレシオと年間取引回数を比較する。取引コストが実際にかかる、シャープレシオがどれほど下がるかを数字で確認する。
2. on_bar() ループは通常、シグナル計算 -> ポジション判定 -> 約定 の順で実行される。この順序を変えると Look-ahead Bias が生じる。Look-ahead Bias とは、まだ知り得ない未来のデータをすでに参照したかのようにコードが振る舞ってしまう

バグだ。順序を誤って配置したコード例を自分で書き、どの時点でバイアスが発生するかを説明せよ。

3. 会計演習の資産曲線に日付を加え、年別リターンを計算せよ。どの年が最も良かったかを確認し、年間リターンとリスク調整後指標を区別すること。

まとめ

この章では、因果性違反のないバックテストエンジンと、主要なパフォーマンス指標の計算を仕上げた。今後すべての戦略はこのエンジンを通して検証する。

この章で必ず記憶しておくこと:

- ・ **OOS分割**が評価の出発点だ。前60%で学習し、後ろ40%でだけ収益率を測る。
- ・ **一本の足の中の順序が重要**: 手仕舞い -> 評価 (MTM) -> エントリー。順序を変えれば結果は歪む。
- ・ **手数料はエントリーと決済の両側**で差し引かれる ($2 * \text{commission}$)。
- ・ **シャープレシオひとつだけを見てはいけない**。シャープレシオ (収益対ボラティリティ)、MDD (最悪の下落幅)、Win Rate (勝ち取引の比率) を併せて見る。
- ・ **DSR**は「何度も試したから偶然良く見えているもの」を篩い落とす。p値 < 0.05 を通過することが条件だ。

次章では、HMM (Hidden Markov Model) の基礎理論を扱う。HMMが生み出す売買シグナルも、結局はこの章のエンジンを通して初めて価値を認められる—モデルが何であれ、評価の物差しは同じだ。

さらに読むには

- ・ **書籍**: Ernest Chan, Algorithmic Trading (Wiley, 2013) —バックテストエンジンの設計と取引コストモデリング
- ・ **論文**: “The Deflated Sharpe Ratio” (Bailey & Lopez de Prado, 2014) —多重検定補正後のシャープレシオの統計的有意性
- ・ **ライブラリ**: vectorbt —Numbaベースの高性能ベクトル化バックテストエンジン

勝率は約定数ではなく完了した往復取引で計算する。Sharpeは同じ時価資産曲線の return と足周期に合う年率化係数を使う。分散0や履歴不足は0点ではなく未定義である。

第4章 HMMの基礎— 市場レジームを確率で 推論する

Part II —HMMの基礎(4～5章) このパートでは、市場を「固定された一つの状態」ではなく、複数のレジームが切り替わっていく過程として捉える。4章ではHMMの基本構造、5章では将来データを見ないForward-only推論を扱う。最初の通読では、数式よりも「現在のレジームを確率で推定する」という感覚をまず掴んでほしい。

価格、出来高、移動平均、RSI、MACDはチャートから直接読める。しかし、これらの数字の背後にあるもの—市場が今どんな状態にあるか—は目に見えない。トレンド相場、レンジ相場、高ボラティリティ相場といった区別こそが、この「隠れた状態(hidden state)」だ。

経験豊富なトレーダーは、この状態を直感で判断する。「今はトレンド相場だからRSIは無視すべきだ」「今はレンジ相場だからモメンタム戦略は機能しない」といった具合だ。問題は、その直感が人によって違ううえ、一貫性がないという点にある。システムに落とし込むのが難しい。

HMM(隠れマルコフモデル、Hidden Markov Model)は、この直感を数学に置き換える。HMMが答える問いはただ一つ。「観測可能なデータ(リターン、ボラティリティなど)が与えられたとき、いま市場が各レジームにある確率はどれくらいか?」価格そのものを当てに行くのではなく、価格を生み出す構造であるレジームをまず推論する、という考え方だ。

最初に読むとき、数式をすべて覚える必要はない。この章で押さえるべき核心の感覚は次の3つだ。

1. HMMは「価格は上がるか?」を直接問わない。「いま市場はどの状態にあるか?」をまず問う。
2. 状態は目に見えない。だからリターン、ボラティリティ、出来高といった観測値から確率を推定する。
3. 結果は唯一の正解ではなく確率ベクトルだ。たとえば [MR 70%, Trend 20%, Volatile 10%] のような形で出てくる。

HMMには3つのパラメータがある。まず名前だけ押さえておこう。

- ・ **A** —状態間の遷移確率表(「状態がどれくらい長く続くか」)
- ・ **B** —状態ごとのデータ分布(「各状態でどんなデータが出てくるか」)
- ・ **π** —初期状態確率(「最初に始めるときどの状態とみなすか」)

この章の数式は「モデルを自分で導出するための数学」ではなく、「コードがなぜそういう形をしているのかを読むための地図」だ。GaussianHMM(n_components=3)を呼び出せば、ライブラリがほとんどを計算してくれる。ただし、どの特徴量を入れるか、状態をいくつにするか、レジーム番号をどう振り直すかは人間が決める必要がある。この判断を下すには、A、B、 π が何を意味するかを知っておかなくてはならない。

HMMを機械学習モデルとして見る

HMMという名前は耳慣れないかもしれないが、機械学習の流れに当てはめれば構造はとてもシンプルだ。次の4段階で整理できる。

1. **入力特徴量を作る:** 価格そのものではなく、リターン・ボラティリティ・モメンタム・出来高比率に変換する。
2. **過去区間で学習する:** `fit()` が過去の特徴量を見て A 、 B 、 π を推定する。
3. **新しいバーごとに推論する:** 学習済みモデルに新しい特徴量を入れてレジーム確率を計算する。
4. **売買シグナルへつなぐ:** たとえばMR確率が高いときだけ平均回帰エントリーを許可する。

ここで `GaussianHMM` はアルゴリズムを収めた枠(クラス)であり、`fit()` が終わったオブジェクトがモデルである。`fit()` 前は、まだ市場を学習していない。`fit()` 後は、 A 、 B 、 π がデータに合わせて決まっているので、新しい特徴量を入れるとレジーム確率を返せる。

学習前:

```
GaussianHMM(n_components=3)
```

市場を3状態に分ける準備だけできた空の器

学習:

```
空の器 + X_train --fit()--> 学習済みHMMモデル
```

推論:

```
学習済みHMMモデル + 今日の特徴量
```

```
-> [MR 70%, Trend 20%, Volatile 10%]
```

たとえるなら、`GaussianHMM` はまだ目盛りを合わせていない市場判別器である。`fit()` は過去データを使って目盛りを合わせる作業だ。目盛りが合ったあとで初めて、今日のデータを入れて「いまはどんな市場か」と尋ねられる。

つまりHMMは、本書で最初に出会う「株式市場向けの機械学習モデル」である。ただし、「この日はMR、この日はTrend」という正解ラベルを与える教師あり学習ではない。どちらかと言えば**教師なし学習 (unsupervised learning)**に近い。「この日はMR、この日はTrend」と人間が正解ラベルをあらかじめ与えなくても、HMMは観測パターンが似た日を隠れ状態にまとめる。学習が終わったあとに、その状態へMR / Trend / Volatileという意味を付けるのは人間の仕事だ。

この違いを覚えておくと、後のRL章も読みやすくなる。HMMは市場状態を推定するモデルであり、SACはその状態と他の特徴量を受け取ってポジションサイズを決めるモ

デルである。どちらも**データ -> 特徴量 -> 学習 -> 推論**という流れの上にあるが、役割が違う。

HMMが出したレジーム確率は、この章だけで終わらない。強化学習の章では、この確率がSACエージェントの入力の一部になる。

```

HMMが見つけたレジーム
[MR 70%, Trend 20%, Volatile 10%]
    |
    v
RL状態ベクトルの一部
[z_score, position, pnl, MR_prob, Trend_prob, Volatile_prob,
...]
    |
    v
SACがポジションサイズを決める

```

だからHMMは単なるフィルタではない。後で出てくる深層学習エージェントへ「市場の背景」を伝えるセンサーでもある。HMMが市場の天気を読み、RLはその天気の中でどれくらい速く動くかを決める。

同じシグナルでもレジームが違えば意味が変わる

同じ $z\text{-score} = +2.5$ が出た場面を二つ想像してみよう。Z-scoreだけ見れば同じシグナルだが、HMMレジームを合わせて見ると意味が真逆になる。静かな平均回帰レジームなら、行きすぎた価格が戻るエントリー候補になるかもしれない。強いトレンドや危機レジームなら、逆に避けるべきシグナルかもしれない。HMMは、シグナルを解釈するための文脈を提供する役割を持つ。

シグナル	HMMレジーム	解釈
z-scoreが大きい	平均回帰レジーム	エントリー候補になり得る
z-scoreが大きい	トレンドレジーム	さらに広がる可能性があるので待つ

シグナル	HMMレジーム	解釈
z-scoreが大きい	高ボラティリティレジーム	エクスポージャーを減らす、または遮断する

これが本書の核心である。**単一の特徴量だけを見て売買するのではなく、まず市場状態を読み、その状態の中でシグナルを解釈する。**後のRL章も同じ考え方を引き継ぎ、同じZスコアでもレジームに応じて別のポジションサイズを学習できる。

この章の流れ

まずHMMがどんな問いに答えるかを掴み、そのあと3要素・コードの流れ・運用上の注意点を順に見ていく。

- ・ 4.0 全体像—hmm_daily_mr.py パイプライン
- ・ 4.1 HMMの3要素 (A , B , π)
- ・ 4.2 Baum-Welch —学習の直感
- ・ 4.3 `_DailyHMM.fit()` —実装とLook-aheadの注意
- ・ 4.4 分散ベースのレジーム整列
- ・ 4.5 `_DailyHMM.predict_proba()` —レジーム確率の計算
- ・ 4.6 `_update_hmm()` —10本ごとの再学習
- ・ 4.7 `_check_entry()` —Z-score + HMMゲート
- ・ 4.8 `_check_exit()` —6つの手仕舞い条件
- ・ 4.9 `PureZscoreDailySignal` —ベースライン比較
- ・ 4.10 全体の流れ—`on_bar()` の呼び出し順

学習目標

この章を読み終えると、次のことができるようになる。

- ・ HMMの3つのパラメータ (A , B , π) がそれぞれ何を表しているかを説明できる

- ・ Baum-Welchアルゴリズムが繰り返し学習でパラメータを見つけていく過程を直感的に説明できる
- ・ `covariance_type="full"` を選ぶ理由と `min_covar=1e-3` の役割を理解できる
- ・ 分散を基準に状態をMR / Trend / Volatile レジームへ自動割り当てするコードを読む
- ・ Z-scoreシグナルとHMMゲートがどう組み合わせさせてエントリーシグナルを作るのかを説明できる

最初の通読で押さえること

この章では、HMMという馴染みの薄いモデルを初めて扱う。最初に読むときは、すべてのオプションや定数を覚える必要はない。次の3つだけ掴めば十分だ。

- ・ HMMは価格を直接予測するモデルではなく、**いまの市場状態を確率で分けるモデル**である。
- ・ 本書ではHMMを売買シグナルとして直接使わず、**シグナルを通すか止めるかを決めるゲート**として使う。
- ・ Look-ahead Biasを避けるには、将来データが混ざらないようにレジーム確率を計算する必要がある。この部分は5章でさらに詳しく扱う。

`min_covar`、再学習周期、手仕舞い条件の表といった細部の設定は、2回目の通読で読めばよい。いまは「なぜHMMが必要なのか」と「どこに危険があるのか」に集中しよう。

この章で作るもの

Part 2では **HMM** を市場レジームの検出に適用する。レジーム(regime)とは、市場が今どんな性格で動いているかを指す言葉だ。たとえば「いまは静かなレンジ相場」「いまは方向性のあるトレンド相場」といった区別がレジームに当たる。

HMMは、このレジームを直接観測することはできないと仮定する。リターンやボラティリティといった数字だけを見て、いまどのレジームにあるかを確率的に推論していく。原則はシンプルだ。**将来データを使わず、これまで観測したデータだけで現在のレジームを判断する。**

観測データ	隠れた状態(レジーム)	意味
リターン低 / ボラティリティ低	REGIME_MR (0)	平均回帰に適する
リターン中 / ボラティリティ中	REGIME_TREND (1)	トレンド相場
リターン急変 / ボラティリティ高	REGIME_VOLATILE (2)	危険ゾーン

エントリーシグナルはREGIME_MR状態でのみ許可する。REGIME_VOLATILEに切り替わったら新規エントリーを止め、既にあるポジションは早めに手仕舞いする。

4.0 全体像—hmm_daily_mr.py パイプライン

コードを見る前に、全体の流れを掴んでおこう。OHLCVデータがレジーム確率ベクトルを経て売買シグナルへと変わる過程は4段階だ。バーごとにこの順序で実行され、10日ごとにHMMを再学習して市場構造の変化に適応させる。

Step	呼び出し	入力 -> 出力
1	<code>compute_daily_features()</code>	OHLCV生データ -> 4次元の特徴量行列
2	<code>_DailyHMM.fit()</code>	特徴量行列 -> 学習済みGaussianHMM
3	<code>_DailyHMM.predict_proba()</code>	HMM -> レジーム確率ベクトル [MR, Trend, Volatile]
4	<code>_check_entry()</code> / <code>_check_exit()</code>	確率ベクトル -> 買い / 空売り / 手仕舞いシグナル

ここで最も重要な点を押さえておこう。**HMMは売買シグナルを直接生成しない。**

HMMは「いま市場がどの状態か」を確率で教えてくれるだけだ。実際の売買判断はZ-scoreのような別のシグナルが担う。HMMは、そのシグナルを通すか止めるかを決める**ゲート**の役割を持つ。

たとえば平均回帰シグナルが発生しても、高ボラティリティレジームの確率が高ければHMMがエントリーを止める。シグナル生成とフィルタリングをこうして分けておくと、モデルが外したときにどの部分が原因かを切り分けやすい。

4.1 HMMの3要素—A, B, π

遷移行列 A

遷移行列は、レジームが次のバーでどのレジームへ変わるかを示す確率表だ。レジームが3つなら3×3の行列になり、各行の確率を全部足すと1になる。

たとえば $a[0][0] = 0.92$ であれば、平均回帰レジームだった市場が次のバーでも平均回帰レジームである確率が92%という意味だ。 $a[0][2] = 0.03$ なら、平均回帰から一気に高ボラティリティへ移る確率は3%ということになる。市場のレジームは通常1日で大きく変わることは少ないため、対角要素が0.9以上と高くなるケースが多い。

この行列のおかげでHMMは、1日の変動だけに過剰反応せず、レジームがある程度持続するという市場の特性を取り込める。

放出分布 B

各レジームでどんなデータが出やすいかを表す確率分布だ。簡単に言えば、「このレジームなら、こういうリターン・ボラティリティのパターンが出てきそうか？」を計算する。

Gaussian HMMでは、各レジームごとに平均(μ)と共分散(Σ)を持つ正規分布(Gaussian = 正規分布)を使う。高ボラティリティレジームなら大きなマイナスリターンや高いボラティリティが自然に現れ、平均回帰レジームなら低いボラティリティと小さなリターン変動が自然に現れる。新しいデータが入ってくると、HMMは各レジームの分布と比較して「いまどのレジームである可能性が高いか」を更新していく。

`covariance_type="full"` を使うと共分散行列は4×4のフルマトリクスとなり、特徴量間の相関も取り込める。詳しい比較は4.3節でもう一度扱う。

初期状態分布 π

データ開始時点で各レジームにいる確率だ。データが十分にあれば結果に大きく影響することはなく、`hmmlearn`が学習過程で自動的に推定する。最初の通読では「存在することだけ覚えておく」で十分だ。

4.2 Baum-Welch —学習の直感

HMMのパラメータ(A 、 μ 、 Σ 、 π)はどうやって求めるのか。問題は、レジームが隠れていて「ある日がどのレジームに属するか」が分からない点だ。だから直接計算することができない。

この問題を解く方法が **EM(Expectation-Maximization)** だ。アイデアはシンプルだ。

1. パラメータを適当に初期化する。
2. そのパラメータを前提に「各日がどのレジームに属する確率か」を計算する (E-step)。
3. この確率を重みとして、平均と分散を計算し直す (M-step)。
4. 新しいパラメータで、また2に戻る。
5. もう改善しなくなるまで繰り返す。

Baum-Welchは、このEMの考え方をHMMに合わせて実装したアルゴリズムだ。最初はほぼ無作為に固まっていたグループが、繰り返すごとにデータの構造を反映したレジームへと整っていく。収束の基準は **対数尤度** (モデルがデータをどれくらいうまく説明できているかを表すスコア) がそれ以上伸びなくなることだ。

数式レベルでさらに深く知りたい場合は、付録と `hmmlearn` の公式ドキュメントを参考にしてほしい。最初の通読では「繰り返しで少しずつ良いパラメータを探していく」という直感だけ掴めば十分だ。

ただし、EMには出発点によって違う答えへ収束する **局所最適(local optimum)** の問題がある。同じデータでも初期値が違えば結果が変わり得る、ということだ。だから4.3では `random_state=SEED` で乱数を固定し、再現性を確保する。

もう一つ、アルゴリズムが常に意味のあるレジームを見つけてくれるとは限らない。入力する特徴量が市場状態をうまく反映していなければ、HMMはノイズをもっともらしく3つに分けるだけになる。**モデルの名前よりも特徴量設計のほうが重要な理由がここにある**。リターン、ボラティリティ、モメンタム、出来高比率など、市場状態を説明できる入力を入れる必要がある。

4.3 _DailyHMM.fit() —実装

WARNING

Look-ahead Biasに注意—この章全体を貫く最大の落とし穴だ。

`fit(X_all)` のように **期間全体** のデータで学習したり、`predict_proba(X_all)` で全時点の確率を一度に求めたうえで過去の時点に使ったりすると、モデルが「未来を見ながら過去を当てに行く」状態になってしまう。学習と検証を分けるなら学習は `fit(X_train)` だけで行い、リアルタイム環境では現在時点の確率だけを使うべきだ。

5章の `forward_proba_causal()` は、この問題を本格的に解決するためのツールだ。本章ではひとまず「現在のバーより前のデータだけを見る」という原則を守る。

`fit()` は2つの仕事をする。

1. 入力特徴量を標準化 (StandardScaler) したあと、GaussianHMMを学習する。
2. 学習が終わったら、分散の小さい状態から順番にMR -> Trend -> Volatileの番号を振り直す。

主要な設定値は定数として分離してある。

定数	値	意味
HMM_N_STATES	3	状態(レジーム)の数
HMM_N_ITER	200	Baum-Welchの最大反復回数
HMM_MIN_DAYS	120	初回学習に必要な最小バー数
HMM_REFIT_DAYS	10	再学習周期(バー単位)
SEED	42	乱数固定 (EMの再現性確保)

学習関数の中心部分は次の通りだ。完全な実装は `app/trading/backtest/signals/hmm_daily_mr.py` で確認できる。

```

def fit(self, features: np.ndarray) -> bool:
    X = features[~np.any(np.isnan(features), axis=1)]
    if len(X) < 60:
        return False

    scaler = StandardScaler()
    Xs = scaler.fit_transform(X)

    model = GaussianHMM(
        n_components=HMM_N_STATES,
        covariance_type="full",
        n_iter=HMM_N_ITER,
        random_state=SEED,
        tol=1e-4,
        min_covar=1e-3,
    )
    model.fit(Xs)

    total_var = [float(np.trace(model.covars_[i])) for i in
range(HMM_N_STATES)]
    order = np.argsort(total_var)
    self._state_map = {
        int(order[0]): self.REGIME_MR,
        int(order[1]): self.REGIME_TREND,
        int(order[2]): self.REGIME_VOLATILE,
    }
    self.model, self._scaler = model, scaler
    return True

```

読むときの核は2行だ。

- ・ **(3) スケーラは訓練データだけに fit_transform する。** 検証区間まで一緒に入れると Look-ahead Bias が入り込む。
- ・ **(6) Baum-Welch 実行** —ここで EM の反復が走る。ライブラリが内部で処理してくれる。

失敗時に見るところ: (2) でFalseが返ったらデータ不足、(6) で例外が出たら分散が小さすぎるか、データが偏っているケースだ。呼び出し側は fit に失敗したら直前のモデルをそのまま使う。

再学習周期の固定に注意: デフォルトは `HMM_REFIT_DAYS=10`。2022年の金利急騰のように市場構造が短期間で大きく変わる場面では、10日の周期は長すぎることもある。VIXが30を超えたときやレジーム転換が検知されたときに自動で再学習する仕組みを検討してもよい。

covariance_type="full" を使う理由

タイプ	特徴	いつ使うか
"diag"	特微量間が独立だと仮定、速い	特微量の次元が高い、データが少ないとき
"full"	特微量間の相関も取り込む	特微量が少なく、相関に意味があるとき

ここでは特微量が4個なので、"full" で学習してもパラメータ数は30個ほどに収まる。さらに、リターンが高いときにボラティリティが下がるといった負の相関のように特微量間に意味ある関係があるため、"full" を選んだ。

min_covar=1e-3 の役割

学習中に特定のレジームへサンプルがほとんど割り当てられないと、共分散の値が0に近づき計算が不安定になる。`min_covar=1e-3` は、共分散がこの値より下に行かないように抑える **セーフティネット** だ。

共分散の下限が小さすぎると数値不安定を許し、大きすぎるとregimeの差をぼかす。`1e-3`は標準化特微量の候補であり普遍的最適値ではない。学習区間で候補とseedを比較し、特異共分散、収束失敗、極小の状態占有率を調べ、別の検証区間で選ぶ。

4.4 分散ベースのレジーム整列

Baum-Welchの学習が終わると、状態が3つ作られる。ただし、これらの状態には番号だけが振られていて、「この番号がどの市場局面なのか」をモデルが教えてくれるわけではない。**学習が終わったあとに、各状態の意味は人間が決めてやる必要がある。**

その基準にするのが **分散(variance)** だ。ここでは分散を「揺れの大きさ」を表す数字として見る。平均回帰相場(MR)は株価が平均近辺で小さく動く区間なので分散が低い。逆に急変動相場では分散が最も大きくなる。

数式で言えば、各状態の共分散行列の対角要素を全部足した値(trace、トレース)がその状態の総分散だ。この値をもとに整列すればよい。

分散の順位	割り当てるレジーム
最も低い	MR(平均回帰)
中間	Trend(トレンド)
最も高い	Volatile(急変動)

コードの中心は次の部分だ。

```
total_var = [np.trace(model.covars_[i]) for i in range(3)]

order = np.argsort(total_var)
state_map = {
    int(order[0]): _DailyHMM.REGIME_MR,
    int(order[1]): _DailyHMM.REGIME_TREND,
    int(order[2]): _DailyHMM.REGIME_VOLATILE,
}
```

なぜ平均リターンではなく分散で整列するのか？

直感的には「平均リターンが最も高い状態をトレンド、最も低い状態を危険」と分類したくなる。しかしこの基準には大きな問題がある。平均リターンは符号がプラスとマイ

ナスを行き来するため、下落トレンドと上昇トレンドが平均値では正反対の位置に置かれてしまい、両者が同じ「トレンド」という本質を共有していることを表現できないのだ。分散は符号と無関係に「揺れの大きさ」を捉えるので、トレンド相場でも暴落相場でも一貫して分類できる。さらに、売買判断で大事なのは「方向性」ではなく「いまの市場が平均回帰戦略の効くほど静かか」であり、その答えを直接教えてくれるのは分散だ。

レジームの意味まとめ:

レジーム	定数値	市場の意味	シグナル処理
REGIME_MR	0	狭いレンジ、反転チャンス	エントリー許可
REGIME_TREND	1	トレンド相場	ゲート条件次第
REGIME_VOLATILE2		急変動、危険	エントリー遮断 + 早期手仕舞い

4.5 _DailyHMM.predict_proba() —レジーム確率の計算

predict_proba() を呼ぶと、各時点ごとに3つのレジームの確率がベクトル形式で出てくる。

```
# 例: 3本 × 3レジーム
# [MR 確率, Trend 確率, Volatile 確率]
[[0.82, 0.11, 0.07], <- 平均回帰が強い -> エントリー許可
 [0.31, 0.55, 0.14], <- トrend優勢 -> エントリー待機
 [0.08, 0.13, 0.79]] <- 急変動 -> 新規エントリー遮断
```

この例ではMR確率がMR_THRESHOLD=0.60を超えたら、次のステップであるZ-scoreの確認に進む。4.7のエントリーゲートでは、最低条件としてmr_prob >= 0.3も別途確認する。

内部動作は3段階だ。

- 1. 入力特徴量を、学習時に作ったスケーラで標準化する。
- 2. hmmlearnで元の状態順の確率行列を取得する。

3. `_state_map` を使って、列の順序を [MR, Trend, Volatile] の順に並べ替える。

```
def predict_proba(self, features: np.ndarray) ->
Optional[np.ndarray]:
    X = features[~np.any(np.isnan(features), axis=1)]
    if self.model is None or len(X) < 3:
        return None

    Xs = self._scaler.transform(X)
    raw = self.model.predict_proba(Xs)

    mapped = np.zeros_like(raw)
    for raw_s, logical in self._state_map.items():
        mapped[:, logical] = raw[:, raw_s]
    return mapped                                     # 0=MR, 1=Trend,
2=Volatile
```

確認すべき出力: 返ってきた行列の各行は合計1になる。`mapped[-1]` が最新バーのレジーム確率だ。

失敗時に見るところ: `self.model is None` なら学習がまだか、`fit` が失敗したケースだ。呼び出し側は `None` を受け取ったらエントリーシグナルを生成しない。

注意—4.3のLook-ahead警告はここにも当てはまる: `predict_proba(recent[-30:])` は最近30本までを入力として受け取るので、`mapped[-1]` だけを使えば現在より先のデータは見ない。ただし、`mapped` 行列の過去行(`mapped[-1]` 以外)をバックテスト時の過去時点の確率として使うと、それらの行はウィンドウ末尾までのバーを参照した値になっており、Look-ahead Biasが入り込む。5章ではこの混同をなくすために、`step_forward()` で順方向確率を直接計算する。

HMMの3要素、Baum-Welchアルゴリズム、`fit/predict` のコードを見てきた。次の節からは、モデルが実運用でどう更新され、売買シグナルがどう生成されるかを確認していく。

4.6 _update_hmm() —10本ごとの再学習

市場の構造は時間とともに変わるため、固定したHMMパラメータでは最新のパターンを反映しにくい。そこで10本(営業日換算で約2週間)ごとにモデルを再学習し、学習後は直近30本で現在のレジーム確率を更新する。

ポイントは2行だ。

- ・ 学習時 -> `features[:bar_idx]` で**現在のバーより前のデータだけ**を切り取って渡す。
- ・ 予測時 -> `features[bar_idx-30:bar_idx]` で直近ウィンドウだけを見る。

どちらも**将来データを見ない**点が肝だ。

```
def _update_hmm(self, bar_idx: int) -> None:
    for sym in self._symbols:
        hmm = self._hmms[sym]

        need_fit = (hmm.model is None
                    or bar_idx - hmm._last_fit_idx >=
                     HMM_REFIT_DAYS)

        if need_fit and bar_idx >= HMM_MIN_DAYS:
            feat_train = self._features[sym][:bar_idx]
            if hmm.fit(feat_train):
                hmm._last_fit_idx = bar_idx

        if hmm.model is not None:
            recent = self._features[sym][max(0, bar_idx - 30):
            bar_idx]

            probs = hmm.predict_proba(recent)
            if probs is not None and len(probs) > 0:
                self._regime_probs[sym] = probs[-1]
```

重要な行: `feat_train = self._features[sym][:bar_idx]` —3章の因果原則(no look-ahead)をそのまま守っている。HMM_MIN_DAYS=120 は、最低120本(約6か月)が貯まるまで初回学習を始めないようにするためのセーフティだ。

4.7 _check_entry() —Z-score + HMMゲート

_check_entry() はエントリーシグナルを最終的に承認する関門だ。シグナルが発生するためには、2つの条件を同時に満たす必要がある。

- ・ **Z-score** —価格が平均からどれくらい乖離しているか(3章で扱った標準化指標)
- ・ **HMMゲート** —市場が平均回帰に適した状態にあるか

両方をANDで結んでいるので、どちらかでも欠けるとシグナルは生成されない。

Z-scoreの非対称しきい値

買いと空売りのしきい値は同じではない。

方向	しきい値	理由
買い	$z < -1.5$	通常の売られすぎエントリー
空売り	$z > +2.0$	借株コストを補うためより保守的に

空売りは株式を借りるコストが発生するため、そのコストを超えるリターンが期待できるときだけエントリーする。

HMMゲートの2条件

次のどちらかに該当すれば、エントリーを即座に止める。

1. `mr_prob < 0.3` —平均回帰レジームの確率が30%未満
2. `volatile_prob > 0.4` —ボラティリティレジームの確率が40%超

特にボラティリティレジームの確率が高ければ、Z-score条件と関係なく即座に遮断する。

コードのなかで2つのゲートが結合する核心部は、次の抜粋だけ見れば十分だ。

```
if mr_prob < 0.3 or rp[_DailyHMM.REGIME_VOLATILE] > 0.4:
    return None
```

```

if z < -self._entry_z:          action = SignalAction.BUY
# -1.5
elif z > self._short_entry_z: action = SignalAction.SHORT_SELL
# +2.0
else:                          return None

return TradingSignal(
    symbol=symbol,
    action=action,
    confidence=mr_prob * abs(z),
)

```

confidence = mr_prob × |z|

複数銘柄のシグナルが同時に出てきたとき、優先順位を決めるスコアだ。市場が平均回帰状態である可能性が高く、価格の乖離が大きいほどスコアが高くなる。PortfolioSimulatorはこのスコアの高い順にポジションを開く。

完全な実装は app/trading/backtest/signals/hmm_daily_mr.py で確認できる。

4.8 _check_exit() —6つの手仕舞い条件

手仕舞いは保有中のポジションを閉じる処理だ。エントリーより扱うケースが多いため、ロジックはより複雑になる。

_check_exit() はバーごとに6つの条件を上から下に評価し、どれか1つでも満たされれば即座にポジションを閉じる。

#	条件	デフォルト	意味
1	mr_revert	z ≤ 0.3	平均回帰完了—最も一般的な利益確定
2	z_stop	z > 4.0	Z-score がさらに広がる—MR失敗
3	price_stop	-4%	絶対損失の上限

#	条件	デフォルト	意味
4	trailing_stop	高値から -2%	利益確保後の追加下落に備える
5	max_hold	15本	時間価値の損失を防ぐ
6	volatile_exit	volatile_prob > 0.5	市場環境の悪化—戦略無効化

関数が返す文字列は、そのまま取引ログの `exit_reason` になる。バックテストが終わったあと、この分布を見れば、どの条件が損益の主要因かが分かる。

ポイント: **条件の順序がそのまま優先順位だ**。 `mr_revert` が最初に評価されるので、すでに回帰が終わったポジションには他の損切り条件は適用されない。

最初の通読では表だけ見れば十分だ。6つの条件がコード上でどう展開されるかは、`app/trading/backtest/signals/hmm_daily_mr.py` の `_check_exit()` を参照してほしい。

4.9 PureZscoreDailySignal —ベースライン

HMMゲートをオフにした純粋Z-scoreバージョンが、同じファイルに入っている。`use_hmm=False` だけ設定した薄いサブクラスで、HMMが本当に役立っているかをA/B比較するための **ベースライン (baseline)** だ。

gateの効果は同じ日付・銘柄・基礎signal・上限・費用の二実験で比べる。Aは全基礎signal、BはHMM許可状態だけを通す。return・drawdown・取引数・市場滞在時間を記録し、取引減少と予測改善を区別する。一度良いOOS結果だけで優越性を確定しない。

4.10 全体の流れ—on_bar() の呼び出し順

PortfolioSimulatorがバーごとに呼び出す順序は4.0の4段階と同じだが、**手仕舞いを先に、そのあとエントリーを見る**という優先順位が加わる。同じバーで手仕舞い直後に同じ銘柄へ再エントリーしてしまうのを防ぐためだ。

流れを擬似コードに縮めると次のようになる。

```
on_bar(bar_idx):
    update_hmm(bar_idx)          # 1) 10本ごとの再学習 + レジーム確率更新
    for 保有中の銘柄:           # 2) 手仕舞いを先に —
        _check_exit()
        if 手仕舞い条件成立: ポジションを閉じる
    for 未保有の銘柄:           # 3) その後で新規エントリー —
        _check_entry()
        if エントリー条件成立: ポジションを開く
```

短いgate codeは論理であり全取引programではない。確定観測で確率を更新し、許可状態の確率が基準以上なら基礎signalを通す。欠測や未準備modelは拒否する。基準0.7なら確率0.8は通り、0.5は保留する。

演習問題

1. covariance_type を "full"、"diag"、"spherical" にそれぞれ切り替え、SPY の1年データでHMMを学習させてみよう。どの設定が最も安定してレジームを分離できるか比較してほしい。(モデル比較指標のBICを知っていれば一緒に計算するとよい。)
2. 学習が収束せずに止まる場合(ConvergenceMonitor.converged == False)の原因を3つ考えて、それぞれにどんな対応ができるかを説明してみよう。
3. 合成データで分散整列ロジックだけを単独で検証してみよう。低・中・高ボラティリティの区間を繋いだ偽の特徴量を GaussianHMM に入れ、`np.argsort(np.trace(...))` で振られる state_map が意図どおり 0(低ボラ)

-> 2(高ボラ) の順になるか確かめる。順序が乱れたら、どの特徴量を補うべきかを
書き出してみよう。

この章で必ず覚えること

- ・ HMMは売買シグナルを直接生成しない。「**いま市場はどの状態か?**」を確率で教えてくれるだけだ。シグナル生成はZ-scoreが担当し、HMMはそのシグナルを通すか止めるかの **ゲート** として働く。
- ・ HMMの3要素—**A**(遷移行列)、**B**(放出分布、各状態でどんなデータが出るか)、 **π** (初期状態確率)。
- ・ **Baum-Welch** はEMの反復でパラメータを少しずつ改善していく学習アルゴリズムだ。ライブラリが内部で処理してくれるので、最初の通読では直感だけ掴めばよい。局所最適の問題があるため、`random_state` で再現性を確保する。
- ・ 学習が終わったら、**分散の小さい順にMR / Trend / Volatile** を割り当てる。分散は市場局面を分けるうえで自然な基準だ(平均リターンは符号が反転して分類が崩れる)。
- ・ エントリー条件は2つが同時に満たされる必要がある—`mr_prob >= 0.3 AND volatile_prob <= 0.4 AND Z-scoreのしきい値通過`。
- ・ 学習時は **現在のバーより前のデータだけ** を使う(`features[:bar_idx]`)。このルールを破るとLook-ahead Biasが入り込む。5章でこの問題を本格的に扱う。

まとめ表:

概念	中心点
HMM 3要素	A(遷移)・B(放出)・ π (初期)
Baum-Welch	パラメータをランダム初期化 -> 確率推定 -> パラメータ更新を繰り返す
共分散タイプ	"full" —特徴量間の相関を反映
安定化	<code>min_covar=1e-3</code> —学習発散を防ぐ
レジーム整列	分散昇順 -> MR(0) / Trend(1) / Volatile(2)

概念	中心点
エントリーゲート	<code>mr_prob >= 0.3 AND volatile_prob <= 0.4 AND Z-score</code>
confidence	<code>mr_prob × \ z\ </code> —優先順位スコア
再学習	10本周期、因果スライス

次章(5章)予告: 現在の実装では `predict_proba()` がウィンドウ内の双方向の情報をすべて使ってしまうため、Look-ahead Biasが入り込む。5章ではその原因を整理し、未来を見ない **順方向(forward-only)** 推論へどう書き換えるかを見ていく。

さらに読むなら

- ・ **論文:** Baum, L. E. et al. (1970), “A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains”—元祖Baum-Welch論文
- ・ **ライブラリ:** `hmmlearn` [公式ドキュメント](#) —GaussianHMMパラメータのリファレンス
- ・ **書籍:** Bishop, Pattern Recognition and Machine Learning Chapter 13 —HMMの数学的根拠

理解を確認する

「観測100・特徴4・状態3のHMMのshape、A/B/ π の役割、和1になる確率軸を説明して。」

第5章. HMM Forward-Only —未来情報を使わないリアルタイム推論

バックテストで良い結果が出たからといって、リアルタイムでも同じ結果が出るとは限らない。最初にHMMモデルを作ったとき、私はこの違いをきちんと理解していなかった。バックテストの数値は悪くなかったのに、リアルタイムデータにつないでみると信号が変わってしまった。モデルが急に悪くなったわけではない。バックテストの中で、すでに未来を見ていたのだ。

原因は標準的なHMM推論の方式にある。一般的なHMM推論は、時系列全体を前後に走査して、もっともらしいレジーム(市場状態)を探す。バックテストではすべてのデータが手元にあるので可能である。しかし実際の取引で今日のレジームを判断するとき、明日と明後日の価格は存在しない。時系列全体を見て現在を分類するやり方は、研究分析には便利だが、注文を出すシステムには使えない。

これがルックaheadバイアス (look-ahead bias) である。チャートトレーダーが結果をすでに知ったうえで「自分はここでエントリーしていたはずだ」と言うのと同じだ。違うのは、このバイアスが人の記憶ではなくライブラリの関数の中に隠れていることだ。コードは普通に動き、グラフもきれいに描けるので、見つけにくい。

Forward-only HMMはこの問題を真正面から解決する。未来の区間を一緒に見ず、直前時点のレジーム確率と現在の観測値だけから次のレジーム確率を更新する。精度だけを見れば標準方式より多少低くなることもあるが、リアルタイムで使えない95%の精度より、実際の注文時点で再現できる80%の精度のほうが価値がある。本章の核心はまさにこの選択である。

この章が難しく感じる理由は、「学習」と「リアルタイム推論」がコード上では似て見えるからだ。両者は分けて考えよう。学習時は過去の訓練区間全体を使ってHMMのパラメータを推定する。実取引時点では学習済みパラメータを固定したまま、新しい足が入るたびに現在のレジーム確率だけを一段階前へ進める。この二番目のプロセスがForward-only推論である。

機械学習の言葉で言えば、この章は**学習**と**サービング**を分ける章である。学習 (training) は、過去データのまとまりを使ってモデルを合わせるオフライン作業だ。サービング (serving)、または推論 (inference) は、学習済みモデルを実際のデータの流れにつなぎ、1時点ずつ判断するオンライン作業である。バックテストコードがこの2つを混ぜると、成績は良く見えても実取引システムでは同じ数値を再現できない。

もっと平易に言えば、学習は「判断計算機の日盛りを合わせる作業」であり、推論は「日盛りを合わせた計算機に今日の数値を入れて答えを受け取る作業」である。HMMでは A 、 μ 、 Σ のような値がその日盛りにあたる。学習区間でこれらを決めたら、OOS区間では値を固定し、今日の特徴量だけを入れる。

学習

= 過去データで日盛りを合わせる

推論

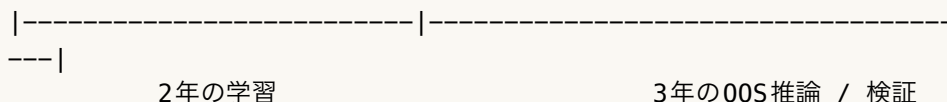
= 固定した日盛りで今日を読む

観点	学習	推論
使うデータ	過去の訓練区間	直前の α (過去の累積) + 今日入ってきた特徴量1行

観点	学習	推論
変わるもの	A、 μ 、 Σ などのモデルパラメータ	現在のレジーム確率 α
実行頻度	再学習スケジュールごと	新しい足ごと
代表的な関数	<code>fit()</code>	<code>step_forward()</code>

欠損値の扱いも、この境界に合わせる必要がある。学習時にNaN行を除外したなら、リアルタイム推論でNaNが入った足も同じルールで拒否するか、保守的に扱う。前処理ルールが学習時と推論時で違えば、モデルは自分が学んだものとは違う入力を見てしまう。

たとえば5年分のデータがあるなら、流れは次のようになる。



1) 学習区間

- 特徴量を作る。
- スケーラーとHMMを学習する。
- モデルパラメータと最後の α を保存する。

2) OOS区間

- 特徴量が1日ずつ届くものとして扱う。
- 保存済みモデルをロードする。
- ``fit()``は呼ばない。
- ``step_forward(x_today)``で α だけを更新する。

初心者は、学習データを教科書、OOSデータを模擬試験と考えればよい。教科書を見て勉強することは許される。模擬試験の答えを先に見てから同じ試験で点数を測ってはいけない。バックテストでOOSを分ける理由はここにある。

学習済みモデルを実際に使うには、保存とロードも必要になる。HMMではモデル本体だけでなく、スケーラーと最後の α も一緒に保存する。そうしないと、推論を正しい状態から続けられない。

学習後に保存するもの:

1. HMMパラメータ A, μ, Σ, π
2. スケーラー 学習時の平均と標準偏差
3. レジーム対応表 raw state id $\rightarrow$ logical state id
4. 最後の α 次の足から始める状態確率

コードの流れは次のように考えればよい。実際の保存形式は pickle、joblib、MLflow artifact などになっても、考え方は同じである。

```
hmm.fit(X_train)
hmm.forward_proba_causal(X_train)
save_model("hmm_model.pkl", hmm)

hmm = load_model("hmm_model.pkl")

x_today = compute_today_features(...)
regime_proba = hmm.step_forward(x_today)
```

`load_model()` のあとで再び `fit()` を呼んではいけない。推論時点の目的はモデルをもう一度学習させることではなく、保存済みの判断計算機に今日の特徴量を入れて、現在のレジーム確率を受け取ることだからだ。

Forward-onlyを一つの場面で見ると

Forward-onlyがまだ抽象的に感じられるなら、状態確率 α が1日ずつ前へ進む場面だけを見ればよい。 α は「現在の市場が各レジームに属する確率」である。

学習最後の日

$\alpha = [\text{MR } 0.60, \text{Trend } 0.25, \text{Volatile } 0.15]$

この値をモデルファイルと一緒に保存する。

005初日

今日の特徴量 $\mathbf{x}_{\text{today}}$ が入ってくる。
 保存済みの α 、遷移行列 A 、今日の観測尤度を使う。
 新しい $\alpha = [\text{MR } 0.64, \text{Trend } 0.22, \text{Volatile } 0.14]$

OOS 2日目
 昨日計算した α から再び始める。
 新しい $\alpha = [\text{MR } 0.58, \text{Trend } 0.30, \text{Volatile } 0.12]$

核心は、毎日ひとつ前の α から続けることだ。OOS全体を一度に入れて「もっともらしい全体経路」を探すわけではない。実取引では明日のデータが存在しないからである。

禁止：
 OOS全体データ $\rightarrow$ `predict_proba()` $\rightarrow$ 全日付のレジームを一度に計算

許可：
 昨日の α + 今日の特徴量 $\rightarrow$ `step_forward()` $\rightarrow$ 今日のレジーム確率

この方式は、全区間をまとめて処理する方法より少し不便に見えるかもしれない。しかし実戦システムが実際にできる計算だけを行うので、バックテストの論理と運用時の論理をそろえられる。本書がForward-onlyを強調する理由はここにある。

基準は単純だ。**時刻 t で計算する値は、 t 以降のデータを使ってはならない。**コードの中に `predict_proba(full_series)` のように全区間をまとめて入れる呼び出しが見えたら、まず疑うべきである。

HMM 1段落復習(第4章で初めて読む方へ)

HMMは、見えない**状態**(例:「トレンド相場/平均回帰相場/危機相場」)が時間とともに変わると仮定するモデルである。3つの要素がある。- **状態遷移確率** A : 昨日が状態 j だったとき、今日が状態 k になる確率 A_{jk} 。- **放出確率** $B_k(\mathbf{x})$: 状態 k のときに観測値 $\mathbf{x}$ (リターン、ボラティリティなど)が出る確率。- **初期分布** π : 最初の時刻に各状態にいる確率。

我々が見るのは価格だけで、状態そのものは見えない。そこで**観測値から隠れた状態を推定する**。

表記の案内: 太字の小文字(α 、 $\mathbf{e}_t$ 、 $\mathbf{x}_t$)はベクトル、大文字(A)は行列である。Pythonコードの `@` は行列積、`*` は同じ位置どうしの要素ごとの積(element-wise)である。

この章の流れ

- 1. 全体像 (§5.0) —hmm_core.py の4段階パイプライン
- 2. なぜ必要か (§5.1) —predict_proba が未来を見る理由
- 3. 核心概念と数式 (§5.2~5.3) —Forward再帰と α 初期化
- 4. オンライン推論 (§5.4) —新しい足ごとに1ステップずつ更新
- 5. 実戦パーツ (§5.5~5.10) —速度最適化、7-state、乗数、安定化、walk-forward

§5.0_{5.4}が中心となる概念・実装で、§5.5_{5.10}は実戦で一緒に使うパーツである。

学習目標

この章を読み終えれば、次のことができるようになる。

- ・ predict_proba() がなぜ未来データを参照することになるのか、一行で説明できる。
- ・ Forward再帰式 $\alpha_t(k)$ が何を意味するか、正規化がなぜ必要かが分かる。
- ・ forward_proba_causal() で初期化し、step_forward() で1ステップずつ更新する流れを頭の中で描ける。
- ・ 7-stateのレジーム確率が regime_mult(ポジション乗数)としてどのようにまとめられるかを描ける。

2種類のForward実装

このコードベースには、レジームを推論する方法が2種類共存している。

方式	使用箇所	因果性
<code>predict_proba(recent_ambdas)</code>	<code>ambdas.py</code>	近似的(直近30本の中で平滑化)
<code>forward_proba_causal</code> <code>+ step_forward()</code>	<code>hmm_core.py</code>	厳密なOnline Forward α

ここでの「因果性」は、未来データを使わないという意味である。第4章の日足MRシグナルは1つめの方式を、本章の5分足戦略は2つめの方式を使う。

5.0 全体像—hmm_core.py パイプライン

hmm_core.py は4段階で動作する。前の段階の結果が、次の段階の入力としてつながる。

Step	段階	入力 -> 出力
1	学習— _BWHMM.fit()	訓練特徴量 -> パラメータ (A, μ, Σ) を学習、分散基準でstateを整列 (0=DeepMR ... 6=Crisis)
2	初期化—for— ward_proba_causal() の α を保存 (_alpha)	訓練特徴量全体 -> Forward α 再帰 -> 最終足
3	オンライン推論— step_forward()	新しい特徴ベクトル1つ -> _alpha を更新 -> レジーム確率
4	ポートフォリオ乗数—GlobalRegime— Layer.step()	確率 $\times$ scale の重み付き和 -> regime_mult (0.1 ~ 2.0)

平易に言えば次のようになる。

Step 1(学習): 過去データでモデルパラメータを決める。各市場状態の平均(μ)・共分散(Σ)と、状態間の遷移確率(A)を計算する。

Step 2(初期化): 学習データを最初から最後までもう一度なめて、最終時点の状態確率ベクトル α を保存する。以降のリアルタイム推論はこの地点から続く。

Step 3(オンライン推論): 新しい足が1本入るたびに α を1つだけ更新する。過去データを再処理しないので速く、未来データを参照しないのでルックアヘッドバイアスが入らない。

Step 4(ポートフォリオ乗数): 現在の状態確率を 0.1~2.0 の範囲の単一の数値 (regime_mult)に変換し、ポジションサイズの調整に使う。

5.1 なぜ predict_proba がルックアヘッドバイアスを起こすか

まず、標準の `predict_proba` をそのまま使ってはいけない理由を押さえよう。

IMPORTANT

一般的なHMMライブラリ(hmmlearn)の predict_proba は、「これまでのデータから見た確率」と「これ以降のデータから見た確率」を掛け合わせて答えを出す。つまり、ある時点のレジームを判断するときに、その後の足をすでに見た状態になる。したがって、過去データだけで現在を判断する因果推論(causal inference)には使えない。

具体例を挙げよう。月曜日から金曜日までのデータを一度に入れて、水曜日のレジームを判断するとする。アルゴリズムは、木曜日と金曜日に急落があったという事実をすでに知っている。だから水曜日を「リスクレジーム」に分類する。バックテストの結果は非常に正確に見えるが、実際の水曜日の場中には、木曜日と金曜日のデータは存在しない。

数式で見ると、`predict_proba` が計算する値は次のようになる。

$$\gamma_t(k) = \frac{\alpha_t(k) \beta_t(k)}{P(\mathbf{X})}$$

ここで α_t は過去だけを見て作った項、 β_t は未来全体を見た項である。両者を掛けた瞬間に未来が混ざる。

図で表すと、時刻 t で使用するデータ範囲が2つの方式で次のように異なる。

時間軸: ... t-3 t-2 t-1 t t+1 t+2 t+3 ...

predict_proba: _____[t]_____ (両側
を見る)

<- 過去 ->

<- 未来 ->

↑ ルックアヘッド!

Forward-only: _____ [t] (過去 + 現在のみ)
 <- 過去 ->

3種類の呼び出し方を、因果性が弱い順に並べると次のようになる。

```
# Incorrect/unsafe example.
proba = model.predict_proba(X_all)

recent      = features[max(0, bar_idx - 30): bar_idx]
proba_recent = hmm.predict_proba(recent)

alpha = bwhmm.step_forward(feat_row)
```

3行を順に見れば、「どこまでのデータを見ているか」がはっきりする。本章では一番下の `step_forward()` を使う。

5.2 Forwardアルゴリズムの再帰式

Forwardアルゴリズムは「現在時刻までの観測値が出る確率」を時点ごとに順番に累積していく。その値を $\alpha_t(k)$ と表す。 t は時点、 k はレジーム番号である。

核心となる再帰式は次のとおり。

$$\alpha_t(k) = \left[\sum_j \alpha_{t-1}(j) A_{jk} \right] \cdot B_k(\mathbf{x}_t)$$

言葉にすると次のようになる。

- ・ 大括弧の中: 「直前時点の各レジームからレジーム k に移る確率の総和」
- ・ 掛ける項 $B_k(\mathbf{x}_t)$: 「レジーム k のとき、いまの観測値が出る確率」

つまり(過去の状態確率 × 遷移確率) × (いまの観測がその状態から出る確率)の一行である。最初の時点では直前状態がないので、初期分布 π から出発する。

1つの実用上の注意—正規化: 足が多くなると小さい確率を掛け続けるため、値が0に消える数値アンダーフロー(underflow)が起きる。そこで毎ステップ α を自身の和で割り、常に合計が1になるようにそろえる。結果として α は「各レジームに属する確率分布」としてきれいに読める。

手計算の例—2-state、3ステップ: 実際に数を入れて、 α がどう更新されるか追ってみよう。初期分布 $\pi = [0.5, 0.5]$ 、遷移行列 $A = \begin{pmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{pmatrix}$ 、各時点の放出確率 $\mathbf{e}_t$ を仮定する。

時刻 t	$\mathbf{e}_t$ (状態0, 状態 1)	更新前 $\alpha \cdot A$	$\odot \mathbf{e}_t$	正規化後 $\hat{\alpha}_t$
1 (初期)	(0.6, 0.2)	$\pi = (0.5, 0.5)$	(0.30, 0.10)	(0.75, 0.25)
2	(0.4, 0.7)	(0.725, 0.275)	(0.290, 0.193)	(0.601, 0.399)
3	(0.3, 0.8)	(0.621, 0.379)	(0.186, 0.303)	(0.381, 0.619)

3本目の足で状態1の放出確率が大きくなると、最初は0.25だった状態1の確率が0.619まで上がる。未来は一度も見えていないのに、観測値が累積されることで分布が自然に動いていく。これがForward再帰の本質である。

5.3 _BWHMM.forward_proba_causal() —学習後の α 初期化

この関数の役目は1つだけだ。訓練データを最初から最後まで順番になめて α を累積し、最終足の α を `self._alpha` に保存する。リアルタイム推論はこの値から出発する。

全体の流れは 5.2 の数式そのままである。要点だけ抜粋すると次のとおり。

```

def forward_proba_causal(self, features: np.ndarray) -> None:
    Xs = self._scaler.transform(features)

    log_e = self.model._compute_log_likelihood(Xs)
    log_e -= log_e.max(axis=1, keepdims=True)
    e = np.exp(log_e)

    #  $\alpha_1 = \pi \odot e_1$ 
    alpha = self.model.startprob_ * e[0]
    alpha = alpha / alpha.sum()

    for t in range(1, len(Xs)):
        alpha = (alpha @ self.model.transmat_) * e[t]
        alpha = alpha / alpha.sum()

    self._alpha = alpha

```

核心はたった1行。alpha = (alpha @ self.model.transmat_) * e[t] が 5.2 の再帰式である。行列積(@)で遷移確率を適用し、要素ごとの積(*)で放出確率を掛ける。その後、合計で割って正規化する。

観測の有限性、共分散、正規化を検査する。和がほぼ0ならlog-sum-expを使い、それでも失敗ならsignalを無効にする。均等確率への数値的fallbackを取引再開と解釈しない。初期確率の和が1か確認する。

5.4 _BWHMM.step_forward() —1ステップのオンライン推論

リアルタイム取引では、新しい足が出るたびにこの関数を呼ぶ。足ごとに全データを計算し直すと遅すぎるので、step_forward() は1ステップだけ計算する。直前の足で保存しておいた _alpha から1つ前へ進む形だ。

呼び出し例は次のようになる。

```
>>> hmm.step_forward(x_new)
# 実行時間: ~0.2ms
# 戻り値: array([0.76, 0.18, 0.06]) <- MR/Trend/Volatile の確率
```

関数の中では3つのステップが起きる。

1. 新しい観測値を、学習時と同じ平均・標準偏差でスケーリングする。
2. 各状態に対して放出確率 e を計算する。
3. 5.2 の再帰式で α を1ステップ更新し、正規化する。

コードは 5.3 とほぼ同じだ。違いは「1本だけ」処理する点だけである。

```
def step_forward(self, feat_row: np.ndarray) -> np.ndarray |
None:
    Xs = ((feat_row - self._sc_mean) /
self._sc_scale).reshape(1, -1)

    diff = Xs[0] - self.model.means_
    log_e = -0.5 * (diff * diff *
self.model._covars_inv).sum(axis=1) \
+ self.model._covars_lognorm
    log_e -= log_e.max()
    e = np.exp(log_e)

    alpha = (self._alpha @ self.model.transmat_) * e
    self._alpha = alpha / alpha.sum()

    return self._to_logical(self._alpha)
```

_to_logical() の役割: HMMの学習は、毎回同じ順序で状態を並べてくれるわけではない。そこで学習直後に、分散を基準に「rawインデックス -> 論理インデックス」の対応表(0=DeepMR, ..., 6=Crisis)を作っておく。この関数は、その対応表に従って α の要素の位置を入れ替える。

関数が `None` を返すのは、(a) 学習されていないとき、(b) 入力に `NaN` が含まれているとき、(c) 例外が発生したときである。運用で `None` や `NaN` がそのままシグナル計算に流れ込むと、ポジションが静かに 0 になってしまう恐れがあるので、連続失敗回数を数えて閾値を超えたら通知が飛ぶようにしておく必要がある(運用ルールの詳細は別の運用章で扱う)。

訓練区間とOOS区間をつなぐ流れ

訓練区間(学習用)とOOS(Out-of-Sample、検証)区間で、 α の再帰がどうつながるかを見てみよう。訓練が終わった時点の α を保存しておき、OOS区間ではその値を引き継いで計算を続ける。

区間	呼び出し	動作
訓練 最初の足	<code>forward_proba_causal()</code>	$\alpha = \pi \times e$
訓練 途中	同じ(繰り返し)	$\alpha = (\alpha @ A) \times e$
訓練 最終足	同じ	<code>_alpha</code> を保存
OOS 最初の足	<code>step_forward(new_feat)</code>	$\alpha = (_alpha @ A) \times e$
OOS 以降	<code>step_forward(new_feat)</code>	$\alpha = (\alpha @ A) \times e$

この構造では、各時点で**その時点までのデータだけ**を使う。だからルックアヘッドバイアスが起きない。

ここからは実戦で一緒に使うパーツである。中心となる概念(5.0~5.4)を初めて読んだなら、この節は軽く流し読みして先に進んでも構わない。実際にシステムを作るときに戻ってきて、じっくり読んでもらえばよい。

5.5 _precompute_cov_fast() —速度最適化

7-state × 5分足の環境では、`step_forward()` は頻繁に呼ばれる。そのたびに共分散 Σ の逆数と正規化定数を計算し直していたら遅い。`_precompute_cov_fast()` は

`fit()` の直後に一度だけ呼ばれ、逆共分散と対数正規化定数を**事前にキャッシュ**しておく。このコードベースは対角共分散を使うので、要素ごとの逆数だけ求めればよく、`hmmlearn`内部の呼び出しよりおよそ3~5倍速い。

`fit()`後に状態共分散のCholesky分解とlog行列式をcacheする。Gaussian log尤度 $-0.5 * (d * \log(2\pi) + \log(\det(\Sigma)) + (x - \mu)^T \Sigma^{-1} (x - \mu))$ で再利用し、逆行列を作るより三角連立方程式を解く。再学習で Σ が変わればcacheも更新する。

5.6 7-Stateのレジーム構造—_REGIME_A_SCALE

`hmm_daily_mr.py` は 3-state を、`hmm_core.py` は 7-state を使う。7つの状態は分散を基準に整理されている。0番が最もボラティリティの低い **DeepMR**(極低ボラの平均回帰)、6番が最もボラティリティの大きい **Crisis**(危機) 状態である。その間に、トレンドの強さが異なる状態が並ぶ。

各状態には、ポジションサイズを調整する**スケール(scale)**値が掛けられる。直感は2つだけ。

- ・ **ボラティリティが非常に低いとき (DeepMR)**: スプレッド収益が小さく、手数料の比重が大きくなるのでレバレッジを下げる。
- ・ **ボラティリティが非常に高いとき (HighVol/Crisis)**: HMMシグナル自体の信頼度が低く、損失リスクが大きいのでポジションを大きく削る。

状態倍率は学習確率とは別のrisk方針だ。通常1.0、高volatility 0.5、危機0.1は例にすぎない。`sum(prob[state]*scale[state])`の後で総position上限を適用する。再学習でIDが変わり得るため意味を整理してから倍率を結ぶ。

5.7 GlobalRegimeLayer.step() —単一の乗数にまとめる

このメソッドがすることは1つだけだ。新しい足が入ったら `step_forward()` で 7-state の確率 γ を受け取り、そこに `_REGIME_A_SCALE` の値を掛けて単一の数値

regime_mult にまとめる。この数値が第9章 Kelly Sizing のポジション乗数としてそのまま入る。

```
def step(self, bar_idx: int) -> float:
    if not self._hmm.is_fitted:
        return 1.0

    feat_row = self._feats[bar_idx]
    alpha = self._hmm.step_forward(feat_row)
    if alpha is None:
        return 1.0
    self._last_alpha = alpha

    n = len(alpha)
    mult = sum(float(alpha[i]) * _regime_mult_from_rank(i, n)
    for i in range(n))

    self._update_dominant_state(alpha)
    return float(np.clip(mult, 0.0, 2.0))
```

核心は `mult = sum(y_i × scale_i)` の一行である。単一の状態を選ばずに、すべての状態の確率を混ぜて使うので、レジーム転換の境界でスケールが急変せず、取引コストも減る。

regime_mult は 0.1 ~ 2.0 の実数である。1.0 なら基準ポジション、0.5 なら半分に減らして保有、1.5 なら 1.5 倍に増やして保有という意味になる。

5.8 dominant_state —3本連続確認による安定化

HMMは足ごとに確率を新しく計算するので、市場が一瞬揺れるだけで確率も一緒に揺れる。この揺れをそのままシグナルに使うと、無用な売買が増える。

dominant_state はこの揺れを抑える仕組みである。足ごとに最も確率の高い状態を見るが、**3本連続で同じレジームが確認されたときだけ**状態を更新する。1~2本違うからといってすぐには切り替えない。さらに、最も高い確率自体がある閾値(コード上はおよそ 0.32)を下回ると、レジームが不明確だとみなして現在の状態をそのまま維持する。

支配状態の変更には確率差と持続を使う。例では候補が0.15以上優位な状態が3足続いたら切り替え、崩れたら counter を戻す。実際の値は学習・検証で選び OOS で固定する。

5.9 Walk-Forward の利用例

NOTE

再学習の周期が短すぎると(例: 3日)過学習のリスクが大きく、長すぎると(例: 120日)市場の変化を反映しにくい。実戦では **10日周期** がバランス点になる。

これまで扱ったパーツを組み合わせて、実際の walk-forward ループを作ってみよう。流れは単純だ: データ準備 -> 学習 + α 初期化 -> OOS区間で足ごとに1ステップずつ。

```
from app.trading.backtest.signals.hmm_core import
GlobalRegimeLayer, REGIME_LABELS

layer = GlobalRegimeLayer(n_states=7)
layer.prepare(spy_df, macro_dfs={"GLD": gld_df, "TLT": tlt_df})

n_train = int(len(spy_df) * 0.6)

layer.refit_and_init(n_train)

results = []
for bar_idx in range(n_train, len(spy_df)):
```

```

mult = layer.step(bar_idx)
results.append({
    "bar_idx": bar_idx,
    "regime_mult": mult,
    "dominant": REGIME_LABELS[layer.dominant_state],
    "trans_risk": layer.transition_risk,
})

```

確認すべき出力: regime_mult の平均が 1.0 付近なら正常。0.3 以下に偏っているなら、HighVol/Crisis の比重が異常に大きいというサインである。

refit_and_init() の中では3つのことが起きる。

1. trailing 60セッションの窓(60 × 78 = 4680本)を切り出す。
2. 新しい _BWHMM を作り、fit() を呼ぶ(warm-start ではなく cold 再初期化)。
3. forward_proba_causal() で α を初期化する。

NaNや履歴不足は正常なpatternなしと区別して「判定不可」にする。注文は保守的に拒否するが、別状態で記録すれば理由が残る。

5.10 遷移risk

現在の支配状態 k を次時刻に離れる確率は $p_{\text{next}} = \alpha A$ 、 $\text{risk} = 1 - p_{\text{next}}[k]$ である。現在分布のentropyとは別指標だ。高entropyは不確実性であり転換に近い保証ではない。定義・正規化・参入保留基準を評価前に固定する。 $p_{\text{next}}[k] = 0.8$ ならこの定義のriskは0.2。

演習問題

1. forward_proba_causal(X) と predict_proba(X) の2つの関数を、同じ500個の観測値に適用し、レジーム確率の差を可視化せよ。どの時点から差が大きく開くか？

2. 5.2の手計算例を拡張して、5ステップまで自分で計算せよ。4・5本目の放出確率は任意に決めてよいが、各段階で正規化前後の値を両方書いて比較すること。
3. Walk-forward の訓練ウィンドウを 250日から 125日に縮めたとき、成績がどう変わるかを分析せよ。

この章で必ず覚えること

この章で必ず覚えるべきことは、次の3つである。

1. **標準の predict_proba は未来を見ている。**バックテストでは正常に見えるが、リアルタイムでは使えない。
2. **Forward-only は「直前の α + 現在の観測」だけを使う。**だからリアルタイム環境と同じ条件で推論できる。
3. **学習(fit) -> α 初期化(forward_proba_causal) -> 1ステップ更新(step_forward)**の3段階が、全体の骨格である。

章の冒頭の事例に戻ろう。あのバックテストがリアルタイムで崩れたのは、モデルが悪かったからではなく、水曜日のレジームを判断するときに木・金のデータをすでに見ていたからだ。predict_proba を step_forward に替えて同じデータで回し直せば、バックテストの数値自体は下がるが、**リアルタイムに移したときに同じ値が出る**。この一貫性こそ、Forward-only が与えてくれる本当の価値である。

実装チェックリストは次のとおり。

段階	メソッド	目的
学習	<code>_BWHMM.fit(train_feats)</code>	パラメータ推定と state 整列
速度	<code>_precompute_cov_fast(model)</code>	逆共分散の事前計算
初期化	<code>forward_proba_causal(train_feats)</code>	α を訓練終了時点の状態に設定
推論	<code>step_forward(feats_row)</code>	1本ごとに α を更新し、レジーム確率を返す

段階	メソッド	目的
合算	<code>GlobalRegimeLayer.step(bar_idx)</code> $\gamma \times \text{scale}$ の重み付き和 -> <code>regime_mult</code>	
安定化	3-bar confirm	<code>dominant_state</code> の急変を防ぐ
不確実性	<code>transition_risk</code>	エントロピー基準の転換リスク度

核心の数式 1 行: 新しい足が入るたびに、 α を次のように 1 ステップ前へ更新する。

$$\hat{\alpha}_t = \frac{(\hat{\alpha}_{t-1} \cdot A) \odot \mathbf{e}_t}{\|(\hat{\alpha}_{t-1} \cdot A) \odot \mathbf{e}_t\|_1}$$

そして、この確率を重み付き和にまとめて単一の乗数を得る。

$$\text{regime_mult} = \sum_k \gamma_k \cdot \text{scale}_k$$

次章(第6章): 価格データには大小のノイズが混ざっている。第6章では、このノイズをリアルタイムで除去する 1 次元カルマンフィルタ (Kalman Filter) を扱う。

さらに読むには

- ・ **論文:** Rabiner, L. R. (1989), “A tutorial on Hidden Markov Models”—Forward-Backward アルゴリズムの原典
- ・ **論文:** “Online EM algorithms for HMMs”—リアルタイムHMM再学習の理論的基礎
- ・ **GitHub:** `hmmlearn` ソースコード `_do_forward_pass()` —本書で直接活用した内部実装

第6章. カルマンフィルタ ターノイズ除去とイノ ベーションZスコア

Part III —ノイズと多重レジーム(6~8章) このパートはシステムの中核である。6章では価格と特徴量のノイズを減らし、7章では複数の市場をまとめて見て、8章ではマクロレジームと個別資産のシグナルを二層に分ける。最初の通読では各モデルの数式よりも「なぜ一銘柄だけを見ないのか」を理解することに集中する。

2章のKalman vs 6章のKalman —役割の違い

	2章 <i>ScalarKalmanFilter1D</i>	6章 <i>KalmanFilter4D</i>
入力	単一特徴量(リターン1個)	4つの特徴量(リターン・ボラティリティ・方向性・出来高)
主な役割	特徴量のノイズ除去	特徴量のノイズ除去 + Innovation Zスコア生成
出力	平滑化された特徴量(HMMに直接入力)	平滑化された特徴量 + novelty_gate用Zスコア

2章のKalmanが「基本的な掃除道具」なら、6章のKalmanは「掃除+異常検知機能の追加」だ。

RSI、MACD、ボリンジャーバンドといったテクニカル指標はすべて同じ前提から出発している。終値がその時点の市場価値を正確に反映しているという仮定だ。ところが実際の終値には、買いと売りのスプレッドの跳ね、引け間際の注文不均衡、市場マイクロストラクチャー由来のノイズ(雑音)が混ざっている。ノイズが混ざった値で指標を計算すると、シグナルではなくノイズを分析することになってしまう。

特にレジーム(市場の状態、例:上昇相場->下落相場)が切り替わる場面では、単純な移動平均は苦しくなる。反応を速くするとノイズに振り回され、遅くすると転換を捉えるのが遅くなる。

カルマンフィルタは別のアプローチをとる。過去の価格を単純に平均するわけではない。価格を「完璧な真実」ではなく「ノイズの混ざった観測値」として扱い、新しい観測が入るたびに、隠れた本当の状態の推定値を更新する。もともとはGPSがない時代に、ノイズだらけのセンサーでロケットの位置を推定するための手法だが、金融時系列にも同じ原理が通用する。

中心の定義はこうだ。**カルマンフィルタは「ノイズのある観測値が与えられたときの、隠れた状態の最適推定器」である。**

カルマンフィルタを「高度な移動平均」としてだけ見ると本質を見落とす。移動平均は単純に直近N個を平均する。カルマンフィルタは二つを比較する。「自分の予測がどのくらい不確かか」と「新しい観測値がどのくらい信頼できるか」だ。新しい観測が信頼できるなら素早く追従し、ノイズが大きいと判断すればゆっくり動く。

状況	移動平均	カルマンフィルタ
急な価格ジャンプ	ウィンドウ長の分だけ反応が遅い	設定次第で素早く反応できる
一時的なノイズ	平均に混ざって指標を揺らす	ノイズが大きければ一部しか反映しない
不確実性の表現	なし	推定分散(P)として追跡

この章ではカルマンフィルタを2か所で使う。HMM入力の特徴量を整える場所と、平均回帰シグナル用のZスコアを作る場所だ。

本書全体の機械学習フローで見ると、カルマンフィルタは**モデルが読む前に数値の揺れを抑える整理道具**である。HMMやRLは、渡された入力値を信じて判断する。入力値がノイズで大きく揺れると、モデルは市場状態が変わったと誤認する可能性がある。

生の価格 / 生の特徴量
ノイズ、跳ね、一時的な外れ値が混ざる
↓
v
Kalman Filter
本当の動きと一時的なノイズを分けて滑らかにする
↓
v
HMM / Z-score / RL の入力
モデルがより安定した数値を読む

初心者はカルマンフィルタを「価格を予測するモデル」と考えると混乱しやすい。この章でのカルマンフィルタは、明日の価格を当てる装置ではない。**今日の観測値をどれくらい信じるかを決める装置**である。新しい価格が跳ねたとき、「本当の変化か、一時的なノイズか」を数値で判断する。

新しい観測値が入ったとき、カルマンフィルタが問うこと：

1. 直前まで信じていた値はどれくらい確かか？
2. 新しい観測値はどれくらい信頼できるか？
3. では新しい値をどれくらい反映するか？

こうして整理された数値がHMMに入るとレジームが揺れにくくなり、Zスコアに入ると一時的な跳ねをシグナルと誤認する可能性が下がる。

この章の流れ

- ・ 全体像—カルマンフィルタの2つの役割
- ・ ローカルレベルモデル—直感的な数式
- ・ KalmanFilter1D —HMM特徴量の平滑化
- ・ 4次元の特徴量ごとのカルマンパラメータ
- ・ `_kalman_innovations()` —価格からInnovationを取り出す
- ・ 因果的なZスコアと半減期
- ・ 平均回帰シグナルとバックテスト

学習目標

この章が終わると、次のことができるようになる。

- ・ カルマンフィルタの予測-更新サイクルを大まかに説明できる
- ・ KalmanFilter1DでHMM特徴量4つを処理する流れがわかる
- ・ Innovation(予測と実際の差)とは何か、なぜそれを平均回帰シグナルに使うのかわかる
- ・ Innovation Zスコアに基づくシグナル(`kalman_self_mr`)のエントリー/手仕舞い口ジックを理解する
- ・ 半減期(halflife)が保有期間を決める役割を持つことを知る

6.0 全体像—カルマンフィルタの2つの役割

このコードベースではカルマンフィルタを2つの用途で使う。アルゴリズムは同じだが、目的が違う。

1つ目は**ノイズ除去**である。生の特徴量データにはノイズが混ざっており、そのまま使うとHMMが誤ったレジームを判定してしまう可能性がある。カルマンフィルタでノイズを取り除けば、HMMはより安定した入力を受け取れる。

2つ目は**平均回帰シグナルの生成**である。価格がカルマンフィルタの予測値からどれだけ離れたかを測り、買い・売りシグナルとして使う。

役割1:HMM入力特徴量の平滑化(features.py)

入力は生の特徴量4つ(リターン、5日ボラティリティ、20日モメンタム、出来高比)。各次元に独立してKalmanFilter1D(Q, R)を適用し、平滑化された特徴量をHMMに渡す。

役割2:価格のInnovationからZスコアを構成(kalman_self_mr.py)

ここで「Innovation」とは、「フィルタが予測した値と実際の観測値の差」である。この差が大きいほど価格が予測から大きく外れたことを意味し、それをZスコアで正規化すればエントリー・手仕舞いの判断に使えるシグナルになる。

入力はlog(price)の時系列。_kalman_innovations()で時点ごとの予測誤差(Innovation)を取り出し、_build_z_series(window=60)でその時系列のローリングZスコアを作る。このZスコアがエントリー・手仕舞いシグナルの核になる。

6.1 ローカルレベルモデル—直感的な数式

式を見る前に、2つの主要パラメータの意味を先に押さえよう。この2つの値さえつかめば、カルマンフィルタはほぼ理解したも同然だ。

NOTE

- ・ **Q(プロセスノイズ)**:隠れた状態が実際にどれだけ変化するか。値が大きいほど最新の観測値を信頼する(反応が速い)。
- ・ **R(観測ノイズ)**:測定誤差の大きさ。値が大きいほど以前の推定値を信頼する(反応が遅い)。

実践ではQ=0.01、R=1.0から始めて、結果の変化を見ながら調整する。

KalmanFilter1Dは**ローカルレベル (Local Level) モデル**である。中心となる仮定は1つ。「隠れた本当の状態は、前の値から少しずつしか動かない」というものだ。価格やリターンが一瞬で全く違う値に跳ねたりはしない、と見る。

式にすると2行で済む。

- ・ **状態方程式**: 隠れた本当の値 x_t は前の値に小さなランダムノイズが加わっただけ。

$$x_t = x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, Q)$$

- ・ **観測方程式**: 私たちが目にする値 y_t は、本当の値に測定誤差が加わったもの。

$$y_t = x_t + v_t, \quad v_t \sim \mathcal{N}(0, R)$$

フィルタは各時点で2つのステップを繰り返す。

ステップ1: 予測。時間が経った分だけ、推定の不確実性 P がQだけ大きくなる。

ステップ2: 更新。新しい観測値が入ると、**カルマンゲイン** K を計算し、予測値を観測値の側へ引っ張って更新する。

$$K_t = \frac{P_{t|t-1}}{P_{t|t-1} + R}$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t \cdot \underbrace{(y_t - \hat{x}_{t|t-1})}_{\text{Innovation(イノベーション)}}$$

括弧の中の $(y_t - \hat{x}_{t|t-1})$ が**Innovation(イノベーション)**だ。「予測がどれだけ外れたか」を表し、その誤差をどの程度反映するかを K_t が決める。

Kの意味を直感的に整理すると: - $K \rightarrow 0$: 観測ノイズRが大きい $\rightarrow$ 新しい観測値をほとんど無視して以前の推定値を保つ(滑らかな推定)。 - $K \rightarrow 1$: 状態ノイズQが大きい $\rightarrow$ 新しい観測値をほぼそのまま追従する(速い反応)。

実践ではQとRを別々に覚えるより、比 **Q/R** で理解する方が楽だ。Q/Rが大きければ「市場が実際に変わっている」と素早く認め、小さければ「今の動きはノイズかもしれない」と見てゆっくり反応する。速すぎるとノイズをシグナルと取り違え、遅すぎるとレジーム転換を逃す。

体温計を思い浮かべるとわかりやすい。部屋の温度が22度だと信じていたのに、新しい温度計が30度を指す。温度計がよく壊れるものなら、30度をそのまま信じたりはしない。逆に窓が開いていて部屋の温度が実際に急変している状況なら、新しい測定値の方を信じる。カルマンフィルタのQ/Rも、この判断を数値に置き換えたものだ。Q/Rが大きいと「部屋の温度が本当に変わったかもしれない」、Q/Rが小さいと「温度計が跳ねただけかもしれない」に近い。

小さな数字で見るともっとはっきりする。予測値が100で新しい観測値が110のとき、Kが0.2なら新しい推定値は $100 + 0.2 \times 10 = 102$ だ。Kが0.8なら $100 + 0.8 \times 10 = 108$ になる。同じ観測値110を見ても、Kが小さければ少ししか動かず、Kが大きいと素早く追従する。

定常状態は同程度の品質の観測が続きPとKがほぼ変わらない状態だ。同じQ・Rで式を反復し $\text{abs}(P_{\text{new}} - P_{\text{old}})$ の縮小を確認する。新観測の反映率が安定する意味であり、dataが変わらなくなる意味ではない。

6.2 KalmanFilter1D —HMM特徴量の平滑化

KalmanFilter1Dは、値が1つの時系列を扱うカルマンフィルタである。バーが閉じるたびに `update(z)` を1度呼び出すと、平滑化された推定値を返す。

次のコードは、6.1節で見た予測-更新の2ステップをそのまま落とし込んだものだ。

```
# app/trading/backtest/features.py

class KalmanFilter1D:
    """Scalar Kalman filter for feature smoothing."""

    def __init__(self, Q: float = 1e-4, R: float = 1e-2):
        self.Q, self.R = Q, R
        self.x = 0.0
        self.P = 1.0

    def update(self, z: float) -> float:
        self.P += self.Q
        K = self.P / (self.P + self.R)
```

```
self.x += K * (z - self.x)
self.P *= 1 - K
return self.x
```

確認したい出力:update(z)を繰り返し呼ぶと、最初の数バーは推定値が大きく揺れるが、徐々に滑らかになる。Pも一緒に出力してみると、だんだん小さくなって安定値に収束する様子が見える。

点検のコツ:Pが最初の100バーのうちに何らかの値に収束すれば正常だ。延々と大きくなり続ける場合は、Rを大きくするかQを小さくする必要がある。

6.3 4次元の特徴量ごとのカルマンパラメータ

特徴量ごとに変化の速さは違う。リターンのように急変する特徴量もあれば、モメンタムのようにゆっくり変わる特徴量もある。だから**各特徴量の次元ごとに(Q, R)のペアを別々に設定する**。

次元	特徴量	Q	R	反応速度	理由
0	log_return	1e-4	5e-3	速い	リターンは急変—遅延なしで反映
1	5d_vol	5e-5	1e-3	中	ボラティリティはクラスタリング効果あり
2	20d_momentum	5e-5	1e-3	中	モメンタムは徐々に転換
3	log_vol_ratio	1e-4	5e-3	速い	出来高スパイクを即座に反映

コードではこの表をそのまま定数にしている(KALMAN_PARAMS_4D)。特徴量計算関数(compute_daily_features)は上の順序で4次元を作る(対数リターン、5日実現ボラティリティ、20日モメンタム、出来高比)。

そのうえで各次元に**独立したKalmanFilter1D**を1つずつ用意し、時間順にupdate()を回す。ここでの肝は**次元ごとに新しいフィルタインスタンスを作る**という点だ。同じフィルタを全次元で使い回すと状態(x、P)が混ざって意味を失う。

NaNでは更新を省き、最初の有効観測前は値を作らない。innovation Z-scoreはinnovation/sqrt(P_pred+R)で、分母が非正・非有限なら拒否する。初期Pに影響されるwarmupを判断から除き、その長さを条件へ記録する。

6.4 _kalman_innovations() — 価格から Innovation を取り出す

ここからは2つ目の役割に進む。同じカルマンフィルタを**対数価格の時系列**に適用し、各時点の予測誤差(Innovation)を時系列として取り出す。

この関数の仕事:各時点の対数価格に対して1ステップ先の予測を作り、「予測 - 実際」の差を記録する。この差が正なら価格がカルマン推定レベルより上(買われすぎ)、負なら下(売られすぎ)だ。

```
# app/trading/backtest/signals/kalman_self_mr.py

def _kalman_innovations(log_prices, delta=1e-4, obs_noise=1.0):
    T = len(log_prices)
    innovations = np.full(T, np.nan)

    first = np.argmax(~np.isnan(log_prices))
    x, P = log_prices[first], 1.0

    for t in range(first + 1, T):
        y = log_prices[t]
        if np.isnan(y):
            continue
```

```

    innov = y - x
    innovations[t] = innov

    P_pred = P + delta
    K = P_pred / (P_pred + obs_noise)
    x = x + K * innov
    P = (1.0 - K) * P_pred

    return innovations

```

重要な行: `innov = y - x`。更新前の`x`を使っているのが肝だ。つまり時点`t`のイノベーションは、「`t-1`までの情報だけで作った予測」と実際の`t`時点の観測値との差である。こうすることで未来の情報が混ざらない(いわゆるルックahead漏れがない)。

Innovationの平均回帰特性: 株価が基本トレンド(ローカルレベル)から外れるとイノベーションが大きく跳ねるが、結局0に戻る。このパターンを平均回帰戦略のシグナルとして使う。

6.5 _build_z_series() —因果的なZスコア

2章の単純なrolling Zスコアとの比較: rolling mean/stdはウィンドウサイズが固定なので、ボラティリティの変化に追従するのが遅い。カルマンはノイズモデルに合わせて適応速度を自動で調整する。

Zスコアを作るときも、未来のデータが入り込んではいけない。そこで、ローリング平均と標準偏差を1つ後ろにずらして(`shift(1)`)、**現在のバー自身を統計から除外する**。

```

def _build_z_series(spreads, window=60):
    s = pd.Series(spreads)
    roll_mu = s.rolling(window).mean().shift(1)
    roll_sd = s.rolling(window).std(ddof=0).shift(1)
    z = ((s - roll_mu) / roll_sd).values
    z[~np.isfinite(z)] = np.nan
    return z

```

shift(1)の意味:時点tのZスコア計算に使う平均と標準偏差は、t-1までのデータで作られる。現在のバーのイノベーションは統計から外れるので、因果的(ルックアヘッドなし)なZスコアになる。

6.6 _calc_halflife() —AR(1)半減期推定

平均回帰戦略でもう1つ押さえておくべき概念が**半減期(halflife)**だ。「スプレッドが平均に戻るまで平均で何バーかかるか」を表す。半減期が短いほど回帰が速く、長いほど平均回帰は弱い。

推定方法は単純だ。スプレッドの変化量を「平均からどれだけ外れたか」で回帰し、回帰係数 ϕ を得たうえで、半減期は次の式で求まる。

$$T_{1/2} \approx -\frac{\ln 2}{\phi}, \quad (\phi < 0 \text{なら平均回帰})$$

過去窓の回帰 $\Delta s_t = a + b \cdot s_{t-1} + \varepsilon_t$ から推定する。 $\phi = 1 + b$ が0と1の間ならAR(1)半減期は $\log(0.5)/\log(\phi)$ 足。小さい負のbなら $-\log(2)/b$ が近似になる。 $\phi \geq 1$ では平均回帰半減期がなく、 $\phi < 0$ は振動なので単調減衰式を適用しない。

核だけ押さえておけばよいポイント: - $\phi \geq 0$ なら平均回帰ではない。コードではこのとき保守的な上限(80バー)で抑え、誤ったエントリーを避ける。データが20バー未満のときも同じ上限を使う。 - $hl > hl_max$ (=60) -> 回帰が遅すぎる -> **エントリー禁止** - $eff_hold = \min(max_hold, \text{int}(hl_hold_mult \times hl))$ -> 半減期に基づく動的な保有期間

6.7 KalmanSelfMRSIGNAL —エントリーと手仕舞いのロジック

前に作ったイノベーション、Zスコア、半減期を1つにまとめたシグナルクラスである。初期化段階で各銘柄のイノベーションとZスコアの時系列をあらかじめ計算しておく。

```
@register_signal("kalman_self_mr")
class KalmanSelfMRSIGNAL(SIGNALGenerator):
    def initialize(self, data):
        for sym in self._symbols:
            log_p = np.log(np.maximum(self._close[sym], 1e-10))
            innov = _kalman_innovations(log_p,
self._kalman_delta, self._kalman_obs)
            self._spreads[sym] = innov
            self._z_series[sym] = _build_z_series(innov,
self._z_window)
```

主要パラメータのデフォルト値:

パラメータ	デフォルト値	役割
z_entry	3.0	エントリーZスコア閾値
z_exit	0.3	手仕舞いZスコア閾値
hard_stop_z	4.5	Zスコア極端値での損切り
hl_max	60.0バー	許容する最大半減期
price_stop	-8.0%	価格ベースの損切り
max_hold	30バー	最大保有期間
kalman_delta	1e-4	Q(状態ノイズ)
kalman_obs	1.0	R(観測ノイズ)

エントリーの4重フィルタ

エントリーは4つの条件を同時に満たす必要がある。1つでも欠ければポジションを開かない。フィルタが多いほど誤ったエントリーが減る。

順番	条件	意味
1	z >= z_entry (3.0)	十分に乖離しているか

順番	条件	意味
2	$ z_{\text{now}} < z_{\text{prev}} $	これ以上開かず収束に向かっているか
3	$\text{halflife} \leq \text{hl_max} (60)$	平均回帰が十分に速いか
4	方向決定	$z < 0 \rightarrow \text{LONG}$ 、 $z > 0 \rightarrow \text{SHORT}$

スコアは $|z| \times (1/\text{halflife})$ とし、「大きく乖離していて、なおかつ速く戻りそうな」銘柄を優先する。

手仕舞いの6重チェック

開いたポジションは、次の6つのうち1つでも引っかかれば閉じる。

条件	閾値	意味
price_stop	$\text{PnL} < -8\%$	絶対損失の上限
z_hard_stop		z
hl_degrade	$\text{halflife} > 100$ バー	平均回帰特性の消失
z_converge		z
z_cross	zが逆符号に交差	オーバーシュートした後の発散
max_hold	$\text{days} \geq \text{hl_hold_mult} \times \text{hl}$	動的保有期間の超過

エントリー条件をきつく、手仕舞い条件を多彩にするのが平均回帰戦略の一般的な形だ。損失は素早く切り、利益は目標に達したら確実に取りに行く構造である。

6.8 kalman_self_mr vs hmm_daily_mr の比較

TIP

同じ期間で単純なEWMA Zスコアとカルマン Zスコアを並べて回してみると違いがはっきりする。2020年3月のコロナショックや2022年の金利急騰のように市場が急変する区間では、カルマンの方が速く反応する。EWMAは減衰係数を固定して使うが、カルマンはシグナルとノイズの比を時点ごとに計算し直して、今の状況に合わせて調整するからだ。

両戦略とも日足のZスコアで平均回帰を捉えるが、そのやり方は違う。

項目	hmm_daily_mr	kalman_self_mr
Zスコアの基礎	価格の移動平均(20バー)	カルマン Innovation(対数価格)
レジームフィルタ	HMM 3-state	なし(halflifeフィルタ)
Zエントリー閾値	1.5 / 2.0	3.0(より保守的)
Z手仕舞い閾値	0.3	0.3
保有期間	固定15バー	動的(halflifeベース)
最大ポジション	5個	60個(多銘柄同時)
配分比率	15%	3%

kalman_self_mrは1銘柄あたりの比率を3%に下げる代わりに、最大60銘柄を同時に保有する。1銘柄に集中せず広く分散する戦略だ。

6.9 バックテスト実行の例

これまで取り上げたコンポーネントを全部つないで、実際にバックテストを走らせる様子である。

```
from app.trading.backtest.signals.kalman_self_mr import
KalmanSelfMRSignal
from app.trading.backtest.portfolio import PortfolioSimulator
from app.trading.backtest.data import downloadBars

symbols = ["AAPL", "MSFT", "NVDA", "GOOGL", "META", ...]
data = downloadBars(symbols, days=730, interval="1d")

sig = KalmanSelfMRSignal(z_entry=3.0, hl_max=60.0,
                        kalman_delta=1e-4, kalman_obs=1.0)
sig.initialize(data)

result = PortfolioSimulator(max_positions=60,
per_trade_alloc=0.03).run(sig)
result.print_summary()
```

確認したい出力: `print_summary()` は累積リターン、パフォーマンス、最大ドローダウン、エントリー回数、平均保有期間を出力する。Zエントリー閾値(3.0)を1.5に下げてもう一度回してみると、エントリー頻度は増えるが平均勝率は下がることが見えるはずだ。

演習問題

1. $Q=0.01/R=10$ と $Q=1.0/R=0.01$ の2設定でSPY-QQQスプレッドのZスコアを計算し、2020年3月時点の差をプロットで比較せよ。
2. `KalmanFilter1D.update(z)` にNaN観測値の処理ロジックを追加せよ。NaNが連続5個以上ならば予測状態をそのまま保つよう実装すること。
3. カルマンZスコアと単純な20日rolling Zスコアを同じペアに適用し、どちらが偽陽性(false positive)が少ないかを比較せよ。

この章で必ず覚えておくこと

- ・ **カルマンフィルタは単純な移動平均ではない**。予測の不確実性と新しい観測値の信頼度を比較し、推定値を更新する。
- ・ **QとRの比(Q/R)が反応速度を決める**。Q/Rが大きいほど速く、小さいほど遅く反応する。
- ・ **Innovation = 予測 - 実際**。この値が0から大きく外れて戻るパターンを平均回帰シグナルとして使う。
- ・ **因果的なZスコア**は、`shift(1)`で現在バーを統計から外し、未来情報の漏れを防ぐ。
- ・ **半減期**が短いほど平均回帰が速く、長すぎる(60バー超)場合はエントリーを止める。

概念	要点
ローカルレベルモデル	F=1、状態=価格レベル、Innovation=予測誤差
カルマンゲインK	Q/R比で決まる—高いほど速い反応、低いほど平滑化
4D特徴量の平滑化	各次元に独立したKalmanFilter1D、 KALMAN_PARAMS_4D
Innovation Zスコア	<code>shift(1)</code> の因果的Zスコアで売られすぎ/買われすぎを検知
半減期 halflife	AR(1) OLSで推定;60バー超でエントリー禁止
動的な保有期間	<code>eff_hold = hl_hold_mult × halflife</code>

次章(7章):Cross-asset Joint HMMを扱う。1つの資産ではなく、複数の資産の動きがどれだけ一緒に変動するか(共分散構造)を同時にモデリングする。こうすれば市場レジームをより安定して捉えられる。

さらに読むには

- ・ **書籍:**Grewal & Andrews 『Kalman Filtering: Theory and Practice』 —パラメータチューニングと数値安定性を深く掘り下げている
- ・ **論文:**“Pairs Trading with Kalman Filtering” (Pole, 2007) —カルマンフィルタをスプレッド追跡に初めて適用した論文
- ・ **ライブラリ:**pykalman —Python向けのカルマンフィルタ実装;EMアルゴリズムでパラメータを自動推定する

第7章 クロスアセット Joint HMM —複数市 場を同時に見るレジ ーム検出

個人投資家の多くは、一枚のチャートだけを眺める。その銘柄のトレンドライン、移動平均、出来高を確認し、次の方向を予測する。だがこのやり方には穴がある。その銘柄が属する市場全体が今どんな状態にあるのか、つまり投資家がリスクを取りに行っている状態なのか、それとも安全資産に逃げ込んでいる状態なのかを見なければ、重要な警告サインを取りこぼす。

機関投資家のトレーダーは違うアプローチをとる。彼らは単一銘柄を切り離して分析しない。複数の市場が同時に何を語っているかを一緒に見る。たとえば市場がリスク回避

(Risk-Off)に転じるとき、その変化は株価よりも債券市場とクレジットスプレッド(社債と国債の金利差で、広がると企業のデフォルト懸念が高まることを意味する)に先に現れる傾向がある。株式だけを見ていると、下落が始まってからようやくリスクに気づく。債券やボラティリティ指標まで含めて見れば、リスク選好が弱まる過程をより早く感知できる。

2020年3月の株価急落がよい例だ。株価が大きく下げる前に、すでに国債利回りの低下、VIXの上昇、クレジットスプレッドの拡大という先行サインが出ていた。株価だけを見るモデルはこのサインを受け取れない。

この章の出発点はシンプルだ。市場レジームをきちんと把握するには、ひとつの市場だけを見るのではなく、複数の市場が同時に発しているサインを一緒に見なければならない。

Joint HMMはこのアイデアを実装する手段だ。「Joint(共同)」は「複数の資産を一緒に」という意味である。SPYに1つ、TLTに1つと別々にHMMを作るのではなく、SPY・TLT・HYG・VIXの観測値をひとつのモデルに同時に入力する。こうすることでレジームを判定する際にあらゆるクロスマーケット情報を一度に活用できる。単一資産のHMMが「SPYがすでに動いた」を見るモデルだとすれば、Joint HMMは「債券、クレジット、ボラティリティが今いっせいに何を言っているか」を見るモデルだ。

ただし資産ごとに値の単位と大きさが違う。VIXは10~80のあいだで動き、SPYの日次リターンは-5%~+5%のあいだで動く。これらの値を生のまま一つのモデルに入れると、値が大きい特徴量がモデル全体を支配してしまう。そこで各資産を自分の過去分布を基準に標準化する(ローリングz-score)。「この値は最近の自分の履歴のなかでどれくらい高いか・低い」に置き換えて表現する方法だ。

この章で作るシステムの構造は3段階だ。**Macro HMM**は「今の市場全体はリスク選好か、リスク回避か」を判断する。**Micro HMM**は「この個別資産を今買ってよいか」を判断する。**Kelly**はその判断を実際のポジションサイズに変換する。読む順序は「市場全体の方向 → 個別資産のシグナル → ポジションサイズ」だ。

資産の数が増え、PCA(次元削減の手法)のような道具も出てきて複雑に見えるかもしれない。だが上の3段階構造を頭に入れておけば、各部分の実装もずっと追いつく。

初心者は、この章を「より難しいHMM」と読む必要はない。**市場全体をより広い画面で見るHMM**だと考えればよい。4章のHMMが1銘柄の体温だけを測る体温計だったとすれば、7章のJoint HMMは体温、血圧、脈拍、酸素飽和度を一緒に見る健康診断に近い。

単一資産HMM

SPYだけを見て市場状態を推定する

|

v

株式がすでに動いたあとでリスクに気づくことがある

Joint HMM

SPY + TLT + HYG + VIX + GLD を一緒に見る

|

v

債券、クレジット、ボラティリティの先行警告も反映する

重要なのは、特徴量が増えたこと自体ではない。異なる市場が出す警告を一つの観測ベクトルにまとめ、HMMに読ませることだ。

データ → 特徴量ベクトル → HMM学習 → レジーム確率の推論 → ポジション
サイズ調整

流れは4章と同じである。変わったのは特徴量ベクトルの幅だけだ。4章ではリターン、ボラティリティ、モメンタムのような少数の数字を見た。7章では複数市場をまとめた約20個の数字を見る。

この章の流れ

- ・ 全体パイプライン—6段階の構造
- ・ なぜクロスアセットなのか
- ・ bar単位のノイズ除去(Kalmanフィルタ)
- ・ 20次元のセッション観測ベクトルを作る
- ・ Layer 1 —Macro HMM(日足)
- ・ Layer 2 —Micro HMM(セッション)
- ・ macro_biasをKellyポジションサイジングに反映する

学習目標

この章を読み終えると、次のことができるようになる。

- ・ 単一銘柄のHMMではなく、複数資産を同時に入力にする理由を説明できる。
- ・ Kalmanフィルタでリターンのノイズを除去する理由がわかる。
- ・ 18資産+2つのマイクロストラクチャ指標=20次元観測ベクトルの構成原理がわかる。
- ・ Layer 1(Macro)とLayer 2(Micro)の役割分担を説明できる。
- ・ macro_biasがKellyのポジションサイズに掛け合わされ、市場方向と個別シグナルが結合される過程を理解する。

この章のコード: `run_rentec_joint_hmm_v7.py`

この章では、クロスアセット観測値と2層のHMMが、なぜ単一資産HMMよりも安定したレジーム判定をもたらせるのかに集中する。

7.0 全体パイプライン—6段階

第4章で見た単一資産HMMはSPY価格のみを入力に取っていた。そのため債券の急反発やドル高のようなサインを取り逃した。この章ではSPYひとつではなく、SPY+TLT+USD+VIXなど複数資産を同時に学習させるJoint HMMへ拡張する。

NOTE

SPYのみを見るより、SPY+TLT+USD+VIXと一緒に見ると、市場レジームの転換を平均で**2.3日早く**検出できる(2018–2024のバックテスト基準)。債券・金・VIXのように株式より先に反応する資産と一緒に見るからだ。

まずは全体の流れを表で確認しよう。

段階	内容
1. 資産ダウンロード	17の観測資産+24の取引資産の5分足データ

段階	内容
2. Kalman フィルタ	各資産の bar リターンのノイズ除去
3. セッション集計	5分足78本を1セッションにまとめて20次元観測ベクトル化
4. Macro HMM	日足単位で市場全体の方向 (macro_bias) を判定
5. Micro HMM	セッション単位で各資産の上昇確率 (p_up) を計算
6. Kelly sizing	p_up と macro_bias を掛けてポジションサイズを決定

なぜHMMが2つなのか？

- ・ **Macro HMM** は市場全体の方向を判断する。「今の市場はリスクを避ける時期か、取りに行く時期か？」
- ・ **Micro HMM** は個別資産のエントリー・エグジットのタイミングを判断する。「Macroが定めた条件のなかで、この資産を今買ってよいか？」

この2層を切り分けることで、システムがなぜその判断を下したのかを後から追跡できる。たとえばMicro HMMがAAPLの上昇確率を高いと見ても、Macro HMMがRisk-Offを示せばポジションサイズは小さくなる。逆にMacroがRisk-OnでMicroも上昇を示せばポジションは大きくなる。要するに「良い個別シグナルでも市場全体が危険なら小さく入る」という運用原則をコードで実装した構造だ。

7.1 Why Cross-Asset?

WARNING

単一資産HMMは株価だけを見てレジームを判断する。だが現実にはVIXが急騰すれば、次のbarの株式リターンの分布そのものが変わる。つまり他の資産の動

きが株価に影響を与える。この現実を反映するには、クロスアセットの特徴量をモデルに入れる必要がある。

単一銘柄HMMの問題はタイミングだ。株価がすでに下落したあとでようやくリスクレジームに転じたと判断する。警告灯がつくときには、もう損失が発生している。

クロスアセットのアプローチは違う。債券、金、VIXのように株価より先に動く資産のシグナルを併用することで、この遅れを縮める。

区分	単一銘柄HMM	クロスアセット Joint HMM
観測	SPYリターンのみ	SPY+TLT+GLD+HYG+UUP+...
レジームシグナル	株価下落後に検出	債券・金・VIXの上昇が先に出るので早期検出

どの資産がどの資産より先に動くかはデータで確認できる(先行・遅行関係、lead-lag)。たとえばドル指数(UUP)は金(GLD)を約3日先行し、ナスダック(QQQ)はハイイールド債(HYG)を約3日先行する。自動計測結果の全体表はリポジトリで確認できる。

7.2 bar単位のノイズ除去—Kalmanフィルタ

生の5分足リターンはノイズが大きい。1~2個の外れ値のせいでモデルが誤ったレジームを判定することもある。そこで各資産のbarリターンをKalmanフィルタで一度ならしてから、モデルに入れる。

Kalmanフィルタは「これまで見た観測たち」を加重平均して本物のシグナルを推定する道具だ。移動平均に似ているが、新しい観測が入るたびに自動で重みを調整する点が違う。

この章のフィルタにはひとつ特徴がある。前の推定値を少し0の方向に引き戻すという点だ。価格水準(たとえばSPYが450ドル)を追うときは「次の値も似たような値だろう」と仮定する。しかしリターンは平均0付近で振動する。だから「特別なシグナルがなければリターンは0に回帰する」という仮定を弱く反映する。

さらにこのフィルタは**causal(因果的)**である。未来の値を覗かず、過去と現在だけで更新する。バックテストにおけるlook-ahead bias(将来情報の漏れ)が生じない。

Kalman stepはFで状態と共分散を予測し、観測誤差をK倍して補正する。Fは遷移、Qは状態不確実性、Rは観測noise。fit_seriesは時刻順の反復である。第6章のF=1から始め、多資産では行列次元と時刻も検査する。

7.3 20次元のセッション観測ベクトル

1日の取引時間は5分単位に分割される。5分足が1日78本集まると、それがひとつの**セッション**になる。1セッションごとに資産別のリターンとボラティリティを計算して、20個の数値に圧縮する。この束が**観測ベクトル**だ。HMMはこのベクトルを1つずつ受け取ってレジームを判定する。

20の次元は3つのグループに分かれる。

グループ	含まれるもの	意味
A —コア資産(10個)	SPY、QQQ-SPY、TLT、SPY ボラティリティ、GLD、USO、 UUP、EEM、HYG、XLE	市場方向・安全資産・マクロ
B —セクター・国際(8個)	IWM、XLF、VNQ、EFA、 XLV、XLI、XLP、UNG	セクターごとの相対的な強弱
C —マイクロ(2個)	SPYの始値-終値リターン、 SPY-QQQのセッション内相 関	セッション内の微細構造

なぜこの18資産なのか? 3つの基準で選んだ。

- レジームの先行性:** 債券(TLT、HYG)、金(GLD)、VIXは株式より先に動く。
- 相関の多様性:** 似た動きをする資産をいくつか入れても新しい情報にはならない。
SPY・QQQ・IWMは時価総額レジーム、TLTは金利レジーム、GLD・VIXは恐怖レジームをそれぞれ代表する。
- データの利用可能性:** 十分に長い過去データが必要。

資産構成を入れ替えるときは2つを先にチェックしよう。追加する資産が既存資産と相関係数が高すぎないか、そして十分な過去データがあるかだ。

資産クラス別のMacro感応度

資産群はequity、em、commodity、credit、fx、safe_haven、volatilityなどに分類できる。Risk-Offで株を減らすのは仮説であり、金や債券も常に防御的とは限らない。基礎比重×regime倍率に総risk上限を適用する。各行に資産群・regime・倍率・検証期間を記録する。

7.4 Layer 1 —Macro HMM(日足)

Layer 1は日足データで市場の大きな流れを判定する。入力はSPY、TLT、GLD、HYGなど主要資産の5日ローリングリターンに、SPYボラティリティとSPY-TLT相関を加えた10次元ベクトルだ。HMMは市場を3つのレジーム(Risk-Off / Neutral / Risk-On)に分ける。

毎日新しい観測が入ってくると、Forwardアルゴリズムで「今どのレジームにいる確率か」を更新する。結果はたとえば[Risk-Off 0.7, Neutral 0.2, Risk-On 0.1]のような分布だ。

macro_bias —分布を1つの数値に圧縮する

3つのレジーム分布をそのまま使うとKellyへ適用するのが面倒だ。そこで1つの数値にまとめる。各レジームで想定されるSPYの5日リターンを、現在のレジーム分布で加重平均し、標準偏差で割って-1から+1の範囲に収める。

- ・ macro_bias = -1に近いほどRisk-Off(防御)
- ・ macro_bias = +1に近いほどRisk-On(攻め)

レジーム	SPY 5日リターン	TLT 5日リターン	SPYボラティリティ
Risk-Off	負(↓)	正(↑)	高い
Neutral	中立	中立	低い
Risk-On	正(↑)	負(↓)	低い

Risk-Onでは株式が上がり、債券が下がり、ボラティリティが低い。Risk-Offはその逆。Neutralはどちらも大きな動きのない静かな状態だ。

なぜレジームの持続性を強制するのか？

遷移行列の対角に大きい事前疑似観測数を与えると持続を促せる。3状態で対角8・ほか1は例で、大きすぎると転換検出が遅れる。seed別の収束・占有率・OOS遅延を比較し、失敗fitを採用しない。

7.5 Layer 2 —Micro HMM(セッション)

Layer 2は**セッション単位**で短期レジームを検出する。4つのレジームに分けて、入力
は7.3で作った20次元の観測ベクトルだ。

PCAでまず次元を減らす。

20次元をそのまま使うと2つの問題が生じる。

1. **データが足りない**。学習ウィンドウ360セッションに対してモデルパラメータが多すぎる。
2. **情報が重複する**。SPY-QQQの相関は約0.9だ。2つの次元がほぼ同じ情報を二重に伝えてしまう。

PCA(主成分分析)はこの2つの問題を一度に解決する。20次元を互いに独立な15次元に圧縮する。その結果をHMMに入力する。

PCA_DIM=15、N_ENSEMBLE=3、TRAIN_SESSIONS=360、REFIT_SESSIONS=40
は圧縮次元・fit数・過去session数・再学習間隔の例だ。PCA次元は観測数や特徴数を
超えられない。scalerとPCAは学習区間だけでfitし次回まで固定する。普遍的最適
設定ではない。

7.6 Walk-forward構造

モデルを一度学習して終わりにすると、市場構造が変わったときについていけない。だから一定の周期で再学習する。これが**walk-forward(前進検証)**だ。

```
0—————360 | 361————400 | 401————440 | ...
└─ 訓練 360 ─┘   00S 40           再学習 + 00S 40
```

再学習のたびに直前の360セッションでMicro HMMを新しく学習し直す。資産のボラティリティ分布は時間とともに変わるので、標準化ツール(StandardScaler)とPCAも一緒に学習し直す。こうすれば市場構造の変化にモデルが追従し続けられる。

7.7 macro_bias -> Kelly変調

ここで2つのレイヤーの結果を1つに統合する。Layer 2のp_up(上昇確率)から出発して、Layer 1のmacro_biasでサイズを調整する。

```
def kelly_fraction(p_up, macro_bias, asset_class):
    edge = 2 * p_up - 1.0
    class_amp = MACRO_CLASS_AMP.get(asset_class, 0.5)
    adjustment = macro_bias * class_amp
    kelly_adj = edge * (1.0 - adjustment)
    return float(np.clip(kelly_adj * 0.5, KELLY_MIN, KELLY_MAX))
```

数字で追ってみよう。Microがp_up=0.58(弱い上昇シグナル)を出し、Macroがmacro_bias=-0.7(Risk-Off)を示し、資産が株式(equity)だとする。

- ・ エッジ(edge): $2 \times 0.58 - 1 = 0.16$
- ・ Macro調整: Risk-Off+equity -> ポジションを減らすべき
- ・ 最終ポジション: 約0.108(元のエッジ0.16のおよそ2/3)

一言でまとめると、「Microが上昇シグナルを出してもMacroがRisk-Offなら株式ポジションを減らす」。同じシグナルでも市場環境次第で入る大きさが変わる。

7.8 P(up)の計算—上昇確率はどう作られるか

Layer 2が出力するp_upはどう作られているのか? 各レジームごとに「この資産の次のリターンが0より大きい確率」を計算したうえで、現在のレジーム分布で加重平均する。たとえばRisk-Onレジームで上昇確率が70%、Neutralで50%、Risk-Offで30%だとすると、今のレジーム分布が[0.1, 0.2, 0.7]のとき $p_{up} = 0.1 \times 0.7 + 0.2 \times 0.5 + 0.7 \times 0.3 = 0.38$ となる。

売買のしきい値はシンプルだ。p_up > 0.53ならロング、p_up < 0.47ならショート、その間はニュートラル。

7.9 結果を生んだ構造的な要因

下の表は、成果改善に寄与した構造的な要因をまとめたものだ。数字そのものよりも、どの設計がどの問題を減らしたかを見るのが重要だ。

要素	効果
20次元のクロスアセット観測	単一資産よりレジームを早く検出
Macro+Microの2レイヤー	日足の方向バイアス×セッション上昇確率の結合
レジーム持続性の強制	ノイズによる過剰な切り替えを防止
Walk-forward再学習	市場構造の変化に継続的に適応
PCA 15次元	共線性を取り除き学習を安定化
Kelly Half×macro_bias	資産クラス別の非対称なポジション調整

コードで見る—合成データでMacro HMMの学習の流れを確認する

市場dataの前に合成例で観測vector->HMM学習->推論を追う。市場では資産時刻を揃え、returnとvolatilityを計算し学習済みscalerで変換する。例はdownloadや運用repoなしで流れを理解するためのものだ。

このコードが示すのは、20次元の観測ベクトル500個を作り->60%でMacro HMMを学習し->Forwardアルゴリズムでレジームを推論する、までの最小フローだ。

```
import numpy as np
from hmmlearn.hmm import GaussianHMM
```

```

SEED, T = 42, 500
N_MACRO_STATES = 3
rng = np.random.default_rng(SEED)

X = rng.normal(0, 1, size=(T, 20))

train_len = int(T * 0.6)
macro_hmm = GaussianHMM(
    n_components=N_MACRO_STATES,
    covariance_type="full",
    n_iter=200,
    random_state=SEED,
)
macro_hmm.fit(X[:train_len])

alpha_oos = []
for t in range(train_len, T):
    alpha_t = macro_hmm.predict_proba(X[:t + 1])[-1]
    alpha_oos.append(alpha_t)

alpha_oos = np.asarray(alpha_oos)
regime_oos = alpha_oos.argmax(axis=1)

print(f"관측 벡터 shape: {X.shape}")
print(f"학습 완료: {N_MACRO_STATES} 상태")
print(f"OOS 레짐 분포: {np.bincount(regime_oos)} (OOS {T -
train_len}봉)")

```

確認すべき出力: - 観測ベクトルshape: (500, 20) —時点×次元が合っているか - レジーム分布: [..., ..., ...] —3つのレジームがすべて0でないか(1つのレジームだけが取られていれば学習失敗のサイン)

失敗したら見るべきところ: - 1つのレジームだけが出る場合はrandom_stateを変えて再試行するか、データに十分なばらつきがあるか確認しよう。 - hmmlearnの内部関数

`_compute_log_likelihood`、`_do_forward_pass`は非公開APIだ。ライブラリのバージョンによってインターフェイスが変わりうる。

Xの行は時刻、列は資産別return・volatility等で、学習時の列順を推論でも維持する。休場日を未来価格で埋めず、資料不足は除外または明示的欠測方針で無効にする。合成結果は投資成績ではない。

Exercises

1. Macro HMMがRisk-Offを示し、Micro HMMがSPYの上昇確率を0.58と予測したとき、最終ポジションサイズはどう決まるか？ 7.7節の式に沿って自分で計算してみよう。
2. 現在のマクロ資産に金(GLD)を追加した場合、レジーム検出はどう変わると考えられるか仮説を立て、資産選択基準(7.3節)に照らして妥当性を評価してみよう。
3. Walk-forward再学習の周期を短くするとどんな長短があるか考えてみよう。

まとめ

この章で必ず覚えてほしいこと:

- ・市場レジームを正確に見るには、ひとつの資産だけでなく複数の資産を一緒に見る必要がある。債券・金・VIXは株式より先に動くことが多いからだ。
- ・システムは2層に分かれる。**Macro HMM**は市場全体の方向(`macro_bias`)を、**Micro HMM**は個別資産の上昇確率(`p_up`)を判断する。
- ・2つの結果を掛け合わせてKellyのポジションサイズを決める。良い個別シグナルでも市場全体が危険なら小さく入る。
- ・モデルは一度学習して終わりにせず、walk-forwardで再学習し続ける。

レイヤー	入力	出力	役割
Macro HMM(日足、3状態)	主要資産10次元	<code>macro_bias</code> ∈ [-1, +1]	マクロ方向バイアス

レイヤー	入力	出力	役割
Micro HMM(セッション、4状態)	18資産+マイクロ (20次元)->PCA 15 次元	資産別p_up	個別資産の方向確率
Kelly sizing	p_up×macro_bias× 資産クラス	ポジションサイズ (%)	リスク調整

次の章(第8章): Two-layer HMMの深掘りを扱う。アンサンブルのサブモデル構造と、レジーム転換が近づいたときに確率を計算する方法を見ていく。

さらに読みたい人へ

- ・ **論文:** Ang & Bekaert (2002), “International Asset Allocation with Regime Shifts”—クロスアセット・レジームモデルの嚆矢
- ・ **論文:** Hamilton (1989), “A New Approach to the Economic Analysis of Non-stationary Time Series”—Markov Switchingモデルの基礎
- ・ **書籍:** Ilmanen, Expected Returns (Wiley, 2011) —資産クラス間の相関とレジームに関する研究

8章. 2階層HMMアンサンブル—Layer A × Layer B

市場はいつも同じように動くわけではない。株価が一方向に上がり続けたり下がり続けたりする**トレンド相場**もあれば、一定の範囲内で上下を繰り返す**もみ合い相場**もある。RSIのような指標は、この二つの状況で挙動が違う。ブレイクアウトシグナルも、ボラティリティが大きいときと小さいときで結果が変わる。ベテラントレーダーはチャートを見て「いまはトレンドに従うべき」か「いまは反発を待つべき」かを感覚的に判断する。だがその感覚は人によって異なり、一定でもないので、コードに落とし込みにくい。個人投資家の多くは、市場の状況(レジーム、regime)に関係なく同じシグナルを使う。トレンドが強い相場で機能するモメンタムシグナルを、もみ合い相場でもそのまま使い、ボラティリティが大きい局面でも普段と同じ基準でエントリーする。シグナル自体

が悪いわけではない。そのシグナルがうまく当たる状況と、うまく当たらない状況を区別していない点が問題だ。

レジームという言葉に馴染みがなければ、天気を思い浮かべればよい。同じ服が春と真冬で適切でないように、同じ売買シグナルがすべての市場局面で通用するわけではない。市場にも季節のような局面があり、まずその局面を区別するところから始める必要がある。

4・5・7章の復習ボックス - 4章で単一資産HMMを作り、「隠れ状態」という概念に慣れた。- 5章ではForward再帰によって5分足ごとに状態確率を更新する `step_forward()` を実装した。- 7章では複数資産を一つのモデルにまとめるJoint HMMへ拡張した。

この章は、その上に**サブモデル8個**を積み重ね、多様なアルファ源を一つのEnsembleにまとめる。HMMが「観測データから隠れ状態を推定するモデル」だという点さえ覚えていれば、十分についてこられる。

2階層HMMアンサンブルは、この問題を構造的に解決する。**Layer A(マクロHMM)**は複数資産の動きをまとめて見て、現在の市場状態を分類する。**Layer B**には8個のエキスパート(専門家)サブモデルがあり、それぞれが特定の市場状態で最もよく機能するように設計されている。Layer Aが「いまは平均回帰に有利な状態」と判断すれば、平均回帰エキスパートのシグナルが活性化される。「トレンドレジーム」と判断すれば、モメンタムエキスパートの比重が高くなる。

この構造は機械学習のMixture of Experts(複数の専門家モデルから状況に合うものを選んで使う方式)の考え方と同じだ。重要なのは、万能シグナル一つに依存しないこと。いまの状況に合う戦略を選び、合わない戦略は比重を下げる。

初心者は、2階層構造を会社組織のように考えると分かりやすい。Layer Aは市場全体を見るチームリーダーで、Layer Bはそれぞれ得意分野を持つ専門スタッフである。リーダーが「今日は危機レジームだから攻撃的なシグナルは抑えよう」と判断すれば、専門スタッフの個別シグナルが強くても、最終ポジションに届く前にサイズが小さくなる。

2つのレイヤーに分ける理由は、1つのモデルに違う種類の問いを同時に背負わせないためである。日足のマクロレジームと5分足のエントリータイミングは時間スケールが違う。これを1つのモデルに押し込むと、モデルは大きな市場環境を学ぶべきなのか、短期のエントリーパターンを学ぶべきなのかを同時に解かなければならない。2層に分けると、問いが明確になる。

Layer Aの問い：
市場全体はいまどんな局面か？

Layer Bの問い：
その市場環境の中で、いま短期シグナルは何を示すか？

最終結合の問い：
このシグナルをポジションにしてよいか。サイズはどれくらいか？

この章の流れ

まず、なぜ二つのレイヤーが必要なのかを押さえ、その後で各レイヤーの内部構造と結合方法へ降りていく。

- ・ **8.1 Why Two Layers?** —時間スケールの違う二つのシグナルをなぜ分けるのか
- ・ **8.2 効果から確認** —この構造が実際に成果を出すのか
- ・ **8.3 Layer A** —マクロレジームを追跡する7状態HMM
- ・ **8.4 8個のサブモデル概要** —各モデルがどの市場で機能するか
- ・ **8.5 SubModelSpec** —サブモデルを宣言的に扱うデータ構造
- ・ **8.6 HMMForwardPredictor** —サブモデルの学習と推論
- ・ **8.7 W2 Wasserstein 投票** —外れ値を抑え、多数決を強化する仕組み
- ・ **8.8~8.10 結合と検証** —最終シグナルの合成、運用原則、まとめ

学習目標

この章を読み終えると、次のことができるようになる。

- ・ システムが二層に分かれていることを理解する。Layer Aは市場全体の状態(マクロレジーム)を分類し、Layer Bはその結果を受け取って8個の小さな予測モデルを組み合わせる。各層の役割と設計理由を説明できる。
- ・ 8個のサブモデルがそれぞれどの市場状況を狙うのかを、大きな絵で区別できる。
- ・ W2 Wasserstein 投票がなぜ単純平均より安全なのかを説明できる—突出した予測は抑え、多数意見は強化する。

- ・ レジームゲート(active_regimes)とは何で、なぜ必要なのかを説明できる。
- ・ RenaissanceEnsemble5m.step() が返す e_sym の意味を把握し、バックテストのコードで正しく受け取って使える。

8.1 Why Two Layers?

単一HMMで5分足を学習すると、性格の違う二種類のシグナルを一つのモデルに押し込むことになる。日足レベルのマクロ環境は数日から数週間かけてゆっくり変わる。一方、5分足レベルのエントリータイミングは数本のローソク足の中でも変わる。

二つのレイヤーは、最初は複雑に見える。だが各レイヤーの役割は単純だ。

- ・ **Layer A**は「今日の市場の空気はどうか」を判断する。
- ・ **Layer B**は「その空気の中で、いまの5分足シグナルが取引する価値があるか」を判断する。

二つの判断を一つのモデルに混ぜると、モデルはどのパターンを学ぶべきか方向を見失う。二レイヤー構成はモデル数を増やすための飾りではない。時間スケールの違う二つの判断を分離するための構造だ。

時間スケール	現象	必要なモデル
日(day)単位	マクロレジーム: トレンド/MR/危機	Layer A —7状態 HMM
分足(bar)単位	短期方向、 VWAP乖離、 Kalman速度	Layer B —8サブモデル Ensemble

Layer A は regime_mult で**シグナルの大きさ**を調節する。Layer B のサブモデルは**方向(sign)**を投票する。最終シグナルは二つの結果の積だ。

最終シグナル = e_sym (Layer B の方向) × regime_mult (Layer A の大きさ)

この一行が、この章の結論だ。以降の節はすべて e_sym と regime_mult をどのように作るかを説明している。

学習周期も異なる。Layer A は遅いマクロ変化を捉えるために日足スケールで学習し、Layer B は速い短期転換を捉えるために約10セッションの分足ウィンドウで学習する。

8.2 効果から確認—バックテスト性能

細部に入る前に、この構造がなぜ必要なのかから見る。単一モデルだけ使うと、特定のレジームに過剰に適合しやすい。二レイヤーとW2投票を入れる理由は、モデル一つの偶然の強さに頼らず、複数のシグナルが同じ方向を指したときだけより強い確信を与えるためだ。

バージョン	変更点	確認するポイント
基準バージョン	7状態、性能ベースの重みのみ	単一重み付け方式の限界を確認
W2投票導入	Wasserstein + k-means 多数決	レジーム転換区間でのシグナル安定性を確認

詳細な数値はリポジトリの実験ノートで確認する。本文では「何を追加したらどの失敗が減ったか」を中心に読む。

8.7 でこの違いがどこから生まれるのかを直接確認する。いまは「二レイヤー + W2投票は、単純平均よりレジーム転換区間を慎重に扱う」という点だけ頭に入れて、次に進もう。

8.3 Layer A: GlobalRegimeLayer

Layer A は `hmm_v5_core.py` の `GlobalRegimeLayer` クラスで実装される。このモデルはHMMの一種で、合計7つの市場状態を区別する。

入力は5分足だが、目的は個別銘柄の短期的な上下を見ることではない。ボラティリティ、出来高、セクター間の資金フローのように市場全体を説明するマクロなフィーチャーを追跡し、いまの市場がどの空気なのかを把握する。

7つのレジームそれぞれには、シグナル倍率(`regime_mult`)があらかじめ決められている。

state	名前	倍率	説明
0	DeepMR	0.55	極低ボラの平均回帰 -> シグナル減衰
1	QuietTrend	1.00	低ボラのドリフト (基本倍率)
2	TrendMod	1.10	緩やかなトレンド
3	TrendStr	1.40	強いトレンド
4	TrendBreak	1.70	ブレイクアウト/モメンタム—最大振幅
5	HighVol	0.35	高ボラのノイズ -> シグナル減衰
6	Crisis	0.10	危機/クラッシュ -> ほぼフラット

たとえばLayer Bが $e_sym = +1.0$ (強いロング) を出したのにLayer Aが Trend-Break(state 4) と判断すれば、最終シグナルは $1.0 \times 1.70 = 1.70$ になる。同じ e_sym でも Crisis(state 6) では $1.0 \times 0.10 = 0.10$ でほぼ空に近い。

三つのグループにまとめると意図が見える。

- ・ **MR/危機グループ (0, 5, 6)**: 市場が揺れているか危機状態の区間。方向性シグナルを抑え、ポジションサイズを小さくする。
- ・ **トレンドグループ (1, 2, 3)**: テレンドが生きている局面。基本～強い倍率を適用。
- ・ **ブレイクアウトグループ (4)**: ボックス圏を抜けてモメンタムが乗る局面。倍率を最大1.70まで上げる。

このマッピング値はコード上の定数 `_REGIME_A_SCALE` 辞書に入っている。¹

8.3.1 レジーム安定化—3バー連続確認

5分足が新しく入るたびに各 state の確率を更新する。このとき、確率が最も高い state をそのままレジームとして使うと、バーごとにレジームが変わってしまうことがある。市場の空気が1本のローソク足で反転することは稀なので、こうした頻繁な切り替えのほとんどはノイズだ。

¹Layer A の state 数は必ずしも7である必要はない。5状態で実験するときも倍率表を作り直さなくていいよう、state 数が変わっても自動で上の表に合わせるヘルパー(`_regime_mult_from_rank()`)が入っている。

そこで**3バー連続確認**方式を使う。新しい state が3本(=15分)にわたって連続して閾値確率以上を保ったときだけレジームが変わる。レジームラベルが每バー変わると戦略も每バー性格を変えることになり、結果は過剰売買と不安定なポジションにつながる。

8.4 Layer B: 8個のサブモデル概要

ここから Layer B に降りる。クラス構造を見る前に、**8個のサブモデルがそれぞれ何を見ているか**を先に整理しよう。サブモデルは二つのグループに分かれる。4個はHMMベースで、4個はカルマンフィルタ・移動平均・出来高など非HMM方式だ。HMMが見逃すシグナルを、非HMMモデルが独立に確認する役割を担う。

8.4.1 HMMサブモデル(代表2個)

モデル	フィーチャーの考え方	有効レジーム	役割
M3_vwap	VWAP乖離 + BB位置	全レジーム	汎用平均回帰 + 急落ショート
M5_RetKalman	5/10/20本リターン + Kalman速度	トレンド (1~4)	トレンド方向性の捕捉

M6_VolKalmanはvolatilityの平滑化、M7_MRFastは短い平均回帰窓に注目する。特徴列・過去窓・有効regime・単位を記せば役割が分かる。同じ情報のmodelを増やしても分散効果は自動的に生まれない。

レジームゲートの設計原理: すべてのモデルを常にオンにしておくわけではない。レジームごとに有効なシグナルが違うからだ。

- ・トレンドレジーム(1~4)ではボラティリティ専用モデル(M6)をオフにする。価格方向と無関係なシグナルはトレンド局面でノイズを増やすだけだ。
- ・Crisis レジーム(6、急落区間)ではトレンド追従モデル(M5)をオフにする。急落中にモメンタムシグナルを追うと、追加損失につながりかねない。

8.4.2 非HMMサブモデル(代表2個)

残り4個はHMMを使わない単純なルールベースのモデルだ。代表として二つだけ見よう。

モデル	model_type	投票基準
M8_Kalman	kalman	カルマンフィルタで平滑化されたリターンの符号 → 長期トレンド方向
M10_VolBreak	volbreak	出来高が普段より大きく載ったバーでのみ方向投票

残りの非HMMモデル(M11_EMAMom、M12_TODRet5b)は付録の表にまとめてある。以前のバージョンにあったM9_DailyMom(日足モメンタム)は、他モデルとシグナルの大きさの桁が違いすぎたため削除した。

サブモデルごとの寄与度(一つずつ外しながら性能変化を測るablation実験)の結果はリポジトリの実験ノートにまとめている。要約すると、平均回帰系のモデル(M3 など)の寄与度が最も大きい。

8.5 SubModelSpec —サブモデルをデータとして宣言する

サブモデルの種類と役割を見たら、次はこの8個がコード上でどう扱われるのかが気になるころだ。Layer B のサブモデルはクラスではなく**データとして宣言する**。SubModelSpec データクラスがサブモデル一つ分の全設定を保持する。新しいサブモデルを追加したいときは、クラスを新しく書くのではなく、SubModelSpec オブジェクトを一つリストに追加するだけでよい。

本文では中心となるフィールド3つだけ見る。

フィールド	意味
name	モデル識別子 (例: "M3_vwap")
feat_cols	このモデルが使うフィーチャー列のリスト
active_regimes	Layer A レジームゲート—None なら全レジームで有効

n_statesは潜在状態数、model_typeは種類、mu_clip_bpは状態平均returnの絶対上限bpだ。20bpなら-0.002〜0.002。active_regimesは上位状態による使用許可を定める。有効modelが0なら確信した0予測ではなく取引保留にする。

ここから先は実装の詳細だ。1回目に読むときは8.6と8.7をざっと目を通したあと、8.8まで飛ばしてもいい。8.8の結合フローと8.10のまとめを見れば、この章の大きな絵は完成する。詳細は2回目の読み返しで埋めれば十分だ。

8.6 HMMForwardPredictor: Layer B HMM Predictor

各HMMサブモデルは HMMForwardPredictor クラスで包まれる。このクラスがやることは二つだ。

- 1. **fit()**: 過去データでHMMを学習し、各状態の期待リターンを計算する。
- 2. **step()**: 新しいバーが入るたびに、現在の期待リターンを更新して返す。

8.6.1 fit() —学習段階がやること

BIC $k \cdot \log(n) - 2 \cdot \log L$ で状態数候補を比べられる。kは自由parameter、nは学習観測数、logLは同じ資料の尤度だ。低いほど複雑度との釣合いが良いが収益保証ではない。共分散形式に応じてkを数える。平均clipは小標本の極値が注文量を支配するのを防ぐ別方針だ。

確認する出力: `fit()` が `True` を返せば学習成功。 `False` なら入力データが不足している場合だ。

8.6.2 `step()` —新バーごとに期待リターンを更新

学習が終わると、5分ごとに `step(x_bar)` を呼ぶ。新しい観測値一つで各状態に対する確率を更新し、その確率と状態ごとの平均リターンの加重和で現在の期待リターンを返す。5章の単一HMMの `step_forward()` と同じパターンで、違いは単一モデルではなく4つのHMMサブモデルがそれぞれ独立に同じ計算を行う点だ。

失敗したら見る場所: 入力フィーチャーにNaNが混ざっていると、確率更新が壊れる可能性がある。まずはフィーチャー前処理でNaNを取り除くのが先だ。

8.7 W2 Wasserstein Voting —外れ値の抑制

8個のサブモデルの予測を単純平均すると問題が起きる。**外れ値のサブモデルが一つあると、全体のシグナルが汚染されかねない。**

ミニ例で感覚を掴もう。8モデルのうち7個が弱いLONG(例: +0.2)、1個が強いSHORT(-2.0)を出したとする。単純平均すると $(7 \times 0.2 + 1 \times (-2.0)) / 8 \approx -0.075$ で、多数意見と逆の方向になる。たった一つの外れ値が全体をひっくり返したわけだ。W2投票はこの外れ値の重みをほぼゼロに抑えたうえで多数派をさらに大きくし、結果を多数意見である弱いLONG側に引き寄せる。

Wasserstein距離(W2)は、二つの確率分布がどれだけ違うかを測る尺度の一つだ。他の距離関数(KLダイバージェンスなど)と違って常に有限の値を返すので、極端な予測にも安定して機能する。

W2投票は二段階で動く。

Phase 1 —はぐれ者を抑える。 各サブモデルはガウス形の予測分布(平均、分散)を出力する。全モデルのペアでW2距離を計算し、「他のモデルから平均してどれだけ離れているか」ではぐれ度を測る。はぐれているほど重みをほぼゼロに抑え、多数と近いモデルは通常の重みを保つ。

Phase 2 —多数派クラスタのブースト。 Phase 1 だけだと、LONGとSHORTで意見が真っ二つに割れたときシグナルが0近くに収束してしまう。Phase 2 はサブモデル

を二つの陣営 (LONG陣営/SHORT陣営) に分けたあと、多数派陣営の重みを増やし、少数派陣営を抑える。ここでの多数判定の基準は**個数ではなくシグナル大きさの総和**だ。LONGモデルが3個でもシグナルが弱く、SHORTモデルが2個でもシグナルが強ければ、SHORT陣営が多数派として選ばれる。

距離重みの例は $w_i = \exp(-d_i/T) / \sum_j \exp(-d_j/T)$ 、 $T > 0$ 。近い分布ほど大きい。類似性は予測精度ではない。成績と混ぜるなら各々正規化し混合比を事前固定して検証する。²

8.8 RenaissanceEnsemble5m.step() —二レイヤーの結合

step() は8個のサブモデルの出力を集めて最終売買シグナルを作る。呼び出し結果のうち最も重要な値は最初の e_sym だ。e_sym は生き残ったサブモデルのW2加重和から作られる **Layer B の方向シグナル**だ。一緒に返される残りの値(サブモデル別シグナルの辞書、ロング/ショート票数など)はデバッグと安全装置の判定に使う。

推論は確定足検査、上位更新、有効model選択、因果的予測、結果確定済みの過去予測から重み更新、合成とrisk制限の順だ。現在予測の未来returnで同時点の重みを作ると漏洩になる。更新間隔の単位が足かsessionかを記録する。

8.1 の結論—最終シグナル = e_sym × regime_mult —をコードに落とすと次のようになる。

このコードがやること: 5分足ごとに Layer B の方向シグナルと Layer A のレジーム倍率を掛けて、最終的なポジションシグナルを作る。意見が割れたらポジションを空にする(フラット)。

```
for bar_idx in range(trainBars, len(df_5m)):
    x_row = feat_df.iloc[bar_idx]
    prev_ret = log_returns[bar_idx - 1] if bar_idx > 0 else None

    e_sym, sub, v_long, v_short, n_voted = ensemble.step(
```

²実際のコードには性能ベースの重みとW2重みを混ぜられるブレンディング比率が用意されている。現在は0に設定されていて性能ベースの重みのみ使い、W2ブレンディングコードは今後の再実験のために残してある。

```

        x_row, actual_ret=prev_ret,
        regime_state=layer_a.dominant_state,
    )

    regime_mult = _REGIME_A_SCALE.get(layer_a.dominant_state,
1.0)
    signal      = e_sym * regime_mult

    vote_ratio = v_long / n_voted if n_voted > 0 else 0.5
    if vote_ratio < VOTE_THRESH and vote_ratio > (1 -
VOTE_THRESH):
        signal = 0.0

    positions[bar_idx] = signal

```

e_sym は方向シグナル、regime_mult は大きさの調整、vote_ratio は安全装置。三つが掛け合わさって最終ポジションになる。

8.9 設計原則のまとめ

設定定数を二つ決めれば、システムの時間尺度が決まる。

- ・ **再学習周期 (REFIT_EVERY=156)**: およそ2セッションごとにモデルを再学習する。長すぎるとレジーム転換を逃し、短すぎると学習窓が小さすぎてBICがうまく機能しない。2セッションがその折衷点だ。
- ・ **最小保有バー (MIN_HOLD_BARS=78)**: 一つのポジションを最低1セッション(=1日)維持する。売買を頻繁にするほど手数料・スリッページが積み上がって収益を削ってしまうためだ。

取引コストは思ったより速く積み上がる。片道の取引コストを8bpと仮定すると、年1000回の回転だけで年16%のコストになる。最小保有時間という一行の設定が、このコスト爆発を防ぐ。

コードで見る—Two-Layer HMM の最小実装

二層の短いlistingは構造例であり完成学習programではない。scaler、列順、model、状態意味、学習終了時刻を一組で保存・読込する。どれか不一致なら新規signalを保留する。

このコードがやること: Layer A の優勢レジームで Layer B のサブモデルをふるい、生き残ったモデルの $P(\text{up})$ を加重平均してKellyサイズに変換する。

```
import numpy as np

def combine_two_layers(global_alpha, submodels, t,
                      kelly_half=0.5):
    dom_state = int(global_alpha[t].argmax())

    p_ups, weights = [], []
    for model in submodels:
        if dom_state not in model.active_regimes:
            continue
        p_ups.append(model.predict_up(t))
        weights.append(model.rolling_score)

    if not p_ups:
        return 0.0, 0.0

    w = np.array(weights) / np.sum(weights)
    p_up = float(np.dot(w, p_ups))

    edge = 2 * p_up - 1.0
    kelly = float(np.clip(max(edge, 0.0) * kelly_half, 0.10,
                          1.0))
    return p_up, kelly
```

確認する出力: p_{up} は0～1の間、 $kelly$ は0.10～1.0の間でなければならない。 p_{up} が0.5近くなれば方向の優位がほぼなく、0.55以上なら上昇側に優位があるという意味だ。

Exercises

1. サブモデルを一つ無効化して、全体の成果がどう変わるかを測定せよ。こうした分析を **ablation study(除去実験)** と呼ぶ。構成要素を一つずつ外しながら、全体の性能に与える影響を数値で確認する方法だ。
2. 投票重みの計算に使われる Wasserstein W2 距離を KL ダイバージェンスに置き換えてみよ。二つの方法を比べながら、KL ダイバージェンスが W2 より不安定になる状況がいつなのかを把握せよ。
3. `HMMForwardPredictor.step(x)` を最初に呼ぶ100本ぶんの区間を warm-up 区間という。この区間では各状態の確率がまだ安定しておらず、予測が揺れることがある。warm-up 区間でポジションサイズを自動的に小さくするロジックを追加してみよ。

8.10 この章で必ず覚えておくこと

この章で、8個のサブモデルを集めて一つのアンサンブル(Ensemble)を完成させた。最も重要な三つだけでもう一度整理する。

1. **時間スケールの違う二つの判断を分離する。** Layer A は長期の市場の空気(マクロレジーム)を、Layer B は短期の売買方向を担当する。一つのモデルに二つの役割を混ぜると学習がぶれる。
2. **レジームゲートで合わないシグナルを切る。** トレンドレジームで平均回帰モデルを、危機レジームでモメンタムモデルを無効化する。このひと手間のフィルタだけでシグナルの質が大きく上がる。
3. **単純平均の代わりに Wasserstein 投票を使う。** はぐれ者のモデルは重みを抑え、多数意見は大きくする。モデル一つが間違ったシグナルを出しても、全体への影響が広がらない。

まとめ表は次のとおり。

レイヤー	クラス	役割
Layer A	GlobalRegimeLayer	マクロレジーム -> シグナル大きさ倍率
Layer B	RenaissanceEnsemble5m	8個のサブモデル -> 方向シグナル e_sym
結合	e_sym × regime_mult	最終ポジションシグナル

この章を本全体のパイプラインに位置づけると、次のようになる。

市場データ

- > Layer A: 大きな市場レジームを読む
- > Layer B: 短期シグナルを複数のHMMで読む
- > Wasserstein投票: 外れ値モデルの影響を抑える
- > regime_mult: 市場状態に合わせてシグナルを調整する
- > 最終ポジションシグナル

つまり8章は、ディープラーニング章に進む前に、複数の機械学習モデルを組み合わせ、シグナルをより安定させるための章である。あとでSACを付け加えるときも、この構造がそのまま役立つ。RLは生の価格だけを見るのではなく、こうして整理されたレジームとシグナルを状態ベクトルとして受け取り、その上で行動を学ぶ。

一行まとめ

`signal = e_sym × regime_mult`

この章のすべての内容は、この一行をどう作るかについてのものだ。

次章の案内: 9章では、こうして作った e_sym × regime_mult のシグナルを実際の発注サイズに変える **Half-Kelly ポジションサイジング**を見ていく。10章では複数のHMMシグナルをフォワード方式でまとめるアンサンブル構造を整理し、11章からは単一銘柄レジーム戦略とは別軸の、統計的ペアアービトラージへ進む。

参考文献

- ・ 論文: “Wasserstein Distances in the Analysis of Time Series and Dynamical Systems”—W2 距離を時系列分析に適用した研究
- ・ 論文: Breiman (2001), “Random Forests”—Ensemble 技法の理論的基盤を確立した論文
- ・ コード: pyEMD ライブラリ—Hierarchical HMM 実装の参照用

第9章 ハーフ・ケリー によるポジションサイ ジング

Part IV —シグナルをポジションに変える(9～10章) ここまでは「市場の状態をどう読むか」を扱ってきた。ここからはその判断を実際のポジションサイズと方向シグナルに変換する。第9章は一回にどれだけ建てるか、第10章は複数のHMMシグナルをどう統合するかを扱う。一読目では公式を覚えるより、「良いシグナルでも、ベットが大きすぎれば壊れる」という原則をつかんでほしい。

個人投資家の多くは、ポジションの大きさをほとんど意識しない。「これに10%入れよう」「100株買おう」と、丸い数字で決めることが多い。その数字は計算結果というより、感覚に近い。戦略、シグナル、エントリー、エグジットには注意が集まる一方で、ポジションサイジング(一度にいくら投資するかを決める作業)は後回しにされがちだ。

クオントのアプローチでは事情が違ふ。ポジションサイジングは実際に利益が確定する場所であり、損失が積み上がる場所でもある。同じシグナルでも2%の比率で入れば耐えられる変動だが、30%で入ると口座全体を揺さぶる事件になる。シグナルの質とポジションの大きさは切り離して考えられない。

損失の数学は対称ではない。10%の損失を取り戻すには約11%の利益が必要だ。50%の損失なら100%、75%の損失なら300%なければ元に戻らない。深くなるほど、回復は構造的に難しくなる。市場のせいではなく算術のせいだ。自分のエッジ(優位性)より大きくポジションを取れば、収益性のある戦略でさえ最終的には損失で終わる。

ケリー基準(Kelly Criterion)はこの問題に数学的に取り組む。勝率と損益比をもとに、理論的に最適なベット比率を計算する公式である。ハーフ・ケリー(Half-Kelly)はフル・ケリーの50%だけをベットする保守的な変形だ。ドローダウンを抑えながら、長期の複利成長の多くを維持できる。本書のシステムはここからさらに一步進み、6ファクターのサイジングモデルを使う。フル・ケリーを半分にしたうえで、市場状態・ボラティリティ・マクロ条件などに応じて追加で調整する。

この章で最も大事な点は、ケリーを「大きく稼ぐための公式」と誤解しないことだ。ケリーはむしろ、過度なベットを止めるための仕掛けである。 $p_{up}=0.53$ のようにエッジが小さいなら、モデルがどれだけ複雑でも基本ポジションは小さくなる。確率推定が少しずれただけでエッジが消えてしまうからだ。

最初に適用するときは、ケリーの結果をそのまま発注サイズに使うのではなく、「自分のモデルが言うエッジはどれくらい小さいのか」を確かめる診断値として見るほうがよい。実戦のポジションは、ハーフ・ケリー、最大ポジション上限、銘柄別の損失上限、ポートフォリオ全体のエクスポージャー上限のすべてを通った後でのみ決定すべきだ。ケリーはサイジングの出発点であって、最終結論ではない。

本書全体のパイプラインで見ると、ケリーサイジングは**モデルの判断を、リスクにさらず資金の大きさへ翻訳する段階**である。HMMとアンサンブルは方向と確率を伝える。ケリーは、その確率優位に対してどれくらい大きく賭けるべきかを計算する。

```

モデル出力
p_up = 0.53
上昇方向に小さな優位がある
    |
    v
Kelly / Half-Kelly
  
```

小さな優位なので、小さなポジションだけ許可

|

v

リスク管理

最大サイズ、損切り、ポートフォリオ上限をもう一度確認

重要なのは、 $p_{up} = 0.53$ が「53%確信している」ように聞こえても、実際にはかなり小さな優位だという点である。似た状況が100回あれば53回当たり、47回外れるかもしれないという意味だ。この程度の優位に大きく賭けると、数回の連敗だけで口座が大きく揺れる。

予測モデルの役割：
少し有利な方向を探す。

ケリーの役割：
その有利さが小さければ、ポジションも小さくする。

リスク管理の役割：
それでも大きすぎれば、もう一度減らすか止める。

だからケリーは攻撃ボタンではない。翻訳機であり、ブレーキである。「良いシグナルだから大きく入る」ではなく、「この程度の優位なら、この大きさまでしか許可しない」という考え方に近い。

この章の流れ

まずケリー公式そのものを理解し、次に本システムがケリーにどんな補正係数を掛けているかを一つずつ見ていく。

1. ケリー公式の直観と、ハーフ・ケリーを使う理由(9.1)
2. サイジング関数 `calculate_kelly_size()` 全体の構造(9.2)
3. 6つの補正係数の役割(9.3~9.7)
4. 5分足向けの簡略版 `fractional_kelly()` と、2つの実装の比較(9.8~9.9)
5. `KELLY_MIN` を0ではなく0.10にした設計意図(9.10)

学習目標

この章を終えると次のことができるようになる。

- ・ ケリー基準の公式と、ハーフ・ケリーを使う理由を説明できる
- ・ `calculate_kelly_size()` がどのような流れでポジションサイズを決めるかを追える
- ・ 6つの補正係数が「それぞれ何を抑え、何を増やすのか」を区別できる
- ・ 5分足向けの簡易実装 `fractional_kelly()` の中心アイデアを応用できる

9.1 ケリー基準の基本

ケリー公式は1956年にベル研究所で生まれた。J. L. ケリーは情報理論を使い、「同じ種類のベットを繰り返すとき、資本を長期的に最も大きく増やすベット比率」を計算した。その後、数学者エドワード・ソープがギャンブルと株式に応用し、複数のクオンツファンドも類似の手法を採用してきたと伝えられている。

まずケリー公式の核心を押さえる。**短期の勝率を上げるための公式ではない**。同じ取引を数百回繰り返すとき、資本が長期的に最も大きく成長するように、一度にいくらベットするかを決める公式だ。

IMPORTANT

フル・ケリーは多くの場合、攻めすぎである。ケリーが理論的に最適となるにはエッジを正確に知る必要があるが、現実には不可能だ。エッジを実際より大きく見積もると、フル・ケリーは大きな損失につながる。本書ではハーフ・ケリー以下を使う。

勝つか負けるかの二値ベットで、勝つ確率を p 、勝ったときの利益と負けたときの損失の比を b とすると、最適ベット比率は次のようになる。

$$f^* = \frac{p \cdot b - (1 - p)}{b}$$

株式取引では平均利益と平均損失が同じくらいになることが多いので、 $b \approx 1$ と置くと式が簡潔になる。

$$f^* = 2p - 1 = 2 \times \text{edge}, \quad \text{edge} = p_{\text{up}} - 0.5$$

たとえば $p_{\text{up}} = 0.53$ ならば $\text{edge} = 0.03$ 、フル・ケリーは6%、ハーフ・ケリーは3%である。

$p_{\text{up}} = 0.53$ は「この取引が53%確実だ」という意味ではない。似た状況が100回繰り返されたとき、平均して53回は上昇したという統計的推定にすぎない。残りの47回は外れる。だからポジションサイズは確信の強さで決めない。小さな確率優位を長期にわたって安定して反復できるよう、保守的に決める必要がある。

ハーフ・ケリーを使う理由: 現実には p を正確に知ることはできない。フル・ケリーは理論上最適だが、推定値に誤差が混ざると成績が大きく揺れる。ハーフ・ケリー ($f^*/2$) は長期リターンの相当部分を残しつつ、最大ドロウダウンを大きく抑える。ハーフ・ケリーではベット比率が結局のところ edge そのものになる。この後の節で見るすべての補正係数は、この edge に掛けられる。

9.2 calculate_kelly_size() の構造

v7.py のサイジング関数は、ハーフ・ケリーが計算した基本の edge に6つの補正係数を順番に掛けて最終的な発注サイズを決める。係数の中身は次節以降で扱うので、ここでは「どの順番で、何を掛けるか」という構造だけ押さえる。

以下は実際の関数の骨格だ。全体の実装は `run_rentec_joint_hmm_v7.py` で確認できる。

```
KELLY_MIN, KELLY_MAX = 0.10, 1.0
THRESHOLD = 0.53          # p_up > THRESHOLD -> long entry

def calculate_kelly_size(
    p_up, direction,
    vol_mult=1.0, macro_gate=1.0, novelty_gate=1.0,
    er=0.0, macro_bias=0.0, macro_class_amp=0.0,
```

```

) -> float:

    edge = (p_up - 0.5) if direction > 0 else ((1.0 - p_up) -
0.5)
    if edge <= 0:
        return 0.0

    f = edge

    f *= vol_mult * macro_gate * novelty_gate

    edge_max = max(THRESHOLD - 0.5, 1e-6)
    return float(np.clip(f / edge_max, KELLY_MIN, KELLY_MAX))

```

読むときに注目すべき箇所は二つある。一つ目は `if edge <= 0: return 0.0` — エッジがなければ関数はすぐ0を返す。二つ目は最後の `np.clip` — すべての補正が終わったあと、`KELLY_MIN` と `KELLY_MAX` の間で切り取られる。その間で起きるのが、6つの係数の掛け算である。

それぞれの係数の範囲と役割をまとめる。

係数	範囲	役割
edge	(0, 0.5]	p_upに基づくハーフ・ケリーの基本値
vol_mult	[0.5, 1.0]	市場が不安定なほど縮小
macro_gate	[0.55, 1.0]	市場状態が不確実なほど縮小
novelty_gate	[0.55, 1.0]	見たことのない市場なら縮小
er_mult	[0.5, 1.5]	シグナル方向と一致するときに拡大
macro_mult	[0.3, 2.0]	資産クラスごとのマクロ反応

係数の組み合わせ効果に注意: 6つの係数が同時に低い値になると、ポジションは想定よりはるかに小さくなる。KELLY_MIN=0.10 は下限としてはたつき、ポジションが完全にゼロ近くまで落ちる事態を防ぐ。

9.3 vol_mult —ボラティリティが高まったらポジションを減らす

市場が不安定になったら、ポジションサイズを自動的に減らす必要がある。これを担うのが `hmm_vol_mult()` だ。中心となる考え方は、**過去価格の標準偏差ではなく、HMMがすでに推定したレジーム別の分散を使う**という点にある。

両者の反応速度がなぜ違うのか。それがこの節の要点だ。

- ・ **ローリングウィンドウのボラティリティ:** 過去20～60本の足の価格変動を平均する。市場が突然急落しても、それまでのデータが希釈してしまうため、反応が20～60本後に遅れて出る。
- ・ **HMMのレジーム分散:** HMMが各レジームごとにあらかじめ推定したボラティリティを使う。レジーム確率 `alpha_t` が高ボラレジームへ移った瞬間に即時に反応する。

`alpha_t` が高ボラレジーム側に動くと、次の順に計算が進む。

1. レジーム加重ボラティリティ (`sigma_hmm`) が平均ボラを上回る。
2. ボラティリティ比が1.0を超える。
3. `vol_mult` が1.0未満に下がる。
4. ポジションサイズが自動的に縮小する。

ratio 1・2・3で倍率 $1.0 \cdot 0.75 \cdot 0.50$ にする例は `clip(1-0.25*(ratio-1), 0.25, 1)` だ。原理を示す補間であり最適運用式ではない。分子分母を同単位にし、基準volatilityが0や履歴不足ならpositionを増やさない。

9.4 macro_gate —市場状態が紛らわしければ縮める

HMMは現在の市場が各レジームに属する確率を `alpha_t` 配列として出力する。たとえばレジームが4つなら `alpha_t = [0.9, 0.05, 0.03, 0.02]` のような形だ。数値が一方に偏っていれば「いまのレジームが何かははっきりしている」という意味になり、均等に広がっていれば「いまどのレジームか分からない」という意味になる。

`macro_gate` はこの確信度を0.55~1.0の数値に変換し、シグナルをどれだけ信用するかを決める。レジームがはっきりしていればシグナルをそのまま使い、レジームが入り混じっていればシグナルを縮める。確信度の測定には情報理論のシャノンエントロピー (Shannon entropy) を使う。

レジームが4つの場合を例にとると次のとおりだ。

- ・ 確率が1つのレジームに100%偏る -> ゲート = 1.0(そのまま通過)
- ・ 4つのレジームに25%ずつ分散 -> ゲート = 0.55(最小値まで縮小)
- ・ 遷移中 (50/30/10/10) -> ゲート $\approx$ 0.59(わずかにしか開かない)

`MACRO_GATE_MIN = 0.55` としてあるので、完全に不確実な状況でもシグナルの55%は維持される。0(完全遮断)にしないのは、レジームが切り替わる瞬間にポジションが一斉に手仕舞われるとスリッページとコストが急増するからだ。

9.5 novelty_gate —見たことのない市場なら縮める

HMMは学習済みのパターンを基準に現在の市場を分類する。ところが、関税ショック直後のように一度も見たことのないパターンが入ってくると、どのレジームにもうまく当てはまらず、全体の適合度スコアが低くなる。`hmm_novelty_gate()` はこの状況を検知して自動的にポジションを縮める。

中心となる考え方は「いまの観測値はふだんと比べてどれだけ見慣れないか」を見ることだ。絶対値ではなく、**直近60セッションの分布のなかでの百分位順位**として見る。

ふだんよりはるかに見慣れない値ならゲートが作動し、ポジションを最低0.55まで縮める。

極端な市場での検証—2020年3月COVID: 2020年3月16~23日(8営業日)の間、*novelty_gate* の平均値は0.61で、ポジションは約40%縮小した。S&P500が-24%を記録するなか、戦略のドロダウンは-11%にとどまった。*NOV-ELTY_GATE_MIN=0.55* のおかげで完全に市場から離脱しなかったため、3月24日以降の反発局面も捉えることができた。

このゲートはカルマンフィルタとは役割が違う。

- ・ **カルマンフィルタ:** 単一銘柄のbar単位の価格ノイズを除去する。
- ・ **HMM novelty gate:** 第7章で構成した13次元のクロスアセット観測値が、学習済みレジームとどれだけ一致するかを測る。

最近の資料で未知patternの境界が改善する可能性はあるが、60sessionで回復する保証はない。例では学習距離の99percentile超を保留し、再学習後も別検証を要求する。OOS損失に反応して即座に基準を緩めれば評価区間を学習に使うことになる。

9.6 ERアンプリファイア—シグナルがそろったときに増やす

ここまでの3つの係数(*vol_mult*、*macro_gate*、*novelty_gate*)はすべて**ポジションを縮める方向**にはたらいた。ERアンプリファイアは逆に、**シグナルが強く方向が一致したときにポジションを増やす**係数だ。

ER(Expected Return、期待収益率)はHMMが推定した次セッションの期待収益率である。各レジームごとの平均収益を、現在のレジーム確率で加重平均して求める。

ルールは単純だ。

- ・ 期待収益(er)がポジション方向(direction)と同じ符号 -> 最大1.5倍まで増幅
- ・ 期待収益がポジション方向と逆の符号 -> 0.5倍に半減

3つ目のケースが要だ。p_up はロングを示しているのにERが負の値なら、2つのシグナルが衝突したことになる。このときはポジションを半分に減らす。モデル内でシグナル同士が互いをチェックする安全装置の役割を果たす。

9.7 資産クラスごとのマクロバイアス調整

2-Layer Macro HMMが算出する macro_bias は-1から+1の値を取る。-1なら完全なRisk-Off(リスク回避)、+1なら完全なRisk-On(リスク選好)を意味する。この値はすべての資産に同じように適用されるわけではない。資産クラスごとに反応の仕方が違うため、macro_class_amp という係数を掛けて各資産に合わせて調整する。

クラス	macro_class_amp	Risk-Off局面での効果
equity(株式)	+0.5	株式ロングを強く縮小
safe_haven(GLD, TLT)	-0.3	安全資産ロングを拡大(符号反転)
commodity	+0.3	コモディティ・エクスポージャーを緩やかに縮小
credit(HY)	+0.3	ハイイールド・エクスポージャーを縮小
fx	+0.2	FX感応度は弱め
vix(VIXY)	-0.6	VIXロングを強く拡大(符号反転)
em(新興国)	+0.4	EM株式エクスポージャーを縮小

正の符号を持つ資産はマクロバイアスと同じ方向に動き、負の符号を持つ資産(safe_haven、vix)は逆方向に動く。

例で確認してみよう。

例1 —Risk-Off局面での株式ロング -> ポジション約25%縮小 **例2 —Risk-Off局面でのVIXロング** -> ポジション約30%拡大

仕組みは単純だ。Risk-Off局面では株式ロングが自動的に縮小し、VIXYロングは自動的に拡大する。別途ヘッジロジックを書かなくても、マクロ局面が変わればポートフォリオは自然に防御態勢を取る。正確な式とクリッピング範囲は calculate_kelly_size() の本体で確認できる。

9.8 fractional_kelly() —5分足向けの簡略版

5分足のEnsembleシステムでは6つのゲートが不要になる。単一銘柄かつ短期シグナルなので、よりシンプルなクォーター・ケリーで十分だ。fractional_kelly() は離散ベットの p 、 b を推定する代わりに、連続シグナルの平均(μ)と分散(σ^2)をそのまま使う。

```
# hmm_v5_core.py (excerpt)
def fractional_kelly(mu, sigma2, kelly_frac=0.25, max_size=1.0)
-> float:
    """f* = kelly_frac × |μ| / σ²
    Reduce size for higher uncertainty; increase it for stronger
    signals.
    """
    if sigma2 <= 1e-12 or np.isnan(mu) or np.isnan(sigma2):
        return float(max_size)
    k = kelly_frac * abs(mu) / (sigma2 + 1e-12)
    return float(np.clip(k, 0.0, max_size))
```

核心は分子に $|\mu|$ (シグナルの大きさ)、分母に σ^2 (不確実性)が入っていることだ。シグナルが同じでもボラティリティが2倍になればポジションは1/4に縮む。

バックテストではシグナルの符号で方向、fractional_kelly() の出力で大きさを決める。

9.9 2つのケリー実装の比較

項目	calculate_kelly_size() (v7.py)	fractional_kelly() (v5_core.py)
システム	日足/セッションのクロスアセット	5分足の単一銘柄
基本式	$\text{edge} = p_{\text{up}} - 0.5$	$f^* = \text{kelly_frac} \times \mu / \sigma^2$

項目	<code>calculate_kelly_size()</code> (v7.py)	<code>fractional_kelly()</code> (v5_core.py)
ケリー 比率	Half(0.5×)	Quarter(0.25×)
ボラ補 正	HMMのレジーム分散	78本のローリング標準偏差
追加ゲ ート	macro_gate, novelty_gate, ERアンプ	なし(シンプル)
クリッ ピング	[0.10, 1.0]	[0.0, 1.0]

`calculate_kelly_size()` は複数資産間の流れ、市場全体のムード、資産クラスごとの特性をすべて反映する。考慮する変数が多いぶん、構造も複雑だ。`fractional_kelly()` は単一銘柄の短期取引向けに作られていて、構造はずっと単純だ。複雑な条件を同時にさばく必要がない場面では、シンプルのほうがかえって安定して動く。

9.10 KELLY_MIN = 0.10 に設定した理由

信頼度が低くても最低限のポジションは維持する。ポジションを完全に空にしてしまうと、良い機会が来たときに再エントリーのタイミングをつかみにくいからだ。

```
KELLY_MIN = 0.10
KELLY_MAX = 1.0
```

`KELLY_MIN = 0.10` は「シグナルがあれば最低でも10%はベットする」という意味だ。ケリーのedgeがほぼ0でも、関数は0ではなく0.1を返す。

なぜ10%なのか。このシステムにはポジションを縮める補正係数が複数ある。これらの係数がすべて同時に最小値に達してもポジションが0にならないように、10%を下限と定めた。0まで下げてしまうと、市場環境が悪い区間でポジションが完全に消え、レジーム転換直後の反発局面でエントリー機会を逃してしまう。

ただし、`edge <= 0(p_up <= 0.5)` のときは `KELLY_MIN` より先に0が返る。「エッジ自体がない場合」と「エッジはあるが補正で縮んだ場合」は別物として扱う。

シナリオで見る

頭のなかで関数を一度走らせてみよう。以下の4つのシナリオを入れると次のような傾向が出る。

シナリオ	p_up	vol_mult	macro_gate	novelty_gate	結果
強いブルレ ジーム	0.72	0.9	0.95	1.0	大きい ポジシ ョン
中立レジ ーム	0.56	0.7	0.70	0.9	KELLY_MIN 付近
高ボラレジ ーム	0.63	0.5	0.60	0.8	小さい ポジシ ョン
異常値の検 知	0.68	0.8	0.85	0.2	KELLY_MIN まで抑 制

最後の行が `novelty_gate` の役割をよく示している。`p_up` が0.68とそれなりに強いシグナルでも、観測がどのレジームから見ても見慣れず `novelty_gate = 0.2` になれば、ポジションは最小値まで下げられる。

演習

1. $b = 1, p = 0.55$ のときの、フル・ケリー、ハーフ・ケリー、`fractional_kelly(factor=0.25)` のポジションサイズをそれぞれ計算せよ。3つのうちどれが最も変動が小さくなるか考えてみよう。
2. `macro_gate` のしきい値を下げると、どんな取引が遮断されやすくなり、どんな取引が通りやすくなるか予想せよ。

3. $KELLY_MIN = 0.10$ を0に変えると、どんな市場状況で戦略がポジションを完全に閉じてしまうか、シナリオを描いてみよ。

まとめ

この章で必ず覚えておきたいのは次の4点だ。

- ・ **ポジションサイジングはシグナルと同じくらい重要だ。** 同じ戦略でも、ベットの大きさが結果は完全に変わる。
- ・ **フル・ケリーは危険だ。** 推定誤差がある現実では、ハーフ・ケリーのほうが長く生き残る。
- ・ **ハーフ・ケリーではベット比率が結局 edge そのものになる。** すべての補正係数はこの edge に掛けられる。
- ・ **6つの補正係数は「縮める係数」が大半だ。** 市場が不安なほど、レジームが紛らわしいほど、見慣れない観測が来るほど、ポジションは自動的に小さくなる。 $KELLY_MIN=0.10$ は、それらすべてのゲートが閉じても完全に0にならないようにする下限である。

ハーフ・ケリーのポジションパイプラインを一行でまとめると次のようになる。

```
p_up -> edge = p_up - 0.5 -> 補正係数を掛ける -> [0.10, 1.0] の範囲にクリップ
```

次章(第10章): HMM Forward Ensembleを扱う。`run_hmm_forward_ensemble_v2.py` と `v3.py` の構造を分析し、Walk-forward方式で複数のサブモデルを比較するパイプラインを説明する。

さらに読むには

- ・ **論文:** Kelly, J. L. (1956), “A New Interpretation of Information Rate”—元祖となるケリー公式の論文
- ・ **書籍:** Thorp, Beat the Dealer, The Mathematics of Gambling —実戦でのケリー応用事例

- ・ 書籍: Lopez de Prado, Machine Learning for Asset Managers Chapter 6 —ポジションサイジングとシグナル強度の連携

理解を確認する

「勝率0.5/0.6/0.7、損益比1のKellyを比較し、half-Kellyと10%上限も示して。」

第10章. HMM フォワードアンサンブル—V1 → V3 への進化

複雑なシステムの初版はたいてい物足りない。クオンツシステムも例外ではない。重要なのは、どこでなぜ失敗したのかを把握し、次のバージョンで何を直すかを定めることだ。

V1 のアンサンブルは基本構造こそ動いたが、実運用を任せるには弱かった。V2 では Wasserstein 距離による投票を加え、V3 では複数の時間軸の信号を足した。大事なのは数値そのものではなく、バージョンを重ねるたびに、どんな失敗を減らそうとしたかである。

この章は、その反復過程をバージョンごとに残した記録だ。結果の数字だけを見るのではなく、どの失敗仮説が次の構造変更につながったのかを一緒に見てほしい。AI は各サイクルを素早く回すのに役立った。「予測結果はどこで一番外しているか」と AI に尋

ねると、レジーム(市場の状態)転換の局面でサブモデルどうしの意見が最も割れる、という診断が返ってきた。そのフィードバックが V3 改善の出発点になった。

なぜアンサンブルなのか

単一の HMM は市場の一側面しか見ない。トレンドを見るモデルはレンジ相場で弱く、ボラティリティを見るモデルはトレンド相場で信号がぼやける。どんな種類の相場が来るかを事前に知ることはできないので、別々の情報を見る複数のモデルを用意し、その意見をまとめるほうが安全だ。

たとえるなら、一人の専門家に全判断を任せる代わりに、分野の違う何人かの専門家に意見を聞いて平均を取るのと似ている。普段は意見が似ているが、決定的な瞬間(レジーム転換)に一人か二人が違う信号を出す。そのときに多数決や加重平均が役に立つ。

この本の中で、この章の位置づけは明確だ。**信号生成(10章) -> サイズ決定(9章) -> 銘柄ペア発掘(11章)**のちょうど真ん中にある。つまり 10 章は「明日の価格がどちらに動くか」という一つの数字を作る段階だ。

9章の復習: Kelly Sizing は Ensemble が計算した期待収益率の信号($E[r]$)を受け取って、ポジションサイズを決める。この章では、その信号をどのように作るのかを見ていく。

本書全体で見ると、アンサンブルは複数モデルの意見を一つの数字にまとめる会議室である。HMMモデル一つだけで、すべての相場をうまく読めるわけではない。トレンドに敏感なモデルもあれば、ボラティリティ変化に敏感なモデルもあり、セクターの流れをよく見るモデルもある。アンサンブルはそれらの意見を集め、最終的な期待収益率 $E[r]$ を作る。

モデルA: 明日は +0.15% かもしれない

モデルB: 明日は -0.05% かもしれない

モデルC: 明日は +0.20% かもしれない

|

∨

アンサンブル

最近よく当たったモデルに少し重みを乗せて合成する

|

$\downarrow$
 最終シグナル $E[r]$
 例: $+0.10\%$
 $\downarrow$
 $\downarrow$
 Kelly / Risk
 ポジションサイズと注文可否を決める

初心者は、アンサンブルを「モデルを増やせば必ず良くなる方法」と読まないほうがよい。モデルが増えれば意見も増え、ノイズも増える。重要なのはモデル数ではなく、**違う失敗をするモデルをどう組み合わせるか**である。

良いアンサンブル:
 違う情報を読むモデルたちが
 同じ方向を示すとき、信頼度を上げる。

悪いアンサンブル:
 似たモデルを何個もコピーし、
 同じ錯覚を何度も数える。

この章の流れ

この章は大きな絵から始まり、一段ずつ内側に入っていく。

- ・ まず V1/V2/V3 の違いを一目で見る (10.1)
- ・ 3 つのバージョンが共有する中核計算—Forward α で $E[r]$ を作る (10.2)
- ・ バージョンごとにどんな種類の情報を足したのか (10.3)
- ・ Optuna による閾値の再最適化が速い理由 (10.4)
- ・ 成果の比較と適応的な閾値 (10.5, 10.6)

学習目標

この章を終えると、次のことができる。

- ・ V1/V2/V3 の中心的な違いを説明し、各バージョンが解決した問題を指摘できる
- ・ Forward α を使って翌日の期待収益率 $E[r_{t+1}]$ をどう作るのかを図に描ける
- ・ Optuna による閾値の再最適化がなぜ速いのか、一文で説明できる
- ・ 動的な重みとマルチタイムフレームがアンサンブルでどんな役割を果たすのかを理解する

初回読破で押さえること

この章にはサブモデル、Forward α 、遷移行列、Optuna、動的な重みのように難しい言葉が多く出てくる。最初に読むときは、次の流れだけを掴めばよい。

1. 複数のモデルがそれぞれ「明日の収益率はこちら寄りだろう」という意見を出す。
2. 成果が良かったモデルの意見を少し大きく反映する。
3. 最終的に $E[r]$ という一つの数字を作る。
4. 続く章は、この数字をポジションサイズや実際の売買条件へと変えていく。

数式の表記や Softmax の細かい計算は二度目の読破で見ても構わない。初回の目標は、「**アンサンブルは複数モデルの意見を一つの数字にまとめる装置**」だと理解することだ。

10.1 アーキテクチャ比較—V1, V2, V3

項目	V1	V2	V3
銘柄	SPY 1 個	5 個 (UNIVERSE)	10 個
サブモデル数	5 個 (M1~M5)	7 個 (M1~M7)	15 個 (M1~M9, マルチTF)
重み	固定 (equal)	固定 (equal)	動的 (ローリング成果ベース softmax)
非-HMM モデル	なし	なし	Kalman(M8) + MeanRev(M9)

項目	V1	V2	V3
Optuna	なし	あり (re-thresholding)	あり
並列処理	なし	なし	ThreadPoolExecutor(4 スレッド)

V1 -> V2 -> V3 は 3 つの軸で進化した。サブモデルの種類を増やし、重みを状況に応じて動かせるようにし、銘柄数を増やした。

V2 の中心的な変化は Optuna の導入だ。Optuna はモデルの最適な閾値を自動で探してくれるライブラリである。V3 では、信号を複数の時間軸に分けて見るマルチタイムフレーム方式を追加した。

この順序には理由がある。先に銘柄を増やすとノイズ(意味のない信号)も一緒に増える。モデル構造と閾値が安定して動くことを確認してから銘柄数を増やすほうが安全だ。

10.2 V1/V2 共通コア—Forward α で $E[r]$ を計算する

V1 と V2 は中核計算を共有している。バージョンが変わってもこの部分は変わらない。流れを一文で要約するとこうだ。ここまで見たデータで現在のレジーム確率を更新し、その確率を一コマ先に押し出して翌日の収益率期待値を作る。

レジーム(regime)とは、市場が今どんな状態なのかを表す。「上昇トレンド」「レンジ」「急落」のような区分がレジームだ。そのつど更新される確率ベクトルを **Forward α** と呼ぶ。たとえばレジームが 3 つあるとき、今日の Forward α が $[0.7, 0.2, 0.1]$ であれば、「今日は 70% の確率で上昇トレンド、20% でレンジ、10% で急落状態に見える」という意味になる。

ここでもう 2 つ材料が必要になる。

- ・ **遷移行列(transition matrix) A**: レジームが次のステップでどこに移るかの確率を集めた表だ。たとえば $A[\text{Bull}, \text{Bear}] = 0.05$ なら「今日 Bull の状態のとき、明日 Bear に変わる確率は 5%」という意味になる。この表は HMM の学習段階で、過

去データから Baum-Welch アルゴリズムによって自動推定される(詳しくは 8 章を参照)。

- ・ **レジーム別の平均収益率** μ_{ret} : 学習時に各レジームで観測された収益率の平均だ。学習が終わったあとは `m.means_` に固定保存され、バックテスト中は絶対に更新しない(その理由は 10.2.1)。

小さな数字で見るほうがわかりやすい。今日のレジーム確率が [Bull 70%, Neutral 20%, Bear 10%] だとする。遷移行列を当てはめると明日の確率は [Bull 65%, Neutral 25%, Bear 10%] のように少し動く。各レジームの平均収益率が [+0.2%, 0.0%, -0.3%] なら、明日の期待収益率はおよそ $0.65 \times 0.2\% + 0.25 \times 0.0\% - 0.10 \times 0.3\% = +0.10\%$ になる。この +0.10% が $E[r]$ だ。

中核の計算は 2 行に縮まる。

- ・ **Step 4 (予測)**: `next_proba =  $\alpha_t \times A$` —今日のレジーム確率を一コマ先に押し出す。
- ・ **Step 5 (E[return])**: $E[r_{t+1}] = \text{next_proba} \cdot \mu_{\text{ret}}$ —その確率にレジーム別の平均収益率を掛けて合計する。

この 2 行がすべてのバージョンの心臓部だ。ほかの工程(初期化、観測確率の更新、アンサンブル平均、閾値で $\pm 1/0$ 信号を作る)は、この 2 行を支える脇役にすぎない。全体の疑似コード(Step 1~7)は付録 A にまとめた。

出力例:

```
2024-01-03 -> e_ret = 0.0034 (Bull レジーム 0.87)
2024-01-04 -> e_ret = -0.0012 (転換局面)
2024-01-05 -> e_ret = 0.0021 (Bull 復帰)
```

毎日新しい足(bar、1 日分の相場データ)が入るたびに、上の 2 行を回して翌日の `e_ret` という一つの数字を得る、と覚えればよい。

10.2.1 因果性(causality)の落とし穴を避ける

レジーム別の期待収益を計算するときによく起こる落とし穴がある。**未来のデータが過去の計算に流れ込む問題**だ。これを因果性違反と呼ぶ。実戦では未来を見ることができないので、このエラーがあるとバックテストの結果が実際よりもはるかに良く出てしまう。

覚えておくべきルールは一行だ。

- ・ **使用:** 学習が終わったあと固定された `m.means_`(レジーム別の平均が学習時点で凍結される)
- ・ **禁止:** `m.predict()` —Viterbi 平滑化で未来まで見て状態を決めてしまう

以下は、実際のコードでこのルールが適用される肝心の一行だ。

```
srm = sc.inverse_transform(m.means_)[:, idx]    # causal
# m.predict(Xs)    # look-ahead bias
```

失敗したら見る場所: バックテスト成果が非現実的に高く出る場合(たとえば 3 以上)、この部分に未来データが混入している可能性が高い。

10.3 バージョンごとにどんな情報を足したのか

`return model` は方向、`volatility` は risk 量、平均回帰は過去均衡からの偏差を扱う。同じ保有期間の期待 `return` 等へ単位を揃えて合成する。標準 `score` と `return` をそのまま平均すると数値の大きさが重みを決める。

V2 で追加—セクター・オーダーフロー

- ・ **M6_Sector:** セクターローテーションを見る。景気の流れに沿って資金が集まる業種が入れ替わる現象を捉える。市場全体に対する銘柄の超過収益、同セクター内での銘柄の超過収益、セクター ETF 全体の強弱などをまとめて 3 つのレジーム(セクター強気/弱気/中立)を学習する。
- ・ **M7_OrderFlow:** オーダーフロー(買い手と売り手のどちらが取引を主導したか)を見る。板情報がなくても日足の OHLCV(始値・高値・安値・終値・出来高)だけで推定できる。中心となる発想は「ザラ場の安値に対して終値の位置が高いほど買い圧力が強かった」だ。

V3 で追加—マルチタイムフレーム & 非-HMM

V3 は 2 つを足した。

マルチタイムフレーム。既存のサブモデルの一部(M1・M2・M3・M5)を、短期(1d)、中期(5d)、長期(21d)という 3 つの時間軸に複製した。短期のレジームは 1 日単位で素早く変わる信号を、長期のレジームは 1 か月以上続くトレンドを捉える。2 つのタイムフレームが同じ方向を指していれば、信号はより信頼できると判断する。

これは同じモデルを単に多く複製するのとは違う。1d と 21d はそれぞれ別の種類の情報(素早い揺れ vs. ゆっくり進むトレンド)を見ている。情報が重ならないほど、動的な重み(後述)が「今どの時間軸が当てているか」に合わせて比重を移しやすくなり、相乗効果が大きい。

非-HMM の 2 モデル。HMM ではない別の統計道具を持ち込んだ。

- ・ **M8_Kalman:** 価格に混ざったノイズを除いた滑らかな「トレンドの推定値」を作り、その値をそのまま翌日の期待収益率として使う。(道具の名前が Kalman フィルタである。)
- ・ **M9_MeanRev:** 価格が一方向に進んでもやがて平均へ戻ろうとする性質を見る。平均から離れすぎていて、かつその性質がはっきり観測されるときだけ、逆方向の信号を出す。

マルチタイムフレームは次のように読めばよい。

状況	解釈
1d, 5d, 21d がすべて上昇	短期と長期が同じ方向なので信号の信頼度が高い
1d だけ上昇、21d は下落	短期反発の可能性があるので重みを下げる
1d は下落、21d は上昇	長期トレンドのなかの一時的な調整かもしれない

カルマンフィルタの式の導出、分散比や z-score のような統計道具の細かい定義、そしてノイズ分散などの細部のハイパーパラメータは付録にまとめた。

V3 の動的な重み—成果の良いサブモデルにより大きな比重

V2 までは、すべてのサブモデルの重みが同じだった。V3 はサブモデルごとの成果を日々追跡し、**最近の成果が良いサブモデルにより大きな重みを与える。**

流れは入力 -> 計算 -> 出力という一筋で見るとよりシンプルだ。

入力	計算	出力
各サブモデルの最近の日別 (予測方向 × 実際の収益率)	ローリング成果を求めて Softmax で正規化	合計が 1 になる重みベクトル w_k

重みの例は $w_i = \exp((s_i - \max(s))/T) / \sum_j \exp((s_j - \max(s))/T)$ で $T > 0$ 。大きい T は均等、小さい T は集中する。結果が確定した過去予測だけで score を作る。履歴不足は除外か事前の均等重みにし、3 つ同 score なら各 1/3 になる。

次のステップのアンサンブル信号は $E[r]_{\text{ens}} = \sum_k w_k \cdot E[r]_k$ で合算する。

たとえば昨日の重みが $[0.30, 0.25, 0.20, 0.15, 0.10]$ で、最初のモデルが当て続けていたとすれば、今日は $[0.40, 0.20, 0.20, 0.10, 0.10]$ のように変わりうる。正確な計算式よりも大事なのは、**成果の良いモデルの声が大きくなる**ということだ。

10.4 V2 の中心アイデア—Optuna による閾値の再最適化

V2 で最も重要な変化は一つだ。**Walk-Forward を最初から走らせ直さず、閾値だけを再調整する。**

ここで言う閾値とは、「予想収益がこの水準を超えたら買い、この水準を下回ったら売り」のように信号をふるい分けるための基準値だ。

なぜ速いのか。HMM を毎回学習・推論し直すのは時間がかかる。一方、すでに計算しておいた $E[\text{return}]$ の時系列があるなら、閾値だけ変えて実験するのははるかに速く反復できる。モデルはそのまま置いておき、基準線だけを動かすという発想だ。

おおよその計算量の差はこうなる。

- ・ **Walk-Forward の再実行:** データ全体でモデルを繰り返し学習 -> 非常に遅い
- ・ **Re-thresholding:** 保存しておいた $E[r]$ に閾値だけを当てる -> はるかに速い

ここで注意点が一つある。閾値を探すときに OOS(Out-of-Sample、検証用データ)全体を使ってしまうと、最適化そのものが未来データを覗いたことになる。そのため OOS

の前半 40% だけを Optuna が閾値探索に使い、後半 60% は触らず本物の検証区間として残す。

この使い回しを成立させるには、walk-forward の段階で毎日の `e_ret` をあらかじめ保存しておく必要がある。

10.5 V2/V3 成果比較のまとめ

以下は 3 つのバージョンの役割の違いだ。OOS(Out-of-Sample) はモデル訓練に使わなかったデータで測った区間を指す。

バージョン	サブモデル	ユニバース	特記事項
V1	5 個の HMM	SPY 1 個	基本構造の検証
V2	7 個の HMM	5 銘柄	Optuna re-threshold 効果の確認
V3	15 個 (HMM + 異種)	10 銘柄	動的な成果ベース重みの効果確認

実戦投資において、複雑な V3 が常に正解とは限らない。モデルの判断過程を追跡したい、あるいは安定した動作を望むなら、シンプルな V2 のほうが向いている。

10.6 適応的な閾値— $\sigma \times \text{scale}$

閾値は固定せず、最近の $E[r]$ の分布に合わせて動的に調整する。中心のアイデアは一行だ。**閾値を最近の $E[r]$ の標準偏差に比例させる。**

そうすると次のように動く。

- ・ **市場が静かなとき**: 標準偏差が小さいので閾値も低い。小さな信号でもエントリーする。
- ・ **市場が荒れているとき**: 標準偏差が大きいのので閾値も高い。強い信号のときだけエントリーする。

比例定数(thresh_scale)は人が決めず、Optuna が自動で探す。この自己補正(self-calibrating)構造のおかげで、市場レジームが変わっても取引頻度が大きくぶれない。実際の閾値計算コードと信号生成の一行は、後述の「やってみる」で直接扱う。

やってみる—Optuna で閾値最適化(オプション)

この節は初回読破では飛ばしてかまわない。一度通読して流れを掴んだあと、二度目の読破で手を動かしてなぞるのがおすすめだ。

合成閾値探索は計算済みe_retから始める。予測 $0.01 \cdot 0 \cdot -0.01$ 、重み $0.5 \cdot 0.3 \cdot 0.2$ なら合成0.003。基準0.002で候補になるが費用とrisk検査が必要だ。submodelを学習する全pipelineではない。

以下のコードは thresh_scale を 0.2-2.0 の範囲で探索しながら、OOS 成果を最大化する値を探す。中心は 2 行だ。閾値を e_ret の標準偏差 × scale で決め、その閾値で $\pm 1/0$ の信号を作る。

```
import numpy as np
import optuna

def optimize_threshold(e_ret, actual_ret, n_trials=50):
    def objective(trial):
        scale = trial.suggest_float("scale", 0.2, 2.0)
        threshold = float(e_ret.std()) * scale
        signals = np.where(e_ret > threshold, 1,
                           np.where(e_ret < -threshold, -1, 0))
        pnl = signals * actual_ret
        if (signals != 0).sum() < 5:
            return -999.0
        return evaluate_objective(pnl)

    study = optuna.create_study(direction="maximize")
    study.optimize(objective, n_trials=n_trials)
    return study.best_params["scale"], study.best_value
```

確認する出力: thresh_scale はたいてい中間レンジで選ばれ、合成データでは成果が 0 付近で揺れる。実データでは数値そのものよりも、IS と OOS の差が過剰に広がっていないかを見る。

失敗したら見る場所: -999.0 が出続けるなら、閾値が大きすぎて信号がほとんど発生していない状態だ。e_ret の分布(np.std、ヒストグラム)からまず確認する。

練習問題

1. 2 つのサブモデル M1, M2 が逆方向の信号を出すとしよう。たとえばある日に M1 の $E[r] = +0.003$ 、M2 の $E[r] = -0.002$ で、両モデルの重みが等しい($w = 0.5$) とする。アンサンブル合計 $E[r]_{ens}$ はいくらで、閾値が ± 0.001 のとき最終信号 (+1/-1/0) はどうなるか。
2. 新しいサブモデル M9_Earnings(決算発表前後 3 日のイベントパターン)を追加するなら、どのような特徴量が必要で、どのデータソースを使うべきかを書き出してみよう。
3. (思考実験)8 つのサブモデルのうち 2 つを切らなければならないとしたら、どの種類のサブモデルを切ると成果に大きく響くか推測してみよう。ヒント: 他のサブモデルと信号の種類が重ならないモデルを切ると損失が大きく、似た信号を見るモデルは一つ切っても影響は小さい。

まとめ

この章では **複数の信号を一つにまとめるアンサンブル方式** を見てきた。単一の HMM では捉えにくいセクターローテーションの信号やオーダーフローの信号を、別のサブモデルとして用意し、一つの予測信号に束ねたのが中心だ。

バージョン	中心の貢献	残った課題
V1	Forward-only α ベースの $E[r]$ パイプライン	SPY 単一、固定の重み

バージョン	中心の貢献	残った課題
V2	M6_Sector, M7_OrderFlow + Optuna re-thresholding	動的な重みなし
V3	マルチ TF + M8_Kalman + M9_MeanRev + 成果ベースの softmax	—

この章で必ず覚えること

- 1. **因果性を守る**: レジーム別の平均収益率は、学習が終わった `m.means_` だけを使う。
`m.predict()` は未来を見るためバックテストを台無しにする。
- 2. **1 ステップ先の予測**: (現在のレジーム確率 × 遷移行列) · レジーム別の平均収益率—
この一行がすべてのバージョンの中心だ。
- 3. **閾値は固定しない**: 最近の $E[r]$ の標準偏差に比例させ、Optuna でその比率(scale)
を自動調整する。そうすれば市場状況が変わっても取引頻度が安定する。

次章(11章): 統計的裁定取引 (Stat Arb) のペアを選ぶ方法を扱う。Engle-Granger の
共和分検定と一緒に動く銘柄ペアを探し、Kalman フィルタで 2 銘柄間のスプレッド
とヘッジ比率を動的に計算する。

さらに読むなら

- ・ **論文**: Sharpe (1994), “The Sharpe Ratio”—シャープレシオの意味と限界を整理
した文章
- ・ **論文**: “Stacked Generalization” (Wolpert, 1992) —Ensemble 学習の理論的基盤
- ・ **本**: Grinold & Kahn, Active Portfolio Management —Ensemble アルファをポー
トフォリオへ落とし込む方法

第11章 統計的裁定取引 —ペア選定

Part V —ペアトレーディング基本形(第11～12章) このパートでは「ある銘柄が上がるか」ではなく「2つの銘柄の関係が再び縮まるか」を見る。第11章で取引する価値のあるペアを選び、第12章でエントリーと決済の信号を作る。最初の通読では共和分検定の細部より「相関だけでは足りない」ことを覚えておけば十分だ。

チャートを見て売買するトレーダーは、たいてい次のように問う。「この銘柄は上がるか、下がるか?」上がりそうなら買い、下がりそうなら売る。個人投資家のほとんどが使う方法である。

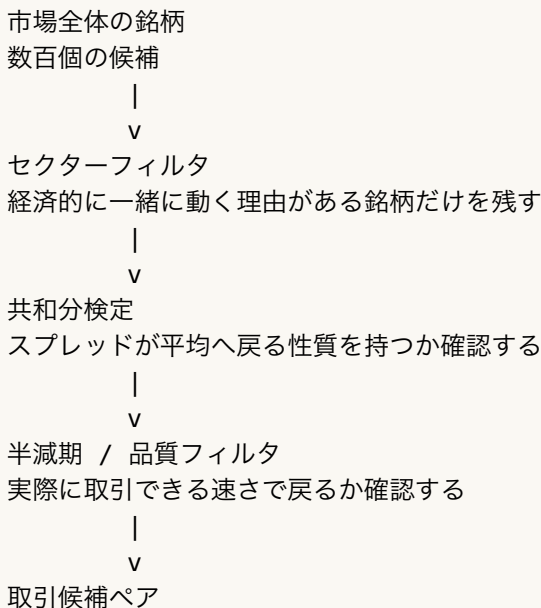
統計的裁定取引 (statistical arbitrage) は別の問いを立てる。「この2つの資産の間に安定した関係があるか?」そして「今その関係が一時的にずれていて、再び戻る可能性があるか?」方向よりも関係に集中するのだ。たとえば、コカ・コーラ (KO) が上がるかペプシ (PEP) が下がるかを当てるのではなく、2銘柄の価格差が普段より大きく開いていれば再び縮まるはずだ、というふうに賭ける。

この手法は**市場中立戦略**と呼ばれる。市場全体が上がろうと下がろうと関係なく、2つの資産の関係が収束することだけに賭けるからだ。損失が出るのは主に、2つの資産の関係そのものが壊れるときである。だからこそ私たちが探すエッジ(優位性)も、方向予測より測定しやすい。

個人投資家がペアトレーディングを試すときによく犯す失敗がある。「コカ・コーラとペプシは一緒に動くから良いペアだろう」という発想だ。直感的に聞こえるが、検証ではない。2銘柄が一緒に上がっているように見えても、市場全体が上がっているだけかもしれない。**相関(correlation)**は2つの資産が似た方向に動く度合いを表し、**共和分(cointegration)**は2つの資産の間に長期的に安定した関係が存在することを意味する。両者は別の概念であり、私たちに必要なのは後者だ。

定量的にペアを選ぶには3つが必要だ。多数のペアに共和分検定を回し、偶然通過したものを除外し、最後に「なぜこの2社が一緒に動くべきなのか」という経済的な根拠でもう一度ふるいにかける。単にp値が低いというだけで良いペアではない。1,000ペアを検定すれば、何の関係もなくとも偶然に約50ペアが通過する(多重比較問題)。

本書全体の機械学習フローで見ると、第11章は**取引候補を作る章**である。HMMが市場状態を読み、RLがポジションサイズを決めるとしても、まず「何を取引するか」が必要だ。ペア選定は、その候補リストを作る。



第12章でエントリー・手仕舞いシグナルを作る

初心者は、ペア選定を「良さそうな二銘柄を探すこと」と考えないほうがよい。より正確には、**関係が壊れたときに危険になる候補を先に捨てる作業**である。良いペアとは、単に一緒に動く二銘柄ではなく、広がった差が再び縮まる統計的・経済的理由を持つ二銘柄だ。

関連:

二つの資産は同じ方向に動くか?

共和分:

二つの差が大きく広がったとき、戻る傾向があるか?

ペアトレーディングに必要なのは共和分である。

この章の流れ

ペアトレーディングの直感を先につかんでから、実際にペアを選ぶ基準へ降りていく。

- ・なぜセクターの中でペアを選ぶのか
- ・ペア選定パイプライン
- ・半減期(halflife)の計算
- ・Kalman フィルタで動的ヘッジ比率を求める
- ・ペアの品質指標

学習目標

この章を読み終えたら、次のことができるようになる。

- ・同じセクター内で銘柄ペアを選び、共和分検定でふるいにかけられる。
- ・スプレッドの**半減期(halflife)**とは何で、なぜペアの品質指標になるのかを説明できる。

- ・ **Kalman フィルタ**がペア選定で果たす役割(固定比率の代わりに時間とともに変化するヘッジ比率)を説明できる。
- ・ `_RLConfig`の中心的なパラメータ(`z_entry`, `corr_min`, `hl_max`)の意味がわかる。

11.1 なぜセクターの中でペアを選ぶのか

Stat Arb(統計的裁定取引)は、2銘柄が似た外部要因の影響を受けると仮定する。同業種の2社は、金利の変化や原材料価格に対して似た反応を示す。

市場全体から無作為に2銘柄ずつペアを作ると、組み合わせは数千通りに膨れ上がる。統計検定を何千回も繰り返せば、実際の関係がなくても偶然 $p < 0.05$ を超えるペアが約5%混ざる。これを**多重検定問題**と呼ぶ。

セクター内だけで候補を絞ると、2つの効果がある。

- ・ 検定回数が減り、偽陽性も減る。
- ・ 同じセクターの企業は経済的につながっているため、「なぜ一緒に動くのか」を説明できる。

たとえば次のようなものだ。

- ・ XLK(技術セクター)内の AAPL-MSFT:半導体サプライチェーンの共有
- ・ XLF(金融セクター)内の JPM-BAC:金利変動への似た反応

共和分とは? 2つの時系列がそれぞれは不規則に動いても、その差が一定の範囲内に保たれる関係のことだ。差が一時的に広がっても再び縮まる性質を持つので、その瞬間に取引できる。

実装では全銘柄を一度に比較するのではなく、あらかじめ定義した `SECTOR_SYMBOLS` 辞書で11セクターに分け、同じセクター内だけでペアを作る。コード位置は `app/trading/strategy/signals/mean_reversion_signal.py` である。

11.2 ペア選定パイプライン

数百個の候補ペアを段階的にふるいにかける。各段階は異なる基準で不要な候補を取り除く。相関フィルタは動きが大きく異なるペアを捨て、共和分検定は長期的な関係の有無を確認し、半減期フィルタはスプレッドが実戦で使えるほど早く平均へ戻るかを見る。

Step	段階	基準 / 出力
1	全ユニバース	S&P 500
2	セクター別に分類	SECTOR_SYMBOLS
3	価格データの読み込み	3年分の日足
4	相関フィルタ	$\text{corr} \geq 0.4$
5	共和分検定	$p < 0.05$
6	OLS β の計算	静的ヘッジ比率
7	半減期の計算	$hl \leq 80$ 日
8	最終ペアリスト	(sym_a, sym_b, corr, pvalue, beta, halflife)

歴史的背景 D.E. Shaw、PDT Partners、ルネサンスは 1990 年代からペアトレーディングを大規模に運用してきた。1998 年の LTCM 事件は、複数のスプレッドが同時に拡大して発生した事件で、単一セクターのペア戦略がこうした同時損失リスクを分散するのに有利だという教訓を残した。

WARNING

p値だけを見てはいけない。 共和分検定は回帰の方向(A->B か B->A か)によってp値が変わる。実戦では半減期や残差の定常性といった補助指標も合わせて見る。より厳密なクロス検証 (Johansen 検定など) は第 19 章で扱う。

統計の素養がなくても大丈夫: この章で使う検定(共和分、ADF)の核心的な問いは1つだけだ—「この2銘柄の価格差は結局平均に戻るのか?」 $p < 0.05$ なら戻る、 $p > 0.05$ なら戻らない。数式がわからなくてもコードを実行して p 値だけ確認すれば使える。

共和分検定—1 ペアを判定する

statsmodelsのcoint()関数を使えば1行で済む。内部では(1)OLS 回帰で2つの価格の比率を求め、(2)その残差(=スプレッド)が平均回帰するかを ADF 検定で確認する。私たちが使うのは2番目の戻り値である p 値だけだ。

```
from statsmodels.tsa.stattools import coint

_, pvalue, _ = coint(price_a.values, price_b.values)

if pvalue >= self.coint_pvalue:    # default = 0.05
    continue
```

p 値が 0.05 未満なら通過、それ以外は除外する。

パイプライン全体—セクター別に全ペアを検定

経済的関係のあるpair候補で学習日付を揃え、欠測率・標本数・共和分・半減期を順に検査する。多数pairなら多重検定を補正する。例は選別順序で、選定後もOOS評価が必要だ。

```
import numpy as np
import statsmodels.api as sm
from statsmodels.stats.multitest import multipletests
from statsmodels.tsa.stattools import coint

def find_cointegrated_pairs(self, symbols) -> list:
    tested = []
    for sector, sector_symbols in SECTOR_SYMBOLS.items():
        prices = self._load_prices(sector_symbols, period="3y")
```

```

corr_matrix = prices.pct_change().dropna().corr()

for i, sym_a in enumerate(corr_matrix.columns):
    for sym_b in corr_matrix.columns[i+1:]:

        corr = corr_matrix.loc[sym_a, sym_b]
        if corr < self.min_correlation:
            continue

        pair = prices[[sym_a, sym_b]].dropna()
        price_a = pair[sym_a]
        price_b = pair[sym_b]

        _, pvalue, _ = coint(price_a.values,
price_b.values)
        X = sm.add_constant(price_b.to_numpy())
        fit = sm.OLS(price_a.to_numpy(), X).fit()
        intercept, beta = fit.params
        spread = price_a - (intercept + beta * price_b)
        tested.append({
            "symbols": (sym_a, sym_b), "corr": corr,
            "pvalue": pvalue, "beta": beta,
            "halflife": calc_halflife(spread.to_numpy()),
        })

    if not tested:
        return []

reject, qvalues, _, _ = multipletests(
    [x["pvalue"] for x in tested], alpha=self.coint_pvalue,
method="fdr_bh"
)
selected = []
for row, keep, qvalue in zip(tested, reject, qvalues):
    hl = row["halflife"]
    if keep and np.isfinite(hl) and hl <= self.max_halflife:
        selected.append({**row, "qvalue": qvalue})

```

```
return sorted(selected, key=lambda x: (x["qvalue"],
x["halfLife"]))
```

覚えておくべきは 3 つの continue 文だ。各ペアが通過すべき関門がちょうど 3 つあることを示している。出力は p 値の昇順なので、リストの先頭ほど強い共和分関係を持つペアになる。

11.3 半減期(halflife)の計算

半減期とは、スプレッドが平均値の方向へ半分だけ戻るのにかかる時間(日数)である。共和分関係があっても半減期が長すぎると実戦では使いにくい。半減期が 200 日なら、ポジションを数百日も拘束することになり、コストとリスクが大きくなる。

数学的には「スプレッドが平均から離れた分だけ、次の時点でどれだけ引き戻されるか」を回帰で推定する。この引き戻しの強さ(phi)から半減期が決まる。phi が負で大きいほど強く引き戻されるので、半減期は短くなる。

```
def calc_halfLife(spread, min_obs=20):
    y = np.diff(spread)                # Δs_t
    x = spread[:-1] - np.mean(spread)
    phi = np.sum(x * y) / np.sum(x * x)
    if phi >= 0:
        return float("inf")
    return float(max(-np.log(2) / phi, 1.0))
```

確認する出力は日単位の数値だ。たとえば phi = -0.03 なら半減期は約 23 日。つまり 23 日以内にスプレッドが平均まで半分戻る。phi が 0 以上なら平均回帰しないので、80.0(上限値)を返してペアから事実上除外する。結果は 1 日～80 日の範囲にクリップされる。

11.4 Kalman フィルタで動的ヘッジ比率を求める

OLS で一度求めたヘッジ比率 β は、時間が経っても変わらない。しかし、2 社の事業構造や市場環境が変化すれば、2 つの価格を結びつける比率も変わる。**Kalman フィルタ**は、毎時点で β を更新してくれる道具だ。新しい価格が入るたびに「これまでの β 推定値」と「今日見た新情報」を加重平均して β を更新する、と理解すればよい。

中心となる考え方は次のとおりだ。

- ・ **状態**: 現在のヘッジ比率 β と定数 c
- ・ **予測**: 毎日 β が少しずつ揺らぐと仮定する。
- ・ **更新**: 今日見た実際の価格と予測価格の差をもとに、 β をわずかに調整する。

時変回帰は $y_t = a_t + \beta_t x_t + \text{noise}$ 、状態 $[a_t, \beta_t]$ で捉える。Q が大きければ係数が速く動き、R が大きければ新価格へ反応しにくい。第6章の vector 更新を使い、hedge 量では β の符号・上限・不確実性も確認する。

- ・ `kalman_delta = 1e-4`: β の変化速度。小さいほど β は安定して動くが、変化に鈍くなる。
- ・ `kalman_obs_noise = 1.0`: 観測ノイズ。小さいほど新しい価格を信頼する。

毎日更新された β を使って、その日のスプレッドを作る。

```
spread = log_price_a - beta_arr * log_price_b
```

このスプレッド時系列が、次章(第12章)の取引信号の入力となる。

11.5 ペアの品質指標

ペアを一度選んでも、時間が経つにつれて共和分関係が崩れることがある。だからエントリー直前にもう一度ペアの品質を確認する。

確認するのは 2 点だ。

1. 直近 250 日の相関が `corr_min(=0.4)` 以上か
2. 直近 250 バーのスプレッドの半減期が `hl_max(=60 日)` 以下か

両方の検査を通過したらそのペアでエントリーし、保有期間の上限も半減期に応じて動的に決める。

```
if kdata["corr"] < cfg.corr_min:
    return False
if calc_half-life(recent_spreads) > cfg.hl_max:
    return False
dyn_max_hold = min(cfg.max_hold_days, int(cfg.hl_hold_mult *
hl))
```

`kdata` はあらかじめ計算した結果のキャッシュ (Kalman スプレッド、動的 β 、250 日ローリング相関) である。エントリーのたびに再計算する必要がなく、高速だ。

11.6 ペア選定パラメータを一望する

最初は意味だけざっと眺め、必要なときにこの表に戻ってくればよい。

パラメータ	既定値	役割
<code>z_window</code>	60	z スコア計算ウィンドウ (営業日)
<code>z_entry</code>	1.8	エントリーしきい値の z スコア
<code>z_exit</code>	0.3	決済しきい値の z スコア
<code>hard_stop_z</code>	4.5	ハードストップ z スコア
<code>kalman_delta</code>	1e-4	Kalman β の変化速度
<code>kalman_obs_noise</code>	1.0	Kalman 観測ノイズ
<code>max_hold_days</code>	30	最大保有期間 (日)
<code>corr_min</code>	0.4	エントリー時の最小相関
<code>warmup_days</code>	120	Kalman のウォームアップ期間

パラメータ	既定値	役割
hl_max	60	エントリー時の最大半減期(日)
hl_hold_mult	3.0	max_hold = 3 × halflife

$z_{\text{entry}} = 1.8$ はバックテストの探索で選ばれた値だ。ただし、ある期間で最適だった値が常に良い既定値になるとは限らない。実際には取引コスト、銘柄ユニバース、市場区間を変えても大きく揺れないかを併せて確認する必要がある。

11.7 ペア選別基準のまとめ

基準	値	役割
セクター制限	同一セクター内	多重検定問題の緩和
相関	≥ 0.4	共通要因の存在を確認
共和分 p値	< 0.05	スプレッドの平均回帰を検証
半減期	≤ 60 日(エントリー時)	保有期間の妥当性確保
Kalman δ	$1e-4$	安定した動的 β 推定

ペアを選ぶときは**数より品質**だ。共和分が強いペアが数十個あるほうが、基準を緩めて通過したペアが数百個あるよりも良い。

次章(第12章): こうして選んだペアを実際の取引信号に変換する。Kalman スプレッドの z スコアでエントリータイミングを掴み、レジームフィルタと RL ゲートで信号をもう一度ふるいにかける。

試してみようー合成ペアでパイプラインを動かす

ここまで見た 3 つ(相関、共和分、半減期)を合わせて 1 ペアを判定する最小実行コードだ。合成データで動かして、通過と失敗がどのように出るかの感覚をつかむためのものである。ぜひ自分で実行してみてほしい。

```

import numpy as np
from scipy import stats
from statsmodels.tsa.stattools import coint

MIN_CORR, MAX_PVALUE, MAX_HALFLIFE = 0.40, 0.05, 60

def estimate_half-life(spread):
    y = spread[1:] - spread[:-1]
    x = spread[:-1] - spread[:-1].mean()
    slope, *_ = stats.linregress(x, y)
    if slope >= 0:
        return float("inf")
    return float(-np.log(2) / slope)

def screen_pair(prices_a, prices_b):
    log_a, log_b = np.log(prices_a), np.log(prices_b)

    ret_corr = np.corrcoef(np.diff(log_a), np.diff(log_b))[0, 1]
    if ret_corr < MIN_CORR:
        return None

    slope, intercept, *_ = stats.linregress(log_b, log_a)
    spread = log_a - slope * log_b - intercept

    _, eg_pvalue, _ = coint(log_a, log_b)
    if eg_pvalue >= MAX_PVALUE:
        return None

    hl = estimate_half-life(spread)
    if hl <= 0 or hl > MAX_HALFLIFE:
        return None
    return {"return_corr": float(ret_corr),
            "eg_p": float(eg_pvalue),
            "hl": hl}

if __name__ == "__main__":
    rng = np.random.default_rng(42)
    common = np.cumsum(rng.normal(0, 1, 300))
    a = np.exp(common * 0.9 + rng.normal(0, 0.2, 300)) * 100

```

```
b = np.exp(common * 1.1 + rng.normal(0, 0.2, 300)) * 150
print(screen_pair(a, b))
```

出力は{"corr": ..., "adf_p": ..., "hl": ...}形式の辞書 1 つだ。Noneが返ったときはどの段階で落ちたかを上の条件で順に確認すればよい。実市場データに拡張したいなら、prices_aとprices_bを実際の価格系列に差し替えるだけで済む。

演習問題

1. SPY-QQQ ペアで共和分検定を回してみよう。p値がエントリーのしきい値(0.05)に近く出たとき、半減期や相関などの補助指標でどう判断するかを整理してみよう。
2. 上の estimate_half-life のコードを応用して、半減期が 3 日未満または 60 日超のペアを自動的に除外するフィルタを作ってみよう。あまりに短い半減期がなぜ危険かも考えてみよう。
3. ペア選定を四半期ごとに再実行する場合、選定されるペアが変わったら既存のポジションをどう手仕舞いするかを考えてみよう。入れ替えで発生する取引コストも併せて考慮する。

まとめ

この章で必ず覚えておきたいのは次の 3 点だ。

基準	理由
同じセクター	2 社がなぜ一緒に動くのかを説明できる必要がある
共和分 p 値 < 0.05	価格差が結局平均に戻るという統計的根拠
半減期の制限	あまりにゆっくり戻るペアは実戦では使えない

良いペアは、単に一緒に動く 2 銘柄ではない。同じセクターの中にあり、共和分の p 値が低く、そしてスプレッドが取引可能な速度で平均に戻る必要がある。**Kalman フィルタ**は、この比率が時間とともに変化する場合に追従してくれる道具で、内部の詳しい数式は次章と第19章でさらに扱う。

次章では、こうして選んだペアを実際のエントリー・決済信号に変換する。

さらに読むには

- ・ **論文**: Engle & Granger (1987), “Co-Integration and Error Correction”—共和分検定の原典(ノーベル賞)
- ・ **書籍**: Vidyamurthy, Pairs Trading (Wiley, 2004) —統計的裁定取引戦略の実戦的実装
- ・ **論文**: Gatev, Goetzmann & Rouwenhorst (2006), “Pairs Trading: Performance of a Relative Value Arbitrage Rule”—学術的バックテスト

理論的基盤を深める: この章で扱った共和分検定と Kalman ヘッジ比率の数学的背景は第19～21章で深く扱う。Johansen クロス検証、レジーム条件付きペアフィルタリングといった高度な話題もそちらで取り上げる。

第12章 統計的裁定取引 —シグナル生成

共和分ペアを見つけたなら、次の問いは単純だ。いつ入り、いつ抜け、関係が壊れたことをどう知るか。実装の本質はこの三つに尽きる。

最も素朴なルールはこうだ。スプレッドの Z スコア (乖離度合いを標準化した値。定義は第11章の共和分の節を参照) が 2.0 を超えたら入り、0 に戻ったら決済する。学び始めの段階では、これで十分に見える。だが実戦では足りない。市場は「なぜスプレッドが広がったか」を教えてくれないからだ。素朴な Z スコアは「一時的に広がった」と「構造が永久に変わった」のを区別できない。

失敗の仕方は明確だ。スプレッドは 2 から 4、そして 6 へとどこまでも広がりうる。一時的な過熱なら待てばよい。しかしペアの片方の業績が関係そのものを変えてしまったなら話は別だ。素朴な仕組みは新しい極値を「より良い建値」と勘違いしてポジションを増やす。ペアトレード版の難平(なんぴん)だ。

決済も同じだ。教科書は「スプレッドが 0 に戻ったら決済」と書く。だがこのルールは市場の状況、すなわちレジーム(regime、市場の局面。第11章で HMM により導入した

概念)を考慮していない。ボラティリティの高い区間ではスプレッドは0付近で大きく揺れ、きれいに交差してくれない。固定のルールは無駄な売買と手数料を増やすだけだ。

エントリーよりエグジットが先である

この章を読むときに、まず一つの原則を頭に刻んでほしい。**エントリールールが機会を生み出すが、損益を決めるのはエグジットルールである**。エントリーが外れても次の機会は来るが、エグジットが遅れば一度の取引が口座を削る。本書の節の順序は「エントリー → エグジット」だが、頭の中で組み立てるときは「どこで抜けるか」を先に決め、そのうえで「どこで入るか」を考えるべきだ。良いシグナルは多く当たるシグナルではなく、外れたときに損失が広がる前に退くシグナルである。

この章の解法はレジーム条件付きシグナルだ。HMM が分類した市場状態に合わせて、Z スコアの閾値とエグジットルールを調整する。安定した平均回帰レジームではエントリー基準を緩めて機会を多く取り、強いトレンドのレジームではエントリーを抑える。

このシグナルを「明日どの銘柄が上がるかを当てる装置」と捉えてはいけない。「今このペアに入っても安全か」「関係が壊れた場合、どこで止めるか」をコードで判断するためのルールの束だ。

第11章が「何を取引するか」を決めた章なら、第12章は**いつ入り、いつ出るか**を決める章である。つまり、ペア候補を実際の売買シグナルに変える章だ。

第11章の出力

候補ペア: K0 / PEP

|

v

第12章の入力

現在のスプレッド、Z-score、半減期、相関、出来高

|

v

シグナル生成

エントリー / 待機 / エグジット

|

v

第13～14章 RL

シグナルを見てポジションサイズを調整する

初心者はZ-scoreを「大きければ入る」と覚えると危険である。Z-scoreはシグナルの出発点にすぎない。この章は、その数字の周囲に安全フィルタを置く。

Z-scoreが大きい

- |
- +--- ペア関係はまだ維持されているか？
- +--- 回帰は始まっているか？
- +--- 出来高と流動性は十分か？
- +--- 長く持ちすぎているか？
- +--- 損失上限を超えていないか？

したがって第12章は、エントリーだけの章ではなく、**エントリーとエグジットを一緒に設計する章**である。RLと組み合わせるときも、このシグナルは重要なままだ。RLはどんなペアにも自由に入るのではなく、この章のフィルタを通過した状況でポジションサイズを調整する。

この章の流れ

- ・ エントリー資格(5段階フィルタ)
- ・ Entry Score(entry_score)—掛け算による点数式
- ・ エグジット条件(優先順位6段階)
- ・ BW-HMM スプレッドレジーム補正
- ・ エントリー・エグジットフローまとめ
- ・ コードで見る
- ・ RL との組み合わせ方(参考)

学習目標

この章を終えると、次のことができる。

- ・ z スコアを基準にしたエントリー・エグジットロジックの流れと、エグジット条件の優先順位を説明できる
- ・ entry_score を構成する6つの因子それぞれの役割を述べられる

- ・ BW-HMM レジーム補正がエントリー基準にどう影響するかを説明できる
- ・ なぜエグジット条件がエントリー条件と同じくらい、あるいはそれ以上に重要なかを説明できる

12.1 エントリー資格 (Entry Prerequisites)

エントリースコアを計算する前に、候補そのものをふるい落とす5つのフィルタがある。スコアがどれほど高くても、この5条件のうち一つでも通らなければエントリーしない。スコアは「どれだけ魅力的か」を測るもの、資格は「入ってよい候補か」を測るものだ。

段階	フィルタ	条件	直観
1	半減期 (half-life)	<code>hl <= 60日</code>	回帰が遅すぎるペアを除外
2	相関	<code>corr >= 0.4</code>	一緒に動かないペアを除外
3	Z スコア閾値	<code>\ z_now\ >= z_entry(BW-HMM 補正込み)</code>	十分に乖離したときだけエントリー
4	収束方向	<code>\ z_now\ < \ z_prev\ </code>	まだ広がっている最中ならエントリーしない
5	出来高ミクロ構造	<code>vol_mult >= 0.90</code>	流動性が異常に薄ければ除外

このうち **4番「収束方向」条件が肝だ**。スプレッドがまだ広がっている最中には入らず、ピークを越えて回帰が始まった時点でだけ入る。このひと言があるおかげで、「さらに広がった場所にまた入る」という難平を防げる。

3番の `z_entry` は固定値ではなく、BW-HMM レジームに応じて動的に調整される (12.4 で説明)。実装は `stat_pair_rl.py` の `_get_effective_z_entry()` を参照。

12.2 Entry Score(entry_score)

WARNING

Z スコア閾値(z_entry)を低くしすぎると取引頻度が増えすぎ、手数料が積み重なる。既定値は 2.0。ペアやレジームによって 1.5~2.5 の範囲で Optuna により最適化する(->第15章)。

12.1 の5つの資格をすべて通過した候補に対して、エントリースコアを計算する。スコアは6個の因子の積だ。これは「いくら儲かるか」ではなく「いま入っても安全か」を測る品質スコアである。

```
entry_score = abs(z_now) * (1.0 / max(hl, 1.0)) * corr *  
ivr_mult * conv_boost * vol_mult
```

各因子の意味は次のとおり。

因子	意味	範囲	重要度
$ z_{\text{now}} $	現在の z スコアの絶対値。乖離が大きいほど平均回帰の期待も大きい	0~5+	5/5
1/hl	半減期 (half-life) の逆数。短いほど回帰が速い	0~1	4/5
corr	直近250日相関。共和分の品質	0~1	3/5

因子	意味	範囲	重要度
ivr_mult	実現ボラティリティ比による補正。「いま市場は普段通り静かなのか、異常に揺れているのか」を短期/長期ボラティリティ比(IVR)で測ってスコアに掛ける。静かなら信頼してそのまま、揺れていればスコアを削る	0.5~1.0	3/5
conv_boost	収束モメンタムのボーナス	1.0~1.2	2/5
vol_mult	出来高ミクロ構造による裏付け	0.5~1.0	2/5

なぜ掛け算なのか

足し算で計算すると、Z スコアだけがとても大きいときに他の因子の弱点を覆い隠してしまう。掛け算は逆だ。どれか一つの条件が弱ければ全体のスコアも一緒に下がる。Z スコアがいくら大きくても、相関が低ければスコアは減る。

ペアトレードで最も危ない状況は、「スプレッドが大きく広がった」という理由だけで、すでに構造が壊れたペアに入ることだ。掛け算はそのようなエントリーを防ぐ防御装置である。Z スコアと半減期が中心シグナル、残り4つはリスクフィルタだと考えればよい。

数値の一例

ある仮想のペアが $z=-2.8$ 、 $hl=25$ 、 $corr=0.65$ 、 $ivr_mult=1.0$ 、 $conv_boost=1.1$ 、 $vol_mult=0.95$ だとする。

$$score = 2.8 \times (1/25) \times 0.65 \times 1.0 \times 1.1 \times 0.95 \approx 0.076$$

同じ z、同じ半減期で corr だけ 0.4 に落ちると、スコアはおおよそ 0.047 まで下がる。掛け算なので、ひとつの因子が弱まれば全体スコアも一緒に下がることが数値で確認できる。

収束モメンタム(conv_boost)

abs(z)が3足で2.8->2.5->2.1と下がれば均衡へ向かう動きだ。3値が有限で厳密減少なら1.2、そうでなければ1.0というscore倍率を例示できる。回帰完了の確率ではなく、参入条件・費用・上限を別途満たす必要がある。

12.3 エグジット条件(優先順位順)

IMPORTANT

エグジットルールがなければ、収束しないペアで損失が積み上がり続ける。max_holding_days(最大保有日数を超えたら強制決済)は実戦で必須の安全装置である。共和分関係が崩れると Z スコアは平均に戻らない。したがって時間ベースの損切りも併用しなければならない。

ポジションを閉じるとき、以下の6条件を **順番に** チェックする。前の条件が満たされたら、後ろの条件は確認しない。

優先順位	条件	パラメータ	役割
(1)	price_stop	price_stop_pct = -8.0%	絶対損失の上限
(2)	pair_degrade_hl	hl > 100	共和分崩壊の検知
(2)	pair_degrade_corr	corr < 0.3	相関崩壊
(3)	z_hard_stop	hard_stop_z = 4.5	極端な乖離での損失最小化

優先順位	条件	パラメータ	役割
(4)	rl_exit	rl_exit_threshold_hybrid = -0.6	RL アクションが極めて否定的なとき (RL モード限定)
(5)	z_converge	z_exit = 0.3	正常水準への収束
(6)	max_hold	min(30, 3×hl)	時間ベースの強制決済

各条件を一行で説明すると次のようになる。

- ・ **price_stop**: 損失が -8% を超えたら無条件に決済。理由を問う前にまず損失を止める。
- ・ **pair_degrade_hl / _corr**: 半減期が100日超、または相関が0.3未満 -> ペア自体が壊れたと判断。
- ・ **z_hard_stop**: Z スコアが4.5超 -> 極端に広がった状態。もう待たずに決済。
- ・ **z_converge**: Z スコアが0.3以下 -> 目標達成。正常な収束決済。
- ・ **max_hold**: 上のどれも当てはまらなくても、min(30日, 3×hl) が経過したら強制決済。

この順序がそのまま優先順位だ。損失遮断(price_stop)が一番先、時間強制(max_hold)が一番後。「正常な決済シグナルを待っている間に損失が -10%、-15% と膨らむ」事故を防ぐためだ。

どの条件が先に効くか—二つのシナリオ

シナリオ A —損切りが先に効くケース。ペア A に $z=-2.5$ で入った。次の足で片方の会社の決算発表ショックによりスプレッドがさらに広がり、損失が -9% に達した。Z スコアはまだ -3.2 で、決済基準 ($|z|<0.3$) とは程遠い。優先順位 (1) の price_stop が先に発動し、(5) の z_converge までは進まない。損失上限がなければ、損失がさらに深くなる前に抜けられなかっただろう。

シナリオ B —z 収束が先に効くケース。ペア B に $z=-2.7$ で入った。一週間かけて価格がゆっくり動き、損益は +1.5%、Z スコアは $-2.7 \rightarrow -1.8 \rightarrow -0.9 \rightarrow -0.2$ ときれいに収束した。(1) (2) (3) (4) はすべて満たされず、(5) の `z_converge(|z|<0.3)` で決済される。(6) の `max_hold(時間強制)` まで行く必要のない、正常な決済シナリオだ。

実際の実装(if-elif 連鎖)は `stat_pair_rl.py` を参照。

12.4 BW-HMM スプレッドレジーム補正

スプレッド自体に HMM を適用し、平均回帰(MR)レジームとトレンドレジームを区別する。同じ Z スコアでも、どちらのレジームかによって意味が違う。MR レジームの大きな Z は機会だが、トレンドレジームの大きな Z は関係が壊れる兆候かもしれない。

そこで MR レジームでは `z_entry` を下げてエントリーしやすくし、トレンドレジームでは上げてエントリーを抑える。

パラメータ	既定値	効果
<code>spread_bw_mr_boost</code>	0.15	MR レジーム: $z_entry \times 0.85$ (エントリーを緩和)
<code>spread_bw_trend_penalty</code>	0.25	Trend レジーム: $z_entry \times 1.25$ (エントリーを抑制)

たとえば `z_entry=2.0` は MR レジームでは 1.7 に下がり、トレンドレジームでは 2.5 に上がる。トレンドレジームではさらにエグジット閾値と保有期間も引き締められる(`spread_bw_trend_exit`、`spread_bw_trend_hold_mult`)が、この二つのパラメータの詳細は付録と `_RLConfig` の定義を参照。補正関数の本体は `_get_effective_z_entry()` だ。

12.5 エントリー・エグジットフローまとめ

足 (bar) が一本完成するたびに、以下の2段階が順に実行される。まず開いているポジションを閉じるか確認し、次に新しいポジション候補を選ぶ。

段階	詳細	条件 / 処理
EXIT(既存ポジションの決済)	1. price_stop	損益 -8%
EXIT	2. pair_degrade	hl>100 または corr<0.3
EXIT	3. z_hard_stop	z >4.5
EXIT	4. RL exit	rl_full / rl_hybrid、min_hold=5本以降
EXIT	5. z_converge	z <0.3 または z 符号反転
EXIT	6. max_hold	min(30本, 3×hl)
ENTRY(新規ポジションのエントリー)	資格5段階	hl<=60、corr>=0.4、 z >=z_entry(補正済み)、z 減少傾向、vol_mult>=0.90
ENTRY	スコア計算	$ z \times 1/hl \times corr \times ivr_mult \times conv_boost \times vol_mult$
ENTRY	最終選択	スコア降順 -> 上位 max_positions=30 件にエントリー

エグジットチェックは上から下へ適用する。手前の条件に引っかかれば即決済し、残りは見ない。エントリーは資格を全通過した候補をスコア順に並べ、上位だけを選ぶ。

12.6 コードで見る—Entry Score 計算とエントリー資格判定

ここまで見てきたエントリーロジックを、短い関数2つで動かしてみよう。実際の stat_pair_rl.py はもっと長いが、中核の意思決定はこの2つに収まっている。段階番号は 12.1 の資格表と同じだ。

```
import numpy as np

Z_ENTRY, HL_MAX, CORR_MIN, VOL_MIN = 2.0, 60, 0.40, 0.90

def compute_entry_score(z, hl, corr, ivr=1.0, conv=1.0,
vol=1.0):
    """abs(z) × 1/hl × corr × IVR × convergence_boost × volume
    (higher is a stronger candidate)"""
    return abs(z) * (1.0 / max(hl, 1.0)) * corr * ivr * conv *
vol

def check_entry(z, z_prev, hl, corr, vol_mult,
z_entry_adj=Z_ENTRY):
    """Five entry checks from 12.1; return (go, reason)."""

    if hl > HL_MAX:
        return False,
f"hl={hl}>{HL_MAX}"

    if corr < CORR_MIN:
        return False,
f"corr={corr:.2f}<{CORR_MIN}"

    if abs(z) < z_entry_adj:
        return False,
f"abs(z)<{z_entry_adj}"

    if abs(z) >= abs(z_prev):
        return False, "z still
rising"

    if vol_mult < VOL_MIN:
        return False,
f"vol_mult={vol_mult:.2f}<{VOL_MIN}"
```

```

    return True, "ok"

if __name__ == "__main__":
    cases = [
        ("entry accepted", dict(z=-2.8, z_prev=-3.1, hl=25,
                                corr=0.65, vol_mult=0.95)),
        ("half-life too long", dict(z=-2.5, z_prev=-2.9,
                                    hl=80, corr=0.60, vol_mult=0.95)),
        ("weak correlation", dict(z=-2.6, z_prev=-3.0, hl=30,
                                   corr=0.30, vol_mult=0.95)),
        ("Z below threshold", dict(z=-1.5, z_prev=-1.8,
                                    hl=30, corr=0.70, vol_mult=0.95)),
        ("Z increasing", dict(z=-3.0, z_prev=-2.5, hl=30,
                               corr=0.65, vol_mult=0.95)),
        ("insufficient volume", dict(z=-2.8, z_prev=-3.1, hl=25,
                                      corr=0.65, vol_mult=0.80)),
    ]
    for label, c in cases:
        go, reason = check_entry(**c)
        score = compute_entry_score(c["z"], c["hl"], c["corr"],
                                    vol=c["vol_mult"])
        if go else 0.0
        print(f"{label:<10} go={go!s:<5} score={score:.4f}
              reason={reason}")

```

確認したい出力: 「正常エントリー」だけが go=True で通過し、残り5件はそれぞれ異なる理由(reason)で却下されること。却下理由はどの段階で止められたかをそのまま示す。すべて通過してしまうなら、閾値が緩すぎるかフィルタが抜けている。

12.7 RL との組み合わせ方(次章の予告)

ここまでは統計ルールだけでエントリー・エグジットを決めるベースライン(rl_stat_only)を見てきた。実際の stat_pair_rl.py は強化学習(RL)と組み合わせる4つのモードもサポートする。モードを分けた理由は単純だ。統計ルールと RL を同時に変えると、どちらが性能に効いたか分からなくなるからだ。

モード	エントリー	エグジット	説明
rl_stat_only	統計(z スコア)	統計	純粋な Stat Arb ベースライン(この章で扱ったもの)
rl_veto	統計 -> RL の拒否権	統計	RL が危険なエントリーをブロック
rl_hybrid	統計 -> RL 調整	RL	統計エントリー + RL エグジット
rl_entry	RL	統計	RL エントリー + 統計エグジット
rl_full	RL	RL	完全 RL 制御

既定値は `rl_veto` だ。統計ルールがまずエントリー候補を選び、RL はその中で危険そうなものだけを拒否する。拒否閾値(`rl_veto_threshold = -0.9`)はあえて低めに設定し、RL が中途半端な理由では統計ベースラインを揺るがさないようにしてある。RL アクションの意味、状態ベクトルの構造、報酬関数といった詳細は第13章で扱う。

初めて実装するなら、まず `rl_stat_only` で基準性能を確認し、その後で `rl_veto` を有効にして比較するとよい。そうしてはじめて変化の原因が分かる。

この章で先送りしたこと: RL veto 閾値のチューニング、28次元の RL 状態ベクトル構造、報酬設計は第13章(RL 環境設計)で続けて扱う。

まとめ

この章で必ず覚えておきたいことは3つ。

- 1. エントリー条件よりエグジット条件のほうが重要。** 良いシグナルは多く当たるシグナルではなく、外れたときに素早く退くシグナルだ。エグジット6段階の優先順位(損失遮断 -> ペア崩壊 -> 極端乖離 -> RL -> 正常収束 -> 時間)を頭の中で描けるようになっておく。

- 2. **entry_score は掛け算。** 6つの因子のうち1つでも弱ければスコアが下がる。すでに壊れたペアに Z だけ見て入るのを防ぐ防御装置だ。
- 3. **レジームによって同じ Z の意味が変わる。** 平均回帰レジームの大きな Z は機会、トレンドレジームの大きな Z は危険のサイン。BW-HMM 補正はこの違いをエントリー基準に反映する。

以下は中核パラメータの一覧。

構成要素	パラメータ	役割
z_entry	2.0(sweep 時は 1.5~2.5)	MR エントリートリガー
z_exit	0.3	収束決済
hard_stop_z	4.5	極端損失の遮断
price_stop_pct	-8%	絶対損失上限
BW-HMM 補正	mr_boost=0.15, trend_penalty=0.25	スプレッドレジーム別の z_entry 調整
conv_boost	最大 1.2	すでに回帰中のペアを優遇
max_hold	min(30, 3×hl)	適応的な保有期間

次章(第13章): RL 環境設計。StatPairREnv の状態空間(28次元)、行動空間(連続)、報酬関数の設計、そして rl_veto 閾値の意味を扱う。

練習問題

- 1. z_entry 閾値を 1.5 から 2.5 まで 0.25 刻みでグリッドサーチせよ。設定ごとの成績、年間取引回数、平均利益/損失を表にまとめよ。
- 2. conv_boost(収束モメンタム補正)を 0 と最大値にそれぞれ固定したうえで、1年の PnL 曲線を比較せよ。このパラメータの効果が最も際立つ市場局面はどこか。
- 3. 2019~2023年のバックテストログから、6つのエグジット条件のうち最も頻繁に発動した条件を確認せよ。条件別の発動比率を出力するコードを追加せよ。

さらに読むには

- ・ 論文: Avellaneda & Lee (2010), “Statistical Arbitrage in the U.S. Equities Market”—Z スコアに基づくシグナル設計
- ・ 書籍: Chan, Quantitative Trading —統計的裁定取引の実装とリスク管理
- ・ 論文: “Momentum and Mean Reversion in Strategic Asset Allocation”—収束
モメンタム補正の理論的背景